基于信息集分层的微网电力调控策略：因果预测、风险定价与滚动优化

摘要

本文以**信息集边界**为主线，将四个微网调控问题统一到母线能量平衡、储能状态递推、容量 / 功率边界与弃光约束构成的公共模型中；各问仅改变可用信息与结算规则。每次滚动决策都继承上一执行块的实测储电量，未启用节点则延续最近有效计划，从而严格保持时序一致性。

问题一为确定性日前调度。以全天购电费最小为目标、首末储电量均为 6000 kWh 为闭环约束建立线性规划，并用“费用—吞吐量—弃光量”三层词典序目标消除退化最优解。最优全天购电量 59482.70 kWh，购电费 35126.95 元，较无储能基准的 48052.05 元节省 12925.10 元（26.8981%）。

问题二在题目未提供当日预报时，主模型采用只依赖历史观测的负载—光伏因果预测和逐日确定性等价优化。5 倍紧急电价在孤立单时段下对应 4 : 1 非对称损失及光伏输出 0.2 分位启发；全年模型通过历史窗口上的非对称费用代理在多个残差分位候选中在线选择，并用“先撤充电、再增放电、最后紧急购电”的实时反馈执行。2 月至 12 月总费用为 14773804.78 元，紧急购电 348036.96 kWh。另设“日前负荷完全已知”的理想信息基准，主模型费用相较该基准高 1497816.65 元（10.14%）。

问题三按题意采用“计划费 + 调整费 + 紧急费”的叠加结算，并用 0–1 互斥约束禁止同时充放电。仅 0:00 决策时费用为 15731164.40 元，依次加入 6:00、12:00、18:00 更新后降至 14510735.12、14213145.74、13917403.19 元，三节点均有正收益，累计节省 1813761.21 元。配对控制把其中 1225632.49 元归因于负载观测刷新、588128.71 元归因于新光伏预报，避免把综合节点收益误称为纯光伏价值。

问题四按题面直接代入附件 4 电价重算，Q4-2 与 Q4-3 主结果分别为 15523884.83 元和 14589632.38 元，滚动更新节省 934252.45 元。另设因果价格扩展，费用分别为 15590096.88 元和 14722175.07 元；两种价格信息情形下滚动更新均有效。

全部指定日期表均由最终结果工作簿自动生成。物理残差、信息泄漏 mutation、结算分解、计划持续性与回归测试全部通过；主方案同时充放电量与费用分解回代差均低于数值容差（1e-8）。

关键词： 微网经济调度；因果预测；滚动优化；混合整数规划；信息集

一、 问题重述

1.1 问题背景

研究对象是一个经公共连接点接入外网的小区级能源系统。光伏出力具有间歇性，居民用电具有日周期，电池则把不同时刻的能量联系起来；三者共同决定每个 10 min 时段应从外网买入多少电。调度既要逐时满足用电需求，又要利用电池在低价或光伏富余时吸收能量、在缺电或高价时释放能量，因而本质上是带状态递推、预测误差和偏差结算的动态优化问题。

本问题的物理对象由三部分组成：光伏发电单元、储能设备与小区负载，并通过公共连接点与外网相连。附录 1 给定储能参数：最大容量 12000 kWh，最大充放电功率 5000 kW，2025 年 1 月 1 日 0:00 储电量为 6000 kWh，运行期储电量必须保持在 1200 ∼ 10800 kWh 之间，充放电效率均为 90%。附件数据的时间分辨率均为 10 min，即一天可分为 144 个调度时段。

问题的核心困难在于**信息的可获得性**：微网必须在决策时刻仅依据当时已知的信息制定策略，而负载、光伏与实际电价在一天内是逐步揭晓的。因此四个问题的差异并不在物理约束，而在于决策时刻能看到什么信息，以及计划与实际不一致时如何结算。

1.2 问题提出

问题一：在典型日电价、负载和光伏预测均给定时，制定 0:00 日前计划，并满足首末储电量相同。论文须按题面表 1、表 2 给出六个指定时段、全天购电量与费用，以及六个 4 h 分块的充放电量；完整结果写入 `result1.xlsx`。

问题二：在负载和光伏逐日变化、计划缺口按 5 倍电价紧急补购时，制定每日 0:00 计划。论文须对 2025-03-20、06-21、09-23、12-21 按表 1–3 报告购电、储能和紧急购电结果；完整结果写入 `result2.xlsx`。

问题三：每日四个发布时刻可获得未来 24 h 光伏预报，并允许调整 0:00 基准购电量。需要写清计划费、上调费、下调违约费与紧急购电费，判断新增预报是否值得采用；同样对四个指定日期给出表 1–3，并写入 `result3.xlsx`。

问题四：将附件 4 的逐时电价代入问题二、三重新求解，给出相应年度指标与四个指定日期结果，分别写入 `result4-2.xlsx` 和 `result4-3.xlsx`。

1.3 研究现状综述

微网经济调度问题在数学上通常被建模为以购电费或综合运行成本最小为目标的线性规划或混合整数规划，外网购电、分布式电源消纳与储能充放电被统一纳入同一组能量平衡约束中 [1, 2]。确定性模型适用于负载、电价与光伏均已知的日前场景，其优点是约束结构清晰、求解稳定可靠。

当光伏出力存在预测误差时，主流处理方式有三类：一是**情景法 / 两阶段随机规划**，用有限个历史或生成情景刻画不确定性，第一阶段确定与情景无关的日前计划，第二阶段按情景确定执行量 [3, 4]；二是**鲁棒优化**，在最坏情景下保证可行性，代价是结果偏保守 [5]；三是**滚动时域优化（模型预测控制）**，在新信息到达后仅重优化剩余时域并只执行近期决策，天然适配“预报分批发布”的决策结构 [6, 7]。含光伏与储能的日前调度研究普遍指出，储能的价值高度依赖预测精度与调度时域长度 [8]，而储能的运行成本又与其循环深度密切相关 [9]。

针对“预测值应偏向哪一侧”的问题，报童模型给出了经典答案：当高估与低估的边际损失不对称时，最优的点预测不是条件均值，而是某个**分位数** [10]。这一结论在电力系统中已被用于发电商的投标策略与偏差电量定价 [11]。在预测方法层面，基于数值天气预报的机器学习模型是目前光伏功率预测的主流 [12]；在市场机制层面，实时电价与需求响应的耦合会显著改变用户的购电行为 [13]。

现有研究多聚焦于算法本身，而对**信息集边界**与**结算口径**的刻画往往不够严格：在业务未提供全天计划负荷时，日前优化若直接使用当天真实负载，会混入理想信息优势；若已提供，则该情形可作为相应的信息口径。若比较策略时同时改变预测方法、约束和费用口径，差异也无法归因。本文据此把信息集边界作为建模原则，并用单因素消融与配对控制支持归因；求解则利用线性 / 混合整数线性结构，不依赖随机性较强的元启发式算法。

二、 问题分析

2.1 总体思路

四个问题共享同一套物理对象与同一组物理约束，差别只在两处：**决策时刻能获得哪些信息**，以及**计划与实际不一致时如何结算**。因此本文采用“一个物理内核+四条信息集主线”的建模框架：先把母线能量平衡、储能状态转移、容量与功率限值、弃光上界写成公共约束组，再对每一问替换相应费用项、扩展决策变量，并重新界定信息集。这样既保持四问口径一致，也能把结果差异归因到信息更新与结算规则本身。

具体地，问题 1 是完全信息下的确定性经济调度，用于验证物理约束与求解器；问题 2 在**完全无法获得当天预报**的极端信息集下，仅用历史观测构造因果预测，并把 5 倍紧急电价转化为非对称损失；问题 3 引入官方多时刻预报，形成“0:00 基准计划 $\rightarrow$ 每 6 h 滚动调整 $\rightarrow$ 实际执行 $\rightarrow$ 状态递推”的闭环；问题 4 只把电价序列换成逐日逐时的波动电价，用于检验前两类策略在价格波动下的稳健性。

整体流程为“数据对齐与单位换算 $\rightarrow$ 公共物理模型 $\rightarrow$ 因果预测与滚动重优化 $\rightarrow$ 主口径求解 $\rightarrow$ 消融、回代与提交文件核验”，各环节共享同一数据索引和费用定义。

2.2 具体分析

问题一：题目给定附件 1 的电价、负载与光伏发电预测，且“每天的电价和小区负载相同”，因此属于确定性优化。富余光伏必须经储能充电或弃光消纳，故公共模型保留弃光变量；本问的最优解实际实现零弃光。题目还要求 0:00 与 24:00 储电量相同，该闭环条件必须作为硬约束显式写入。

问题二：题目未提供任何当天预报，本文主模型按最严格信息边界，仅依赖已经发生的历史观测构造因果预测。若业务另行提供日前全天负荷，则可采用负荷完全已知的口径；本文将其作为理想信息基准，用于量化两种信息条件的费用差异。5 倍电价使“高估光伏”的局部边际损失是“低估光伏”的 4 倍，据此可在单时段报童损失下给出保守分位点；全年调度则采用因果点预测的确定性等价模型。计划在 0:00 锁定后，实际偏差由储能进行 10 min 级实时缓冲。

问题三：本问引入 6:00、12:00、18:00 三个新增预报时刻，题目据此询问“是否需要引入”。本文把“允许更新决策的节点集合”作为消融变量，构造 S_0（仅 0:00）$\to S_1$（+6:00）$\to S_2$（+12:00）$\to S_3$（+18:00）四组对照；节点未启用时，最近一次有效计划持续执行，绝不回退到 0:00 基准计划。由于启用节点会同时获得已实现负载观测与新发布光伏预报，主消融量度的是**整个决策更新节点的价值**；再用配对控制实验分别识别负载刷新与光伏预报刷新的贡献。题目将计划费、调整费与紧急购电费并列相加，故叠加结算作为主口径，差额结算仅作敏感性对照，两者分别重新优化。

问题四：本问按题面字面要求，直接把附件 4 给定的逐日实时电价代入问题二、三重新计算，并将这一口径作为主答案与结果工作簿来源。附件 4 的电价在日内与日间均有明显波动（最低 0.0076 元/kWh，最高 1.7936 元/kWh），峰谷价差扩大了储能套利空间。考虑到实际市场中完整日内价格的发布时间可能受交易规则限制，本文另设只使用已发生价格的因果预测情形作为现实性扩展；二者之差用于说明价格预知信息的价值，但不替代题面主结果。

2.3 求解框架

问题一、二及差额结算对照为线性规划；问题三和问题四的叠加结算主模型加入充放电互斥的 0–1 变量，形成混合整数线性规划。二者均由 HiGHS 求解。连续模型再以“真实费用—储能吞吐量—弃光量”词典序目标排除无经济意义的退化最优解；叠加模型由互斥约束直接禁止同一时段充放电。所有电量按 10 min 时段能量（kWh）核算，功率与能量换算系数为 $\Delta t = 1/6$ h。

三、 模型假设

为突出购电、光伏与储能之间的主要矛盾，并保证四问的信息边界严格一致，本文作如下假设。每条假设都将在相应模型中显式使用。

1. **调度周期与时间尺度：**以 10 min 为一个调度时段，一天划分为 $T = 144$ 个时段，单时段长度 $\Delta t = 1/6$ h。附件中标注 0:00+1 的列即当天 24:00。
2. **不允许向外部电网售电：**题目仅给出微网从外网购电的机制，未给出反向售电的交易规则，因此外网购电量恒为非负，光伏富余只能用于储能充电或弃光。
3. **储能建模的简化：**储能充放电效率在全年内恒为 $\eta_c = \eta_d = 0.9$（口径敏感性见 7.5.2 节），不计自放电、容量衰减、循环寿命成本与启停损耗；叠加结算主模型用 0–1 变量严格禁止同一时段充放电，其余连续模型由词典序目标排除退化解。
4. **储能功率约束按能量折算：**最大充放电功率 5000 kW 对应单时段最大充放电量 $5000\Delta t = 833.33$ kWh。
5. **初始储电量：**2025 年 1 月 1 日 0:00 储电量为 6000 kWh；1 月 31 天作为储能状态与预测误差的预热期，正式结果自 2 月 1 日起输出，这既与官方结果模板的日期范围一致，也避免冷启动误差污染统计。
6. **负载预测口径：**题目只为光伏提供了专门预报数据，未给出负载预报。因此本文用历史实际负载构造因果预测，方法族为持续法、7/14/30 日滚动均值、14/30 日线性加权均值与同星期均值，并在历史执行块上滚动在线选型，选型标准为平均绝对误差。
7. **附件 1 作为冷启动先验：**1 月 1 日之前无任何历史观测，此时以附件 1 给出的典型日负载与光伏预测曲线作为先验基预测。该先验仅在历史观测为空的当天使用，一旦有了至少一天观测便自动失效。
8. **实时储能反馈：**计划锁定后，日内允许储能依据当前时刻已经发生的真实光伏与当前储电量做 10 min 级调整，调整顺序为“先撤充电、再增放电、最后紧急购电”，不得借助任何未来信息。
9. **终端储电量策略：**题目仅对问题一明确要求 $S_{0:00} = S_{24:00}$。本文主模型在问题二、三的计划层同样采用“计划终端储电量等于当日实际初始储电量”的日循环口径（记为 daily_cycle），以保证储能长期可持续运行；同时给出不做终端约束（free）的敏感性结果，二者的差异在 7.5.1 节报告。
10. **问题四的价格口径：**题面主结果直接使用附件 4 给定的电价数据；因果预测仅作为交易信息受限时的扩展情形，其预测只使用决策时刻之前已经发生的价格，跨日部分由历史同刻价格外推。

四、 符号说明

本文使用的主要符号及其含义如表 1 所示。表中未列出的局部符号将在正文首次出现处说明。

表 1　主要符号说明

符号	单位	含义
d, t	—	日期下标与日内 10 min 时段下标（$t = 1, \ldots, 144$）
Δt	h	单个调度时段长度，$\Delta t = 1/6$
$L_{d,t}$	kW	小区负载功率
$P_{d,t}$	kW	光伏实际发电功率
$\hat{P}_{d,t}$	kW	决策时刻可用的光伏预测功率
$\pi_{d,t}$	元/kWh	外网正常购电单价
$G^{p}_{d,t}$	kWh	0:00 制定的计划购电量（第一阶段变量）
$G^{a}_{d,t}$	kWh	执行前最后一次有效调整后的购电量（第二阶段变量）
$C_{d,t}$	kWh	母线侧进入储能的充电量
$H_{d,t}$	kWh	储能送往母线的放电量
$S_{d,t}$	kWh	第 t 时段结束时储能内部储电量（状态变量）
$W_{d,t}$	kWh	未被利用的弃光电量
$E_{d,t}$	kWh	供电不足时的紧急购电量
$u_{d,t}$, $v_{d,t}$	kWh	调整购电量相对计划的上调量 $(G^a - G^p)^+$ 与下调量 $(G^p - G^a)^+$
$z_{d,t}$	—	充放电互斥变量；$z = 1$ 允许充电，$z = 0$ 允许放电
η_c, η_d	—	充电效率与放电效率，均取 0.9
$S_{\min}$, $S_{\max}$	kWh	储电量上下限，1200 与 10800
$\bar{E}$	kWh	单时段充放电量上限，$\bar{E} = 5000\Delta t = 833.33$
$\mathcal{I}_{d,\tau}$	—	第 d 天决策节点 τ 已经获得的信息集

五、 数据预处理

附件 1 为某典型日的电价、小区负载与光伏发电预测，共 144 个时段；附件 2 给出 2025 年 1 月 1 日至 12 月 31 日每天 144 个时段的小区负载与光伏实际功率；附件 3 给出每天 0:00、6:00、12:00、18:00 四次发布的未来 24 小时整点光伏预报，共 $365 \times 4 \times 24$ 个数值；附件 4 给出 365×144 的实时电价。所有附件的时间标签均按“列 j 对应时刻 $j \times 10$ min”的约定统一，不作人为平移。

原始功率单位为 kW，进入能量平衡前统一乘以 $\Delta t = 1/6$ h 换算为每时段电量（kWh）；电价单位为元/kWh；弃光、紧急购电、充放电均以 kWh 表示。逐项检查表明：各附件所需区域均为有限数值，负载与光伏非负、电价为正，附件 2 与附件 3 的日期完全对齐，附件 3 每行严格按 $0{:}00, 6{:}00, 12{:}00, 18{:}00$ 排列。全年负载电量 40.52×10^6 kWh，光伏电量 20.25×10^6 kWh，光伏高于负载的时段占 19.27%，其中 241 个时段的光伏富余功率超过 5000 kW，因此公共模型必须允许弃光。附件 1 电价的峰谷比为 3.76，而附件 4 的实时电价在 0.0076 ～ 1.7936 元/kWh 之间波动，两者的日平均电价均为 0.7662 元/kWh，使固定电价与波动电价的结果具有可比基础。

图 1 给出了数据总览。由图 1(a) 可见，典型日的光伏出力集中在 6:00 ～ 18:00，而电价的高峰出现在上午与傍晚两个时段，二者并不重合，这正是储能可以创造价值的空间；图 1(b)(c) 显示负载与光伏都存在明显的季节性：夏季负载与光伏同时走高，冬季负载抬升而光伏显著衰减；图 1(d) 表明实时电价在日内呈现规律的峰谷结构，但日间水平差异较大。

图 2 进一步量化了光伏与负载的匹配程度。净负荷（负载减光伏）在春、秋两季的中午时段大量转为负值，说明存在系统性富余；夏季光伏渗透率最高，6 ～ 8 月的中位数接近 1，即光伏出力与负载相当；冬季净负荷整体抬升，储能的价值主要体现在峰谷套利而非消纳光伏。

相关性复算表明，净负荷与负载、光伏的相关系数分别为 -0.1140、-0.8796，实时电价与日内相位的相关系数为 0.3912。净负荷受光伏变化的影响更强；附件 4 的价格同时具有日内结构与日际波动，不能用附件 1 的典型日电价代替，故问题四必须独立重算。

六、 模型的建立与求解

6.1 公共调度模型

四个问题共用同一套母线侧能量流结构。本节先给出公共决策变量、目标函数与约束，后续各问只替换信息集与费用项；加入充放电互斥变量时，模型由线性规划变为混合整数线性规划。

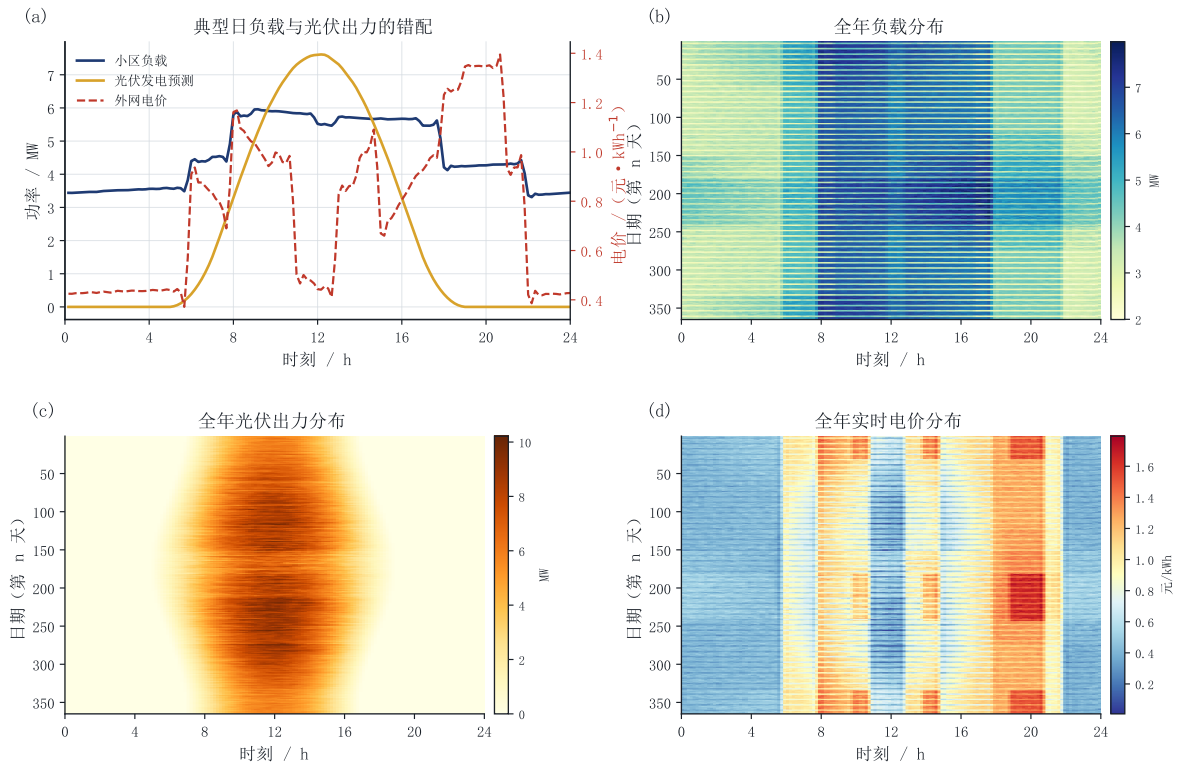

图 1　附件数据总览

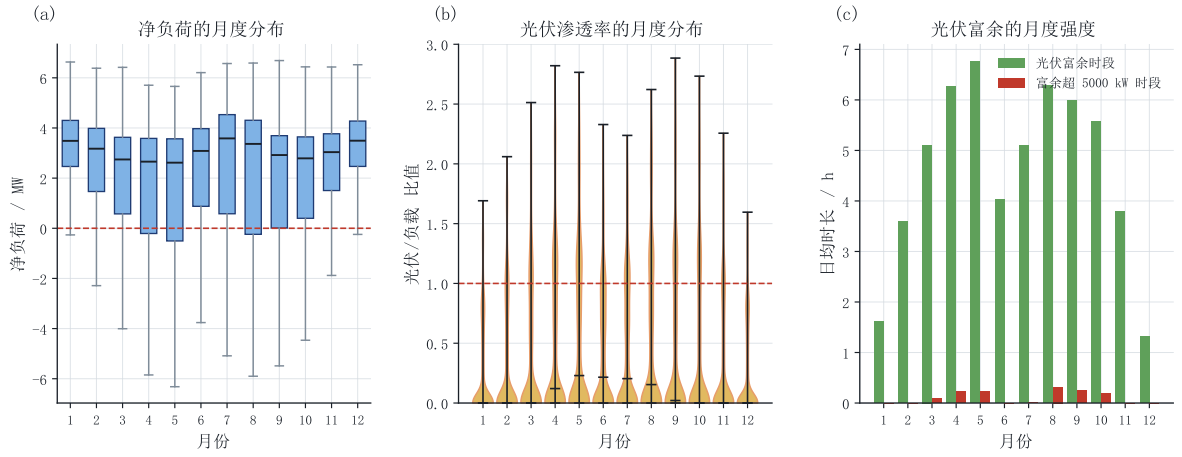

图 2　净负荷与光伏渗透的季节特征

6.1.1 决策变量

以第 d 天第 t 个 10 min 时段为基本决策单元，定义非负决策变量：

- 计划购电量 $G^{p}_{d,t} \geq 0$：0:00 制定的、全天固定不再改动的外网购电量（kWh）；
- 调整购电量 $G^{a}_{d,t} \geq 0$：执行前最后一次有效调整后的外网购电量（kWh）；问题一中 $G^{a} \equiv G^{p}$；
- 充电量 $C_{d,t} \geq 0$ 与放电量 $H_{d,t} \geq 0$：母线侧进入 / 流出储能的电量（kWh）；

- 弃光电量 $W_{d,t} \geq 0$：光伏中未被负载或储能吸收的部分（kWh）；
- 紧急购电量 $E_{d,t} \geq 0$：实际执行时供电不足的补充量（kWh）；
- 储电量 $S_{d,t}$：状态变量，第 t 时段结束时储能内部电量（kWh）。

需要特别强调的是 G^p 与 G^a 的角色差异：G^p 是在**不确定性揭晓之前**确定的与场景无关的变量；G^a、C、H、W、E、S 则是与真实场景相关的执行变量。这一“计划—执行”两阶段结构是问题二与问题三的建模基础。

6.1.2 目标函数

按题面“计划购电费用、调整相关费用和紧急购电费用相加”的字面口径，主模型的费用写为

$$\min\ F = \underbrace{\sum_{d,t} \pi_{d,t} G^{p}_{d,t}}_{\text{计划购电费}} + \underbrace{\sum_{d,t} \pi_{d,t}(1.5u_{d,t} + 0.5v_{d,t})}_{\text{调整相关费用}} + \underbrace{\sum_{d,t} 5\pi_{d,t} E_{d,t}}_{\text{紧急购电费}}, \tag{1}$$

其中 $u = (G^a - G^p)^+$、$v = (G^p - G^a)^+$。问题一、二没有调整交易，令 $u = v = 0$；问题三、四按式 (1) 作为叠加结算主口径。因子 5 来自题目“紧急购电电价是交易时刻电价的 5 倍”。差额结算只在 6.4.2 节作为口径敏感性对照。

6.1.3 约束条件

约束 1（母线能量平衡）。这是全模型的核心守恒关系：任一 10 min 时段内，微网的电力来源必须等于电力消耗。

$$\underbrace{G_{d,t} + P_{d,t}\Delta t + H_{d,t} + E_{d,t}}_{\text{电力来源}} = \underbrace{L_{d,t}\Delta t + C_{d,t} + W_{d,t}}_{\text{电力消耗}}, \qquad \forall d, t. \tag{2}$$

式中 $P_{d,t}\Delta t$ 为该时段可用的光伏电量，$W_{d,t}$ 为被主动舍弃的光伏电量（弃光）。该式之所以必须写成等式而非“供电量不低于负载”，是因为光伏富余时必须有出口，否则模型将不可行。

约束 2（储电量状态转移方程）。储能是唯一跨时段耦合的元件，其状态递推本模型最关键的约束：

$$S_{d,t} = S_{d,t-1} + \eta_c\, C_{d,t} - \frac{H_{d,t}}{\eta_d}, \qquad S_{d,0} = S_d^{\text{init}}, \qquad \forall d,\ t = 1, \ldots, 144. \tag{3}$$

式 (3) 表明：本时段末储电量等于上一时段末储电量，加上充电量经效率折算后真正进入电池的部分，再减去为送出放电量而从电池内部取出的部分。$S_{d,t}$ 是跨时段状态变量；每个预报节点都以当时实际测得的储电量为初值，所以后续方案随实际执行状态递推更新。

约束 3（储电量安全区间）。为避免过充过放，储电量必须始终保持在安全区间内：

$$1200 \;\le\; S_{d,t} \;\le\; 10800, \qquad \forall d, t. \tag{4}$$

约束 4（充放电功率上限）。最大充放电功率为 5000 kW，折算到单时段电量为

$$0 \;\le\; C_{d,t} \;\le\; \bar{E}, \qquad 0 \;\le\; H_{d,t} \;\le\; \bar{E}, \qquad \bar{E} = 5000\Delta t = 833.33\,\text{kWh}. \tag{5}$$

约束 5（购电非负与不售电）。题目未给出微网向外网售电的机制，因此

$$G^{p}_{d,t} \ge 0, \qquad G^{a}_{d,t} \ge 0, \qquad E_{d,t} \ge 0. \tag{6}$$

约束 6（弃光上界）。弃光不能超过该时段实际可用的光伏电量：

$$0 \;\le\; W_{d,t} \;\le\; P_{d,t}\Delta t. \tag{7}$$

约束 7（问题一首末储电量相等）。题目对问题一明确要求 0:00 与 24:00 的储电量相同：

$$S_{d,144} = S_{d,0} = 6000\,\text{kWh}. \tag{8}$$

问题二、问题三的计划层采用“计划终端储电量等于当日实际初始储电量”的日循环口径（daily_cycle），其敏感性见 7.5.1 节。

约束 8（信息集与非预见性）。设 $\tau \in \{0, 6, 12, 18\}$ 为第 d 天的计划节点，$\mathcal{I}_{d,\tau}$ 为该节点已经获得的数据。节点 τ 生成的计划变量 $X^{(\tau)}_{d,t} = (G^{a}, C, H, W, S)^{(\tau)}_{d,t}$ 必须关于该信息集可测：

$$X^{(\tau)}_{d,t} \in \sigma(\mathcal{I}_{d,\tau}), \qquad t \ge 6\tau + 1. \tag{9}$$

等价地，若两个数据场景在 $\mathcal{I}_{d,\tau}$ 上历史完全相同，则其节点 τ 生成的计划必须相同。各节点的信息边界与执行区间见表 2；当前执行时段的真实功率只进入实时反馈，不得进入节点计划器。

表 2　滚动节点的信息集与执行区间

节点	可使用的信息 $\mathcal{I}_{d,\tau}$	当次执行区间
0:00	前一日及更早实际负载、光伏、价格；0:00 发布的光伏预报；当日初始储电量	0:00–6:00
6:00	加入 0:00–6:00 已实现观测、6:00 光伏预报及实测储电量	6:00–12:00
12:00	加入 6:00–12:00 已实现观测、12:00 光伏预报及实测储电量	12:00–18:00
18:00	加入 12:00–18:00 已实现观测、18:00 光伏预报及实测储电量	18:00–24:00

该约束不是普通线性不等式，而由程序的数据切片和预测器接口保证，并用 mutation 测试验证（见七、节）。

6.1.4 模型类型与求解

公共模型由目标式 (1)、母线平衡式 (2)、状态递推式 (3)、边界式 (4)–(7) 和非预见性条件 (9) 组成。问题一、二和差额结算对照均为线性规划；对叠加结算主模型，另引入互斥变量 $z_{d,t} \in \{0,1\}$：

$$0 \le C_{d,t} \le \bar{E} z_{d,t}, \qquad 0 \le H_{d,t} \le \bar{E}(1 - z_{d,t}), \tag{10}$$

从而严格保证 $C_{d,t} H_{d,t} = 0$，得到混合整数线性规划。连续模型采用“真实费用—储能吞吐量—弃光量”三层词典序目标；混合整数主模型直接最小化真实费用。全部模型均由 HiGHS 求解，并在输出后逐时回代验证。

6.2 问题一：确定性日前调度

6.2.1 模型建立

问题一中附件 1 的电价、负载与光伏预测均视为已知，$G^a \equiv G^p$，$E \equiv 0$，且不存在调整量（$u = v = 0$）。目标函数退化为

$$\min \; F_1 = \sum_{t=1}^{144} \pi_t \, G_t, \tag{11}$$

约束为式 (2)∼(7) 加上首末储电量约束式 (8)。式 (11)∼(8) 构成问题一的确定性线性规划模型。

6.2.2 求解结果

求解得到全天计划购电量 59482.70 kWh，购电费 35126.95 元。储能充电 20740.67 kWh、放电 16799.94 kWh，首末均为 6000 kWh，且无弃光。禁用储能时费用为 48052.05 元，故净节省 12925.10 元（26.8981%）。

表 3 按题面格式给出问题一结果；完整 144 时段明细见支撑材料 `result1.xlsx`。

由图 3 可见，储能呈现清晰的“低价充电、高价放电”策略：在 0:00 ∼ 4:00 的低价时段和中午光伏富余时段充电，在 4:00 ∼ 8:00、10:00 ∼ 12:00 及 18:00 ∼ 20:00 的较高电价时段释放电量；储电量始终处于 1200 ∼ 10800 kWh 的安全区间。表 4 也显示 4:00 ∼ 8:00 分块以放电为主，与最优明细一致。

表 3　问题一指定时段购电量及全天购电量与购电费

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.00	12:00–12:10	486.40	14:00–14:10	0.00
16:00–16:10	394.93	18:00–18:10	636.98	20:00–20:10	0.00
全天购电量/kWh	59482.70	全天购电费/元	35126.95		

表 4　问题一储能分时段充放电量与首末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00–4:00	4500.00	0.00	12:00–16:00	5286.04	91.10
4:00–8:00	833.33	6365.84	16:00–20:00	0.00	5780.13
8:00–12:00	4787.96	1703.00	20:00–24:00	5333.33	2859.87
0:00 储电量/kWh	6000.00		24:00 储电量/kWh	6000.00	

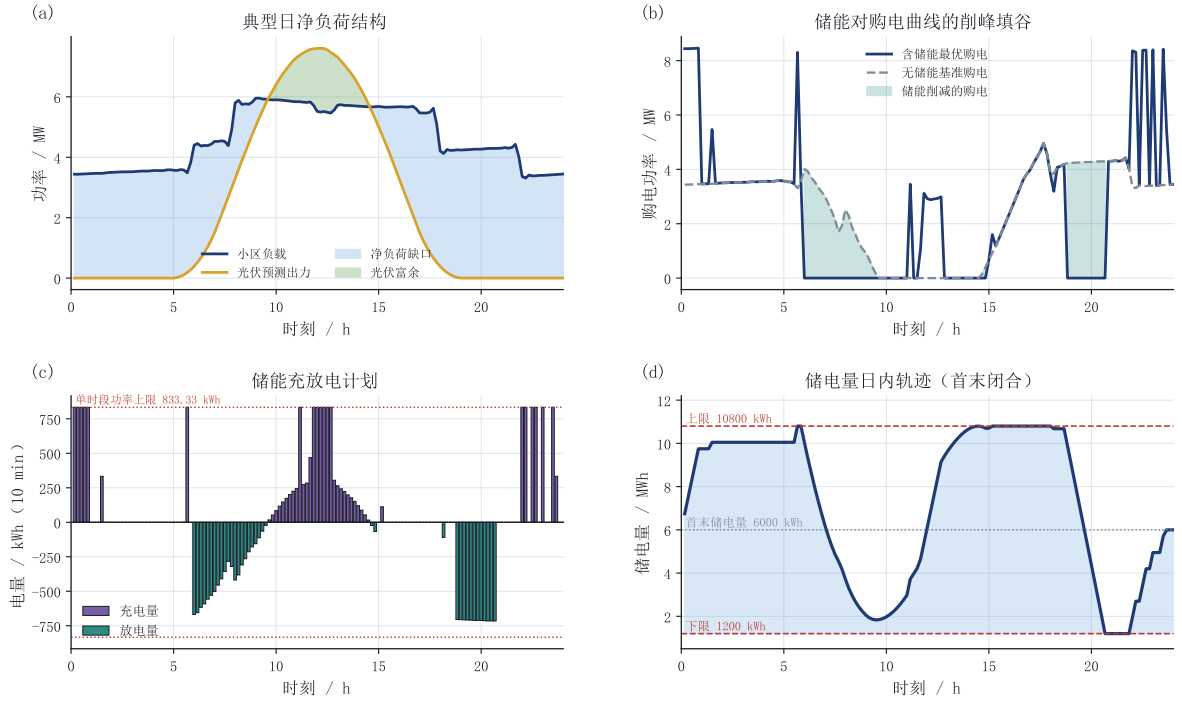

图 3　问题一购电量、储能充放电与储电量曲线

6.3 问题二：因果预测与实时储能反馈

6.3.1 信息集界定与建模难点

问题二与问题一的信息集有本质区别：题目没有给出负载预报，也没有给出光伏预报，只提供历史实际数据。因此主模型按最严格信息边界，仅依据 0:00 之前的历史观测制定计划；另把“日前全天负荷已知”设为理想信息基准 A，用于衡量不同信息条件下的费用差异。

于是问题二可写成如下的两阶段形式：

$$\min_{G^p} \ \mathbb{E}\left[\sum_t \pi_t G_t^p + \sum_t 5\pi_t E_t\right], \tag{12}$$

期望取在负载—光伏的联合不确定性上。由于赛中有限历史不足以稳健估计联合场景及其概率，实际求解不显式生成情景树，而采用**逐日因果确定性等价**：用决策前历史构造 $\hat{L}, \hat{P}$，据此求解全天计划，再以真实实现值进行逐时反馈和费用结算。因此式 (12) 给出决策结构，数值算法则是其可复现近似；近似误差由消融实验与历史窗口上的费用代理量化。

6.3.2 风险定价：单时段 0.2 分位启发

直接用条件均值预测光伏并不最优。考虑单位电量上“高估光伏”与“低估光伏”的边际损失：若预测偏高 $\delta > 0$，计划购电将少买 δ，真实缺口需按 5π 紧急购入，相比按 π 正常购入多支出 $4\pi\delta$；若预测偏低 δ，计划多买 δ，仅多支出 $\pi\delta$。因此单位偏差的非对称损失为

$$\ell(\hat{P} - P) = 4\pi\,(\hat{P} - P)^+ + \pi\,(P - \hat{P})^+, \tag{13}$$

即高估的权重是低估的 4 倍。设光伏的条件分布为 F，最小化 $\mathbb{E}[\ell]$ 得到的一阶条件为

$$F(\hat{P}^\star) = \frac{b}{a+b} = \frac{1}{4+1} = 0.2, \qquad \hat{P}^\star = F^{-1}(0.2), \tag{14}$$

其中 $a = 4$、$b = 1$ 分别为高估与低估的局部损失权重。式 (14) 是经典报童模型的分位点结论 [10]：在**孤立单时段、其他条件固定**时，5 倍紧急电价对应光伏输出的 0.2 条件分位数。这一风险定价思路也用于可再生能源投标 [11]。由于全年模型还含负载误差、SOC 跨期耦合、容量与功率上限及实时补救，该结论不等同于对全年最优性的证明；本文把它作为候选安全边际，并通过历史窗口上的非对称费用代理在线选择。

据此构造光伏预测。设第 m 个基方法给出 $B_{d,t}^m$，并明确定义历史残差为

$$R^{m}_{d,t} = B^{m}_{d,t} - P_{d,t}. \tag{15}$$

若希望把输出移到光伏条件分布的 α 分位，则应减去残差的 $1-\alpha$ 分位数：

$$\hat{P}^{(m,q)}_{d,t} = \max\left\{0,\ B^{m}_{d,t} - Q_q(\mathcal{R}_m)\right\}, \qquad \alpha = 1-q, \tag{16}$$

其中 $Q_q(\mathcal{R}_m)$ 为残差的经验分位数。故式 (14) 对应 $q=0.80$，而非残差的 0.20 分位。为适应跨期约束，程序把 $q = 0.80, 0.85, 0.90, 0.95$（即输出分位 $0.20, 0.15, 0.10, 0.05$）连同多个基预测器共同作为候选，在线选型准则为

$$J_m = \sum_{d'\in\mathcal{W}} \sum_{t} \pi_{d',t} \left[4\left(\hat{P}^{(m)}_{d',t} - P_{d',t}\right)^{+} + \left(P_{d',t} - \hat{P}^{(m)}_{d',t}\right)^{+} \right] \Delta t, \tag{17}$$

其中窗口 $\mathcal{W}$ 为过去 30 天。每日只依据该历史窗口上已经实现的非对称费用代理选择次日候选；四个残差分位候选的累计费用代理与被选天数分别为：$q=0.80$ 时 1889056.42 元、162 天，$q=0.85$ 时 1866485.67 元、109 天，$q=0.90$ 时 1944258.96 元、53 天，$q=0.95$ 时 2278103.51 元、10 天。该记录由程序直接导出，且选型只使用已经发生的误差，满足式 (9)。

负载侧同理构造因果预测器：候选方法族为持续法、7/14/30 日滚动均值、14/30 日线性加权均值与同星期均值；日内 6:00 之后的决策还可利用当天已经执行完的时段对基预测做水平校正。由于负载预测的损失基本对称，选型准则采用平均绝对误差。

6.3.3 计及实时储能反馈的执行阶段

0:00 计划一经确定便不再修改，但日内允许储能依据当前时刻已发生的真实光伏做 10 min 级调整。记计划值为 $(\cdot)^{p}$，执行规则为：

(1) 若真实供电不足，记缺口为 $D_t = L_{d,t}\Delta t + C^{p}_{t} - G^{p}_{t} - P_{d,t}\Delta t - H^{p}_{t} > 0$。先撤销充电量 $r_t = \min(C^{p}_{t}, D_t)$，令 $C_t = C^{p}_{t} - r_t$；剩余缺口 $D_t - r_t$ 再由增加放电补充（受功率与储电量下限约束），仍不足时才按 5 倍电价紧急购电。

(2) 若真实光伏富余，记富余电量为 $S_t > 0$：先减少放电，再增加充电（受功率上限与储电量上限约束），仍有余量则弃光。

该规则在每个时段只读取当前的真实光伏与当前储电量，不涉及任何未来信息。若关闭该反馈（$C_t \equiv C^{p}_{t}$、$H_t \equiv H^{p}_{t}$，仅做储电量边界的被动钳位），缺口将全部转为 5 倍紧急购电，因此二者的费用差即为实时储能调节的价值。

6.3.4 求解结果与原因拆解

主方案（因果负载预测 + 非对称光伏风险边际 + 实时储能反馈）在 2 月至 12 月共 334 天的总费用为 14773804.78 元，其中计划购电费 12877657.23 元、紧急购电费

1896147.55 元；紧急购电量 348036.96 kWh，弃光 2326222.67 kWh，共 232 天发生紧急购电；负载预测平均绝对误差 188.88 kW，光伏预测平均绝对误差 198.14 kW。

为回答“原先偏低的费用究竟由什么造成”，本文固定其余条件不变，只改变单一因素，构造四组对照（表 5）：A 组用当天真实负载（完美信息基准），B 组用因果负载预测，C 组把光伏风险边际换成无安全下修的点预测，D 组关闭实时储能反馈。

表 5　问题二单因素对照实验

方案	负载信息	光伏预测	实时反馈	总费用/元	紧急购电/kWh	弃光/kWh
A	完美	风险边际	开	13275988.13	61482.52	2000031.09
B	因果	风险边际	开	14773804.78	348036.96	2326222.67
C	因果	点预测	开	16038813.56	656904.88	1688048.18
D	因果	风险边际	关	15360709.59	672280.72	2566068.96

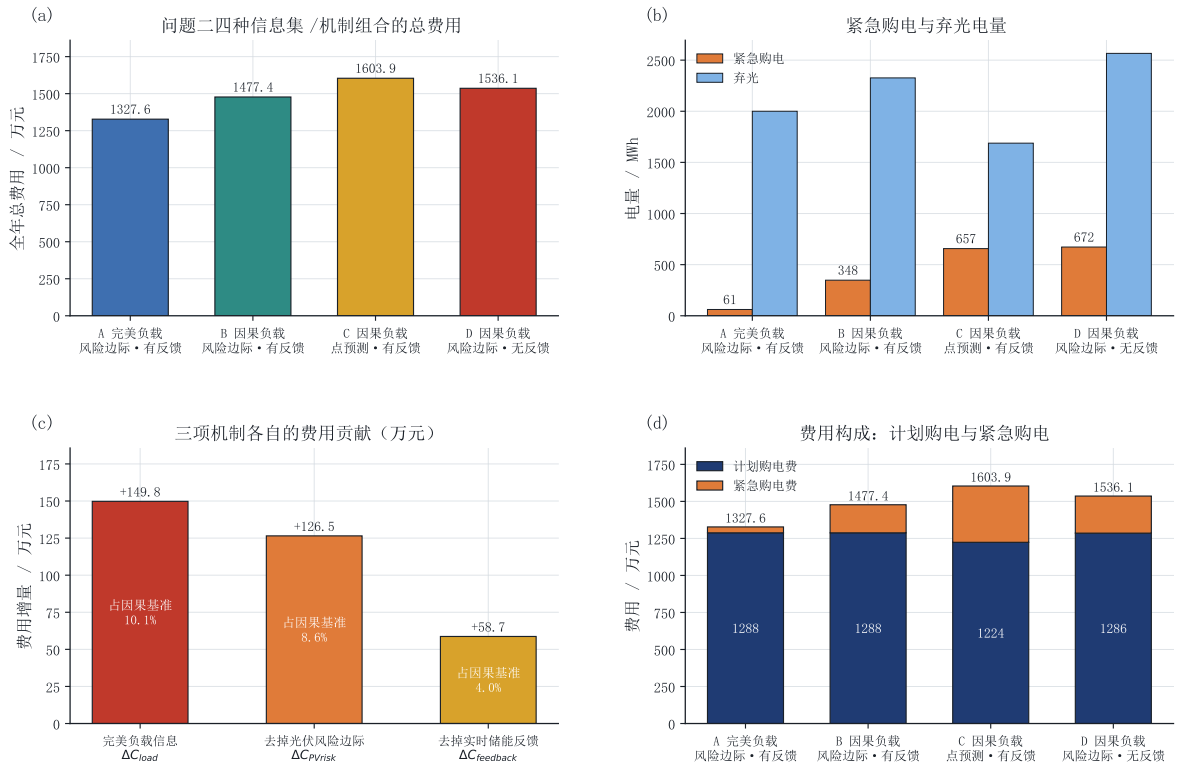

图 4　问题二费用构成与原因拆解

由图 4(c) 的瀑布分解可以量化三个因素：

$$\begin{aligned} \Delta C_{\text{load}} &= C_B - C_A = 1497816.65 \text{ 元}, \\ \Delta C_{\text{PVrisk}} &= C_C - C_B = 1265008.78 \text{ 元}, \\ \Delta C_{\text{feedback}} &= C_D - C_B = 586904.81 \text{ 元}. \end{aligned} \tag{18}$$

结果表明，A 组是日前负荷完全已知的理想信息基准。主模型 B 的全年费用比 A 组高 1497816.65 元，占主模型费用的 10.14%；该差额刻画了两种负荷信息边界的价值。去掉风险边际后费用增加 1265008.78 元；关闭实时反馈后费用增加 586904.81 元，紧急购电由 348036.96 kWh 增至 672280.72 kWh。

图 4(b)(d) 还显示，A 组的弃光量最大而紧急购电最少：日前负荷完全已知使计划与执行几乎无偏差，光伏富余被完整地“计划”进储能与弃光通道。该基准能否在实际部署中使用，取决于业务是否确实提供完整的日前负荷曲线。

按题面要求，表 6–8 给出四个指定日期的购电、储能和紧急购电结果；所有数值由最终 `result2.xlsx` 自动抽取，完整 144 时段明细见支撑材料。

表 6　问题二指定日期购电结果

日期	10:00	12:00	14:00	16:00	18:00	20:00	全天/kWh	计划费/元
2025-03-20	0.00	628.79	0.00	478.85	716.29	0.00	70508.76	42753.80
2025-06-21	0.00	0.00	0.00	136.81	458.04	0.00	33673.85	18879.29
2025-09-23	0.00	507.89	0.00	519.64	831.20	5.90	71887.69	45083.22
2025-12-21	0.00	947.47	0.00	821.34	725.20	0.00	92741.53	59280.18

表 7　问题二指定日期储能结果

日期	各 4 h 时段充电量/放电量（kWh）						$S_{0:00}$	$S_{24:00}$
日期	0:00–4:00	4:00–8:00	8:00–12:00	12:00–16:00	16:00–20:00	20:00–24:00	kWh	kWh
2025-03-20	356.39/185.93	1361.00/4608.26	6269.69/2777.38	2604.97/288.51	112.53/5894.56	9737.25/2860.63	9963.05	9899.29
2025-06-21	2376.47/537.80	1359.38/1738.75	9741.37/0.00	0.00/0.00	0.00/6210.53	158.74/2511.19	1200.00	1252.06
2025-09-23	3.43/2.78	158.81/5967.15	5693.12/1896.62	4388.33/430.85	0.00/5509.97	10293.05/2827.40	10800.00	10800.00
2025-12-21	3646.73/622.21	1638.21/2255.21	5301.47/6488.22	8099.28/2311.29	177.27/5918.77	6323.72/2804.89	6832.77	6833.46

表 8　问题二指定日期紧急购电结果

日期	紧急购电时段及对应电量/kWh	合计/kWh	当日总费用/元
2025-03-20	20:30–20:40（127.41）；21:40–21:50（5.16）	132.57	43662.61
2025-06-21	20:30–20:40（56.42）；21:30–23:40（555.36）	611.79	20614.91
2025-09-23	20:20–20:30（6.38）	6.38	45126.21
2025-12-21	7:50–8:00（19.18）；9:50–10:50（1391.06）；20:30–20:40（89.74）	1499.99	66728.16

6.4 问题三：多时刻滚动优化与节点价值消融

6.4.1 滚动优化框架

问题三在每天 0:00、6:00、12:00、18:00 可获得未来 24 h 整点的光伏预报。本文的决策链为：

(1) 0:00 **基准计划**：由节点 0 的预报降尺度得到 144 个时段的光伏预测，在公共约束下得到全天基准计划 G^{p}，并执行 0:00 ∼ 6:00 的 36 个时段；

(2) **滚动调整**：在 6:00、12:00、18:00 各节点，以当时实际测得的储电量为初值，用新到达的预报重新优化未来 24 h 的购电与储能，但只执行到下一个决策节点前的 36 个时段；

(3) **状态与计划递推**：执行块结束时把实测储电量传递给下一个节点；若某节点不允许更新，则继续执行最近一次有效计划的对应切片，不回退到 0:00 方案。跨午夜只传递储电量，次日 0:00 生成新基准计划。

这一“重优化—短执行—状态传递”的结构使每次更新都建立在上一执行块的**实际状态**之上；未更新时则保持最近计划继续生效。该机制既符合题目的时序语义，也与微网能量管理中的滚动时域控制一致 [6, 7]。

附件 3 给出的是整点值，需降尺度到 10 min 端点。本文比较两种方式：阶梯保持（每个小时值重复 6 次）与线性插值。图 5(c) 显示，在真正会被执行的 6 h 块上，线性插值的平均绝对误差为 110.22 kW，显著低于阶梯保持的 294.37 kW，因此主方案采用线性插值。此外，本文对每个发布节点分别维护最近 30 天已执行块的预报残差，并按式 (13) 的费用代理在 0.80 ∼ 0.95 分位数间在线选择下修边际，实现**分发布节点的因果偏差校正**。

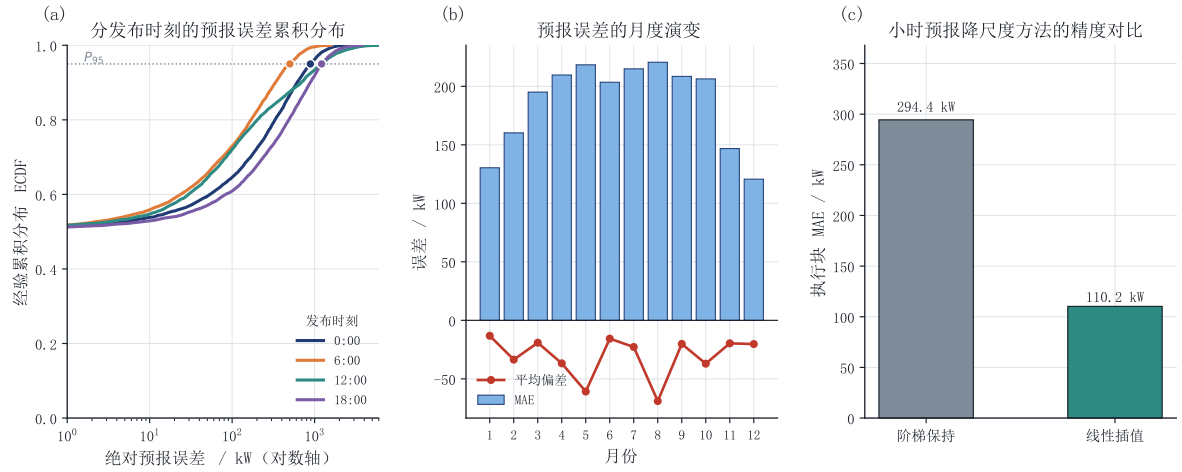

图 5　附件 3 预报误差诊断与降尺度对比

6.4.2 主结算口径与敏感性口径

题目明确写出总费用由计划购电费、调整相关费用和紧急购电费相加，故本文以**叠加结算**作为主口径。设 G^p 为 0:00 计划购电量、G^a 为执行前最后一次有效计划，并记 $u=(G^a-G^p)^+$、$v=(G^p-G^a)^+$，单时段主结算费用为

$$C_t^{\text{add}}=\pi_t G_t^p+0.5\pi_t v_t+1.5\pi_t u_t. \tag{19}$$

为检验文字解释的稳健性，另设**差额结算**敏感性口径，即把调整后的实际购电结算额与违约 / 增购费用合并：

$$C_t^{\text{rep}}=\pi_t\left[\min(G_t^p,G_t^a)+0.5\,v_t+1.5\,u_t\right], \tag{20}$$

当 $G^a<G^p$ 时实付 $\pi(G^a+0.5v)$，当 $G^a>G^p$ 时实付 $\pi(G^p+1.5u)$。

两种口径的目标函数不同，因此**最优调度本身也不同**。本文把 u_t,v_t 显式引入模型，并施加等式约束

$$G_t^a-G_t^p=u_t-v_t,\qquad u_t,v_t\geq 0, \tag{21}$$

从而精确表示两侧调整量。式 (21) 必须为等式；若放松为不等式，下调时会漏付违约费用。主模型还施加式 (10) 的充放电互斥约束，避免连续松弛利用内部耗散绕过功率上限。两种口径均从头重新优化，并以直接回代费用验证，而非对同一调度事后换算。

叠加口径下总费用可严格拆成“计划购电费 + 调整相关费用 + 紧急购电费”；差额口径只用于敏感性分析，不作为题面提交工作簿的来源。

6.4.3 决策节点价值消融实验

题目询问“是否需要引入其他时刻的预报”。先把**允许更新决策的节点集合**作为消融变量，构造四组方案：

$$\begin{aligned} &S_0=\{0{:}00\}, && S_1=\{0{:}00,6{:}00\},\\ &S_2=\{0{:}00,6{:}00,12{:}00\}, && S_3=\{0{:}00,6{:}00,12{:}00,18{:}00\}. \end{aligned} \tag{22}$$

四组的算法、终端策略与实时反馈保持一致；节点关闭时沿用最近一次有效计划。由于一次更新同时使用截至该节点的负载观测和新发布的光伏预报，$S_0\sim S_3$ 给出的是**综合节点价值**。定义

$$V_6=C_{S_0}-C_{S_1},\qquad V_{12}=C_{S_1}-C_{S_2},\qquad V_{18}=C_{S_2}-C_{S_3},\qquad V_{\text{total}}=C_{S_0}-C_{S_3}. \tag{23}$$

为避免把综合节点价值误称为“纯光伏预报价值”，再为每个节点加入一个配对控制组 C_k：在该节点允许用新增负载观测重优化，但光伏预测冻结为上一有效节点版本。于是

$$V_\tau^L = C_{S_{k-1}} - C_{C_k}, \qquad V_\tau^P = C_{C_k} - C_{S_k}, \qquad \tau \in \{6, 12, 18\}, \tag{24}$$

在给定“先负载、后光伏”的更新顺序下，两式分别表示负载信息刷新和官方光伏预报刷新的配对增量贡献，两项之和严格等于该节点综合价值。由于两类信息与优化决策存在交互，若交换更新顺序，各自分解值可能改变，故该分解不是顺序无关的唯一因果归因。

6.4.4 求解结果

叠加结算主口径下的四组结果列于表 9。

表 9　问题三决策更新节点消融结果（叠加结算）

方案	可用更新节点	总费用/元	增量价值/元	累计价值/元	紧急购电/kWh	调整电量（上/下）/kWh
S_0	0:00	15731164.40	—	0.00	539486.13	0.00/0.00
S_1	+6:00	14510735.12	1220429.28	1220429.28	165129.37	949099.00/29976.01
S_2	+12:00	14213145.74	297589.38	1518018.66	146857.80	706080.83/73982.00
S_3	+18:00	13917403.19	295742.55	1813761.21	68481.28	786757.05/106014.05

由图 6 与表 9 可以得到明确结论：

$$\begin{aligned} V_6 &= 1220429.28 \text{ 元}, \quad V_{12} = 297589.38 \text{ 元}, \\ V_{18} &= 295742.55 \text{ 元}, \quad V_{\text{total}} = 1813761.21 \text{ 元}. \end{aligned} \tag{25}$$

在题目未设置预报获取和调整操作固定成本的前提下，三个更新节点的综合增量收益均为正，全年累计节省 1813761.21 元，因此应引入 6:00、12:00、18:00 三次更新。其中 6:00 节点价值最大；18:00 节点的纯光伏配对增量仅为 10865.70 元，若实际部署存在固定成本，则需对该节点另作成本—收益判断。图 6(c) 的季节分解显示，这一结论并非由单一日期驱动。

配对控制进一步给出表 10。例如 C_1 允许 6:00 用新增负载观测重优化，但继续使用 0:00 光伏预测；从 $S_0 \to C_1 \to S_1$ 可在“先负载、后光伏”的给定顺序下把综合收益拆为两部分。三个节点累计的负载刷新贡献为 1225632.49 元，官方光伏预报刷新贡献为 588128.71 元；若交换更新顺序，二者的分解值可能变化。因此 $S_0 \sim S_3$ 应解释为决策更新的综合价值，而不是把全部收益归给光伏预报。

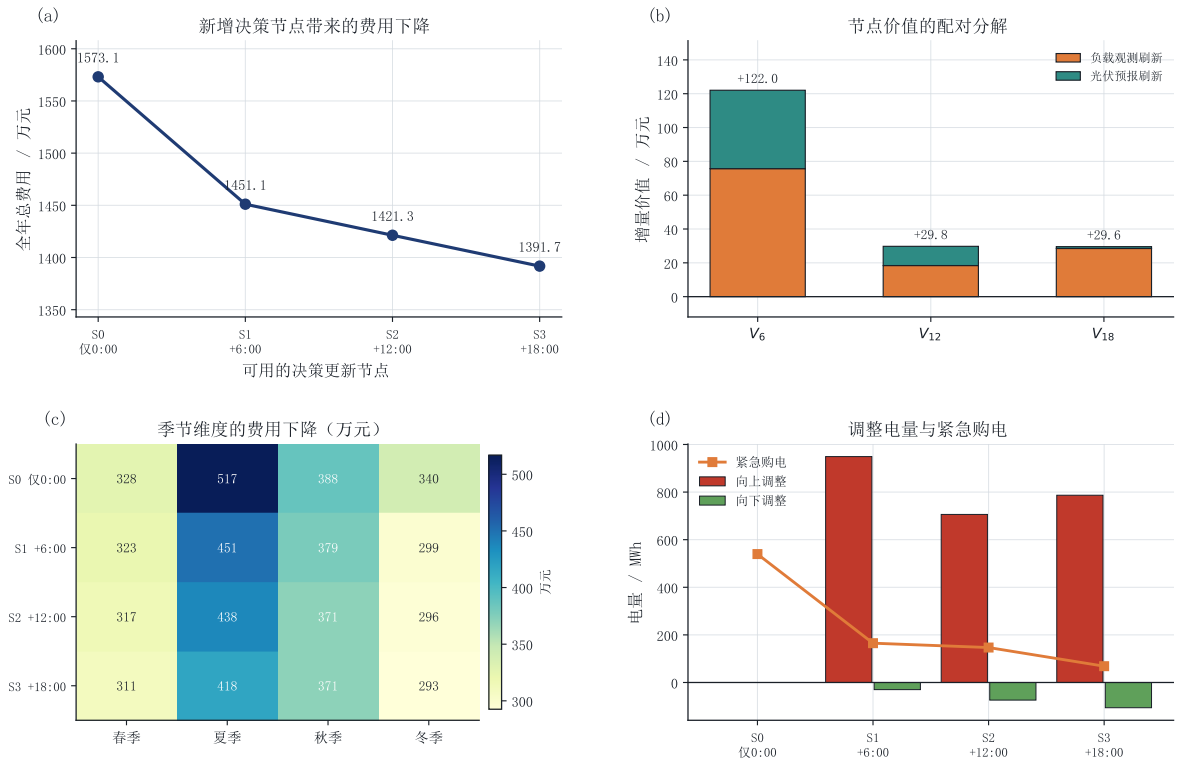

图 6　决策更新节点的增量价值与季节分解

表 10　节点价值的负载刷新与光伏预报配对分解

节点	控制组费用/元	负载刷新贡献/元	光伏刷新贡献/元	两项合计/元
6:00	14974345.92	756818.48	463610.80	1220429.28
12:00	14326797.96	183937.16	113652.22	297589.38
18:00	13928268.89	284876.85	10865.70	295742.55
合计	—	1225632.49	588128.71	1813761.21

主方案（S_3，叠加结算）的全年总费用为 13917403.19 元，且

$$12640688.81 + 934466.71 + 342247.67 = 13917403.19 \text{ 元}, \tag{26}$$

三项依次为计划购电费、调整相关费用与紧急购电费；其中调整相关费用由下调违约费 24867.42 元和上调增购费 909599.29 元组成，两者之和为 934466.71 元。费用回代差低于 10^{-8} 元，全年紧急购电 68481.28 kWh。相对问题二主方案，滚动调整节省 856401.59 元。差额结算对照的 S_3 费用为 13727989.23 元，累计节点价值为 2003175.17 元，结论方向一致，但它不作为提交主答案。

按题面要求，表 11–13 给出四个指定日期的计划 / 调整购电、储能和紧急购电结果；数值均由最终 `result3.xlsx` 自动生成。

表 11　问题三指定日期购电结果

日期	口径	10:00	12:00	14:00	16:00	18:00	20:00	全天/kWh	费用/元
2025-03-20	计划	0.00	561.89	0.00	615.11	716.29	0.00	69520.52	42031.40
	调整	0.00	561.89	0.00	615.11	716.29	0.00	71118.10	43975.30
2025-06-21	计划	0.00	0.00	0.00	75.05	399.26	0.00	31110.55	17009.78
	调整	0.00	0.00	0.00	75.05	399.26	0.00	32775.70	19480.32
2025-09-23	计划	0.00	416.97	0.00	566.94	831.20	5.90	73888.09	46335.87
	调整	0.00	436.86	0.00	566.94	831.20	5.90	72865.79	46796.73
2025-12-21	计划	0.00	937.06	0.00	796.19	725.20	0.00	90602.15	57486.24
	调整	0.00	937.06	0.00	825.73	725.20	0.00	94204.59	61748.15

表 12　问题三指定日期储能结果

日期	各 4 h 时段充电量/放电量（kWh）						$S_{0:00}$	$S_{24:00}$
日期	0:00–4:00	4:00–8:00	8:00–12:00	12:00–16:00	16:00–20:00	20:00–24:00	kWh	kWh
2025-03-20	1774.80/185.93	1263.10/5143.78	5669.44/2777.38	3965.38/254.72	19.98/5853.36	8370.56/2897.33	8685.45	8628.49
2025-06-21	1469.50/642.66	1255.97/1285.96	10322.21/0.00	0.00/0.00	172.24/5758.17	397.48/2685.89	1200.00	1930.46
2025-09-23	3.43/2.78	138.54/6205.13	6293.08/1896.62	4068.83/403.62	36.12/5596.21	10129.96/2827.40	10800.00	10589.91
2025-12-21	3261.87/622.21	1944.62/1464.57	5424.47/7563.28	8080.05/2298.66	47.67/5925.83	6759.39/2804.89	7236.46	7225.56

6.4.5 典型日调度过程

图 7 给出 2025 年 6 月 21 日的调度全景。该日为夏季高辐照日，光伏峰值超过 8 MW，而小区负载仅约 3.8 MW，二者严重错配。

图 7(a) 显示，储能在凌晨与上午充电，功率一度达到 −5 MW。图 7(c) 中，储电量由 1.2 MWh 升至 10.8 MWh 上限，并在中午前后维持高位，表明 12 MWh 额定容量不足以吸纳该日全部富余光伏。不能向外网售电后，余量只能弃光；图 7(d) 的弃光峰值超过

表 13　问题三指定日期紧急购电结果

日期	紧急购电时段及对应电量/kWh	合计/kWh	当日总费用/元
2025-03-20	20:30–20:40（127.39）；21:40–21:50（5.16）	132.55	44884.02
2025-06-21	6:20–7:20（452.79）	452.79	21390.79
2025-09-23	20:20–20:30（6.38）	6.38	46839.73
2025-12-21	7:50–8:00（29.09）；10:10–10:50（630.86）；20:30–20:40（89.74）	749.70	65550.73

4500 kW，而同时段紧急购电接近零。

图 7(b) 对比了 0:00 基准计划与最终调整购电：两者走势基本一致，说明 0:00 的预报已经把握了当日的主要形态；红色（向上调整）与绿色（向下调整）区域集中出现在 18:00 ～ 21:00 的傍晚时段，这正是 6:00、12:00、18:00 新预报对午后云量变化做出修正的结果。

这一典型日也揭示了模型的主要经济矛盾：**夏季的正午弃光与傍晚高价购电同时存在**，二者都由储能容量与充放电功率上限决定，而非由预测精度决定。这解释了为什么图 6(c) 中夏季的费用改善幅度最大——后续预报能在傍晚前提前调整储能的放电节奏，把有限的容量留给真正高价时段。

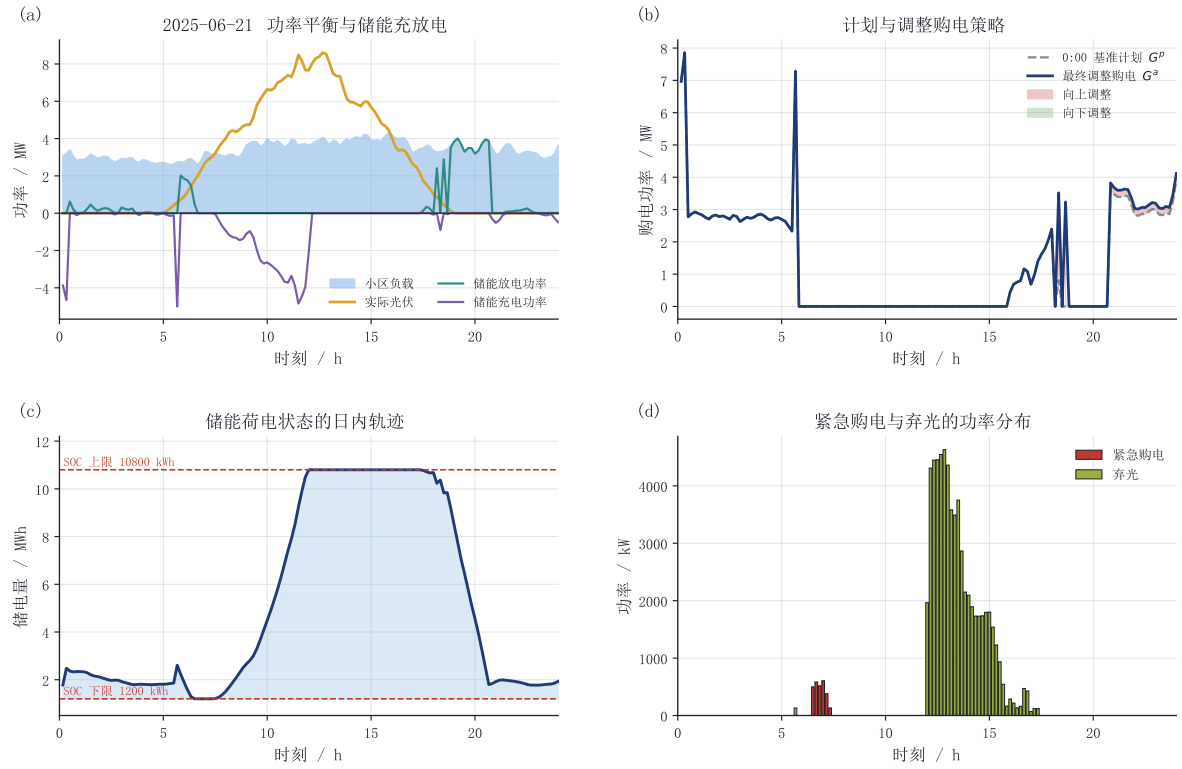

图 7　典型日调度过程（2025-06-21）

6.5 问题四：波动电价下的重算与价格信息边界

6.5.1 模型迁移

问题四保持问题二、三的数据范围、负载 / 光伏信息集、储能反馈与结算规则不变，只把固定典型日电价 π_t 替换为附件 4 的 $\pi_{d,t}$。题面明确要求“根据附件 4 中的电价数据”重算，因此本文以附件 4 的逐时电价为基准，其结果同时作为 `result4-2.xlsx` 与 `result4-3.xlsx` 的交付依据。另考虑实际交易中价格可能逐步发布的情形，构造因果价格预测扩展；其候选方法族与负载侧一致，只使用决策节点之前已经发生的价格，并

按历史平均绝对误差在线选型。

- **基准情形（主结果）**：调度目标中的逐时价格直接取自附件 4；
- **因果价格（扩展）**：仅使用决策时刻之前已发生的价格预测后续价格，用于考察发布机制受限时的稳健性。

6.5.2 求解结果与价格信息价值

四组结果列于表 14。

表 14　问题四波动电价下的四组结果

方案	价格信息	总费用/元	紧急购电/kWh	弃光/kWh
Q4-2	基准（附件 4）	15523884.83	355425.99	2365390.66
Q4-2	因果预测	15590096.88	354599.54	2322027.45
Q4-3	基准（附件 4）	14589632.38	87429.50	2391744.79
Q4-3	因果预测	14722175.07	84956.17	2387567.36

相对于因果扩展，直接使用附件 4 所降低的费用为

$$V_{\text{price},Q2} = 66212.05 \text{ 元 } (0.42\%), \qquad V_{\text{price},Q3} = 132542.68 \text{ 元 } (0.90\%), \tag{27}$$

这两个差额衡量的是**价格发布信息受限的机会成本**，而非对题面主口径的修正。其相对幅度见式 (27)。一方面，因果预测已经抓住较稳定的日内结构；另一方面，不允许售电时，储能套利受 12000 kWh 容量和 5000 kW 功率的双重限制，即使掌握完整价格也无法无限转移电量。

与问题二的负载信息对照相比，负载未来信息泄漏造成的费用低估为 1497816.65 元，明显大于上述价格情形差额。工程上应优先改善负载与光伏预测；但竞赛交付仍严格采用题面指定的附件 4 直用结果。

另一方面，动态电价下的滚动调整价值为

$$C_{Q4\text{-}2}^{\text{perfect}} - C_{Q4\text{-}3}^{\text{perfect}} = 934252.45 \text{ 元}, \qquad C_{Q4\text{-}2}^{\text{causal}} - C_{Q4\text{-}3}^{\text{causal}} = 867921.82 \text{ 元}, \tag{28}$$

两者均为正，说明在附件 4 直用与因果价格扩展下，**多时刻决策更新都能降低费用**。其数值差别来自价格序列改变了储能充放电时点，而不改变滚动状态递推的机制。

因果扩展中，0:00 与 6:00 的价格 MAE 分别为 0.053238、0.107166 元/kWh；12:00 与 18:00 分别为 0.112235、0.064665 元/kWh。0:00 **最小**、18:00 次之，误差并非随发布时间单调下降。

题面指定日期的完整重算结果列于表 15–20，均由附件 4 直用主方案的两个最终工作簿自动生成。

表 15　问题四对应问题二指定日期购电结果

日期	10:00	12:00	14:00	16:00	18:00	20:00	全天/kWh	计划费/元
2025-03-20	0.00	628.79	0.00	478.85	716.29	709.49	70933.40	43543.79
2025-06-21	0.00	0.00	0.00	136.81	458.04	0.00	34143.61	16105.02
2025-09-23	0.00	530.79	0.00	542.54	831.20	487.35	73783.38	47932.82
2025-12-21	0.00	947.47	0.00	821.34	725.20	0.00	92738.65	69147.78

表 16　问题四对应问题二指定日期储能结果

日期	各 4 h 时段充电量/放电量（kWh）						$S_{0:00}$	$S_{24:00}$
日期	0:00–4:00	4:00–8:00	8:00–12:00	12:00–16:00	16:00–20:00	20:00–24:00	kWh	kWh
2025-03-20	1896.69/1399.06	2151.59/5232.95	6269.69/2777.38	2615.35/288.51	138.93/6617.32	9701.81/2172.38	9897.92	9852.79
2025-06-21	1626.39/1304.00	3854.32/1733.02	8935.37/0.00	0.00/0.00	0.00/6210.53	890.45/3117.22	1200.00	1237.25
2025-09-23	945.57/1171.57	771.33/5594.21	5534.21/1896.62	3946.84/407.95	116.98/6713.26	10237.57/1673.93	10800.00	10800.00
2025-12-21	4516.10/563.53	818.23/2242.93	5301.47/6493.23	7698.88/2313.01	579.78/5273.92	6224.28/3480.03	6715.04	6710.31

表 17　问题四对应问题二指定日期紧急购电结果

日期	紧急购电时段及对应电量/kWh	合计/kWh	当日总费用/元
2025-03-20	20:30–20:40（113.09）；21:40–21:50（5.16）	118.25	44311.65
2025-06-21	0:30–0:40（10.28）；1:10–2:30（532.14）；3:30–3:40（151.46）；20:30–20:40（56.42）；21:30–23:30（527.96）	1278.26	19150.64
2025-09-23	20:20–20:30（6.38）	6.38	47976.90
2025-12-21	7:50–8:00（19.18）；10:00–10:50（1386.06）；20:40–20:50（81.48）	1486.71	79176.58

七、 模型检验与分析

本文构建物理可行性、信息集 mutation、计划持续性、结算分解和历史配置回归五类自动验证，最终主结果全部通过。

表 18　问题四对应问题三指定日期购电结果

日期	口径	10:00	12:00	14:00	16:00	18:00	20:00	全天/kWh	费用/元
2025-03-20	计划	0.00	561.89	0.00	615.11	716.29	709.49	69636.12	42721.49
	调整	0.00	561.89	0.00	615.11	716.29	709.49	71055.34	44468.01
2025-06-21	计划	0.00	0.00	0.00	75.05	399.26	0.00	31580.30	14682.69
	调整	0.00	0.00	0.00	75.05	399.26	0.00	33253.38	16931.47
2025-09-23	计划	0.00	416.97	0.00	566.94	831.20	487.35	74177.20	48179.34
	调整	0.00	436.86	0.00	566.94	831.20	487.35	73123.14	48590.66
2025-12-21	计划	0.00	937.06	0.00	796.19	725.20	0.00	90599.28	66885.94
	调整	0.00	937.06	0.00	825.73	725.20	0.00	94456.90	72431.25

表 19　问题四对应问题三指定日期储能结果

日期	各 4 h 时段充电量/放电量（kWh）						$S_{0:00}$	$S_{24:00}$
日期	0:00–4:00	4:00–8:00	8:00–12:00	12:00–16:00	16:00–20:00	20:00–24:00	kWh	kWh
2025-03-20	1896.69/772.49	2225.87/5234.86	5669.44/2777.38	3986.75/254.72	46.92/6576.11	8120.06/2172.85	8442.96	8429.22
2025-06-21	1626.39/1304.00	2765.24/1733.02	10024.45/0.00	0.00/0.00	206.50/5792.43	1152.74/3286.53	1200.00	1935.59
2025-09-23	945.57/1171.57	623.69/6189.47	5952.05/1896.62	4406.19/403.62	116.98/6713.26	9953.80/1673.93	10800.00	10544.61
2025-12-21	4011.81/609.78	2026.32/2127.30	5424.47/7563.28	7737.10/2298.66	390.62/5280.98	6746.94/3468.33	7210.53	7193.69

7.1 物理可行性与数值精度

对问题一、问题二四组对照、问题三主消融与配对控制、结算敏感性以及问题四全部年度方案逐时回代，检验母线平衡、SOC 递推、边界、互斥和费用分解，结果见表 21。

连续模型由词典序目标排除无效吞吐，叠加结算主模型由式 (10) 严格禁止同时充放电。问题三总费用与“计划费 + 调整费 + 紧急费”直接回代一致。五个结果工作簿保留附件 5 的工作表结构与格式，回读数值与逐时明细一致。

表 20　问题四对应问题三指定日期紧急购电结果

日期	紧急购电时段及对应电量/kWh	合计/kWh	当日总费用/元
2025-03-20	20:30–20:40（112.63）；21:40–21:50（5.16）	117.78	45232.79
2025-06-21	0:30–0:40（10.28）；1:10–2:30（532.14）；3:30–3:40（151.46）	693.88	18396.97
2025-09-23	20:20–20:30（6.38）	6.38	48634.74
2025-12-21	7:50–8:00（29.09）；10:20–10:50（630.86）；20:40–20:50（81.48）	741.43	77404.53

表 21　模型检验结果汇总

检验项目	最大值	验收阈值	结论
母线能量平衡残差	$< 10^{-12}$ kWh	10^{-7} kWh	通过
储电量递推残差	$< 10^{-12}$ kWh	10^{-7} kWh	通过
储电量取值区间	[1200, 10800] kWh	同上	通过
主方案同时充放电量	$< 10^{-8}$ kWh	10^{-5} kWh	通过
问题三费用分解回代差	$< 10^{-8}$ 元	10^{-7} 元	通过
Excel 回读与逐时明细差异	$< 10^{-10}$	数值一致	通过

7.2 未来信息泄漏的 mutation 测试

物理约束只能保证结果可行，不能保证结果可实现。为此本文设计了四组因果性篡改实验与一组计划持续性检验，直接检查信息集因果性：

(1) **问题二负载因果性**：把第 d 天真实负载整体乘以 10，在不改动 d 日之前历史数据的条件下重新求解，要求第 d 天 0:00 的计划输入完全不变（最大偏差 $< 10^{-10}$）；

(2) **问题二光伏因果性**：同法篡改第 d 天真实光伏，要求 0:00 的光伏计划输入完全不变；

(3) **问题三节点因果性**：篡改第 d 天 6:00 之后的真实负载与真实光伏，要求 6:00 决策输入与结果不变；后续节点可合法使用已经执行完的观测；

(4) **问题四价格因果性**：篡改第 d 天 6:00 之后的附件 4 真实电价，要求因果价格模式下 6:00 决策不变，而附件 4 直用模式下应当改变。

(5) **计划持续性**：仅启用 0:00, 6:00 时，核对后续切片均来自 6:00 更新的有效计划，而非回退到 0:00 基准计划。

五组测试全部通过。第 (4) 组中同一篡改确实会改变附件 4 直用结果，说明因果扩展下的“不变”来自信息隔离而非篡改无效；第 (5) 组直接覆盖了节点消融最容易出现的计划回退错误。

7.3 结算口径的单元测试

按题目文字直接手算校验两种口径的费用公式。取 $\pi = 1$ 元/kWh：

- replacement：$G^p = 900,\ G^a = 700 \Rightarrow 700 + 0.5 \times 200 = 800$；$G^p = 700,\ G^a = 900 \Rightarrow 700 + 1.5 \times 200 = 1000$；
- additive：$G^p = 900,\ G^a = 700 \Rightarrow 900 + 0.5 \times 200 = 1000$；$G^p = 700,\ G^a = 900 \Rightarrow 700 + 1.5 \times 200 = 1000$。

两组手算结果与程序输出一致。在随机负载 / 光伏 / 价格序列上，优化目标与直接

回代公式之差小于 10^{-7}；两种口径分别重优后，各自费用均不高于把另一口径方案直接代入所得的费用。

7.3.1 充放电互斥的混合整数检验

连续松弛在叠加结算下可能利用“边充边放”耗散 SOC；本文在主模型的每个 24 h 滚动窗口中设置至多 144 个二元变量 z_t，由式 (10) 保证充放电互斥。全年主方案最大同时充放电量低于 10^{-8} kWh，所有滚动子问题均达到最优状态，说明互斥约束已排除该退化现象。

7.4 历史配置的回归复现

为确认修订没有破坏原有工程，本文在旧参数组合（完美负载、附件 4 直用、实时反馈、非对称风险边际、差额结算、日循环终端且不用典型日先验）下重新运行四问，与既有结果逐项比对：

表 22　历史配置结果回归比对

结果	修订前/元	修订后/元	绝对偏差/元
问题一	35 126.9486	35 126.9486	$< 10^{-3}$
问题二	13 276 966.0012	13 276 966.0012	$< 10^{-3}$
问题三	13 048 124.9508	13 048 124.9508	$< 10^{-3}$
问题四-2	13 949 811.6640	13 949 811.6640	$< 10^{-3}$
问题四-3	13 734 900.0034	13 734 900.0034	$< 10^{-3}$

表 22 说明，新增的信息集模式、结算口径与参数化储能没有改变历史配置行为。旧结果不是数值错误，而是完美信息基准；本文仅用它量化信息泄漏造成的乐观偏差。

7.5 灵敏度分析

7.5.1 终端储电量策略

题目只对问题一明确要求 $S_{0:00} = S_{24:00}$，问题二、三并未提出该要求。本文主模型采用“计划终端储电量等于当日实际初始储电量”的日循环口径（daily_cycle），以保证储能长期可持续运行；同时构造不做终端约束（free）的对照组。结果为：daily_cycle 总费用 14773804.78 元、全年末储电量 7107.51 kWh；free 总费用 14728422.40 元、全年末储电量 1235.46 kWh。

free 口径在数值上便宜约 0.307%，但其日末储电量均值仅为 1660 kWh，全年末储电量为 1235.46 kWh，接近 1200 kWh 下限。这是有限时域优化的末端透支：不计后续

价值时，模型倾向于在窗口末端放空电池。

图 8(c) 中 free 的日末 SOC 标准差为 896 kWh，确实小于 daily_cycle 的 3798 kWh；这只是因为 free 结果集中在低 SOC 区间，不能解释为更可持续。综合费用差、均值位置和全年末状态，本文保留 daily_cycle 为主口径。

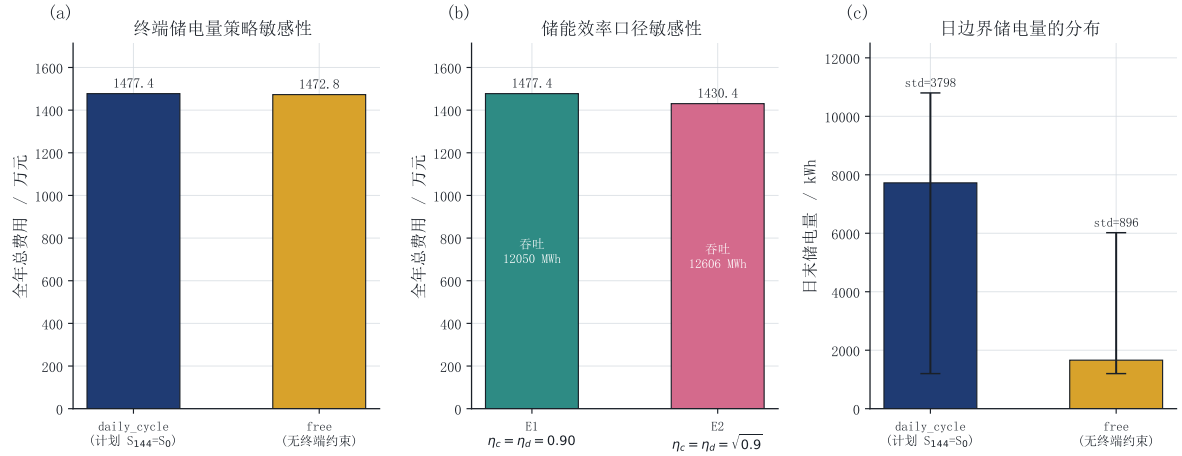

图 8　终端策略与储能效率口径敏感性

7.5.2 储能效率口径

题目只写“充放电效率为 90%”，存在两种解释：其一，充电效率与放电效率各为 0.9（往返效率 0.81）；其二，往返总效率为 0.9（充放电效率各为 $\sqrt{0.9} \approx 0.9487$）。前者是更保守也更为工程界常用的口径，本文主模型采用前者。作为对照，两种口径下的结果列于表 23。

表 23　储能效率口径敏感性

效率口径	全年总费用/元	储能吞吐量/kWh	紧急购电量/kWh
口径 A：$\eta_c = \eta_d = 0.90$（往返 0.81）	14773804.78	12049920.76	348036.96
口径 B：$\eta_c = \eta_d = \sqrt{0.9}$（往返 0.90）	14304371.70	12606462.13	349968.83

采用往返效率 0.90 时费用更低，因为储能损耗更小、可用于峰谷套利的电量更多。两种口径的结论方向完全一致，说明本文的策略结论不依赖于效率口径的具体解释。

7.5.3 其他误差来源

主要误差来源包括三方面：光伏与负载点预测误差、小时预报到 10 min 的降尺度误差，以及实际市场价格发布时间的不确定性。前两者已通过历史窗口上的费用代理在线选择、实时反馈和节点消融量化；第三项不改变题面“附件 4 直用”的主答案，因果

价格扩展仅用于说明部署时可能的机会成本。这些误差不破坏数值结果的内部一致性，但限制模型直接外推到真实长期运行的精度。

八、 模型的评价

8.1 模型的优点

- **物理内核统一、四问口径一致**。四个问题共用式 (2)∼(7) 的物理约束，各问只改变信息集与费用项；全部年度方案均通过逐时回代。
- **信息集边界严格且可自动验证**。所有预测器只使用决策时刻之前已发生的观测；五组 mutation 与计划持续性测试把“无未来信息泄漏”变成可重复执行的验收项。
- **风险边际有可解释依据**。5 倍紧急电价在单时段下导出 $4:1$ 非对称损失和光伏输出 0.2 分位启发；全年模型不直接外推该局部结论，而在多个残差分位候选中按历史窗口上的非对称费用代理在线选择。
- **状态与计划递推严格**。每次滚动优化以上一执行块实测 SOC 为初值；节点关闭时最近有效计划持续生效，不会回退到日初方案。
- **结论可归因**。问题二 A/B/C/D 拆分负载信息、风险边际与实时反馈；问题三在主节点消融外增加配对控制，分别量化负载观测刷新和光伏预报刷新；问题四区分附件 4 直用主结果与因果价格扩展。
- **结算与物理约束一致**。叠加结算按题面作为主口径，费用三项可直接相加；u, v 等式精确表示调整方向，z_t 二元变量严格排除同时充放。差额口径从头重优，仅作敏感性对照。
- **工程可复现**。四问均有独立求解脚本，输出保留环境信息、随机种子与结果哈希；历史参数组合可复现修订前数值。

8.2 模型的不足

- 问题二与问题三采用“计划终端储电量等于当日实际初值”的日循环口径。该处理抑制日末透支，但限制跨日峰谷套利；灵敏度分析中费用影响为 0.307%。
- 光伏预测未引入数值天气预报、辐照度、温度与云量等外生变量，仅使用同刻历史序列与残差分位数。当气象形态发生突变（例如连续阴雨转晴）时，预测与安全边际的调整会滞后一天左右。
- 当前安全边际对每个预测器采用全天统一的标量 kW，下修幅度未刻画日内与季节异方差；后续可按时段或季节分别估计残差分位数。
- 储能模型忽略自放电、容量衰减、循环寿命成本与效率随功率变化的特性，因此年度费用中没有反映电池的长期老化代价，得到的充放电深度可能比实际最优更大。

- 问题四按题面直接使用附件 4 全部价格；若真实市场并非提前发布完整曲线，则因果扩展更接近部署条件，但其预测精度仍可提高。
- 问题二采用逐日滚动的确定性等价形式求解两阶段模型，安全边际由历史残差分位数估计。当可用历史不足（如年初的前 7 天）时，分位数估计的稳定性有限，本文以附件 1 的典型日曲线作为冷启动先验，但这仍是一个近似。

九、 模型的改进与推广

9.1 模型的改进

（1）把单日滚动扩展为多日滚动时域。当前模型每天重新生成基准计划，跨日只传递储电量。若把优化窗口扩展到 3 ～ 7 天并只执行首日决策，同时在窗口末端加入基于未来价格与负载的储能残值函数，就可以量化跨日套利收益并进一步降低边界效应。

（2）把确定性等价升级为显式随机规划。本文只在单时段获得 0.2 分位启发；若显式建立负载—光伏联合情景，可进一步构造情景抽样（SAA）或条件风险价值（CVaR）模型，直接刻画跨时段风险，而无需把局部分位结论近似推广到全年。

（3）引入更完整的价格预测。可在电价预测中加入日内周期项、工作日 / 节假日效应与温度特征，或采用分位数回归直接给出价格分布，从而把电价不确定性也纳入目标函数而非仅做信息集分层。

（4）细化储能模型。可把循环深度相关的寿命损耗、效率曲线、自放电与维护成本写入目标函数，并用历史运行数据重新标定参数；当模型规模增大时，可采用模型预测控制与 Benders 分解等算法保持滚动求解速度。

9.2 模型的推广

本文的“统一物理内核+ 信息集分层+ 可解释风险定价”框架不依赖本题数据。园区综合能源系统可将母线平衡扩展为电、热、气多能流平衡，并增加储热、储气状态[14]；电动汽车充换电站与工业用户侧储能可分别替换负载对象、增加最大需量费；含风光互补的微网可对各电源误差分别设置非对称损失。更一般地，库存、水库与供应链等“状态逐期递推且计划—执行存在偏差结算”的问题，也可复用本文的滚动优化、单因素消融和信息集验证方法。

AI 工具使用声明

本参赛队使用了 MathModel 内置 AI 助手（OpenAI GPT-5），用于审阅既有模型、协助修改 Python 求解与测试代码、分析报错和数值矛盾、生成绘图与 LaTeX 排版草稿，以及语言校订。团队向工具提供赛题、既有工程和阶段性审阅意见，逐项要求修正结算口径、滚动计划持续性、指定日期表格及提交材料。所有采纳内容均由人工复核；数值结

论由本地脚本实际运行，约束与费用逐时回代，并经自动测试和 Excel 回读核验。详细提示过程、采纳范围与人工核验见支撑材料 `AI 工具使用详情.pdf`。

参考文献

[1] Wu Hongbin，Liu Xingyue，Ding Ming. Dynamic economic dispatch of a microgrid：mathematical models and solution algorithm[J/OL]. International Journal of Electrical Power & Energy Systems，2014，63：336-346. http://dx.doi.org/10.1016/j.ijepes.2014.06.002.

[2] Nemati M，Braun M，Tenbohlen S. Optimization of unit commitment and economic dispatch in microgrids based on genetic algorithm and mixed integer linear programming[J/OL]. Applied Energy，2018，210：944-963. http://dx.doi.org/10.1016/j.apenergy.2017.07.007.

[3] Wu Zhi，Zeng Pingliang，Zhang X-P，et al. A solution to the chance-constrained two-stage stochastic program for unit commitment with wind energy integration[J/OL]. IEEE Transactions on Power Systems，2016，31（6）：4185-4196. http://dx.doi.org/10.1109/tpwrs.2015.2513395.

[4] 孙硕，杨仕友. 考虑光伏出力不确定性的分布鲁棒优化调度方法[J/OL]. 激光与光电子学进展，2025，62（9）：0925001. http://dx.doi.org/10.3788/lop241935.

[5] Zhang Yan，Fu Lijun，Zhu Wanlu，et al. Robust model predictive control for optimal energy management of island microgrids with uncertainties[J/OL]. Energy，2018，164：1229-1241. http://dx.doi.org/10.1016/j.energy.2018.08.200.

[6] Elkazaz M，Sumner M，Thomas D. Energy management system for hybrid PV-wind-battery microgrid using convex programming，model predictive and rolling horizon predictive control with experimental validation[J/OL]. International Journal of Electrical Power & Energy Systems，2020，115：105483. http://dx.doi.org/10.1016/j.ijepes.2019.105483.

[7] Silva V A，Aoki A R，Lambert-Torres G. Optimal day-ahead scheduling of microgrids with battery energy storage system[J/OL]. Energies，2020，13（19）：5188. http://dx.doi.org/10.3390/en13195188.

[8] Luo Liang，Abdulkareem S S，Rezvani A，et al. Optimal scheduling of a renewable based microgrid considering photovoltaic system and battery energy storage under uncertainty[J/OL]. Journal of Energy Storage，2020，28：101306. http://dx.doi.org/10.1016/j.est.2020.101306.

[9] Maheshwari A，Paterakis N G，Santarelli M，et al. Optimizing the operation of energy storage using a non-linear lithium-ion battery degradation model[J/OL]. Applied Energy，2020，261：114360. http://dx.doi.org/10.1016/j.apenergy.2019.114360.

[10] Qin Yan，Wang Ruoxuan，Vakharia A J，et al. The newsvendor problem：review and directions for future research[J/OL]. European Journal of Operational Research，2011，213（2）：361-374. http://dx.doi.org/10.1016/j.ejor.2010.11.024.

[11] Polat E，Güler M G，Ulukuş M Y. Renewable GenCo bidding strategy using newsvendor-based neural networks：an example from turkish electricity market[J/OL]. Electric Power Systems Research，2024，231：110301. http://dx.doi.org/10.1016/j.epsr.2024.110301.

[12] Markovics D，Mayer M J. Comparison of machine learning methods for photovoltaic power forecasting based on numerical weather prediction[J/OL]. Renewable and Sustainable Energy Reviews，2022，161：112364. http://dx.doi.org/10.1016/j.rser.2022.112364.

[13] Wang Qi，Zhang Chunyu，Ding Yi，et al. Review of real-time electricity markets for integrating distributed energy resources and demand response[J/OL]. Applied Energy，2015，138：695-706. http://dx.doi.org/10.1016/j.apenergy.2014.10.048.

[14] 刘继春，周春燕，高红均，et al. 考虑氢能-天然气混合储能的电-气综合能源微网日前经济调度优化[J/OL]. 电网技术，2018，42（1）：170-178. https://doi.org/10.13335/j.1000-3673.pst.2017.0966.

附录

附录 A　支撑材料文件列表

表 24　支撑材料文件清单

文件	说明
README.md	支撑材料目录、运行环境与唯一复现命令
__init__.py	Python 包标记，保证从项目根目录稳定导入
AI 工具使用详情.pdf	AI 工具名称、版本、使用环节与人工核验说明
results/	题面要求的五个最终结果工作簿，文件名与附件 5 一致
solve/	求解、报告生成、结果验证、制表与绘图 Python 源码
tests/	信息集因果性、计划持续性、结算、主结果和历史配置回归测试
tools/	代码同步、LaTeX 检查、论文编译、支撑材料打包与交付脚本
paper/	LaTeX 源码、参考文献、数值宏、矢量图、附录代码副本及编译后的论文
reports/	MODEL_AUDIT.md、model_audit_raw.json 与文献核验记录

附录 B　源程序代码

全部代码使用 Python 3.12 编写，线性规划和混合整数线性规划均由 SciPy 调用 HiGHS 求解。项目根目录的唯一测试命令为 `python -m pytest MathModel/tests -q --import-mode=importlib`；各问可用 `python -m MathModel.solve.<` 模块名 `>` 复现。以下按调用关系列出建模求解、验证与绘图的核心源码。为压缩篇幅，仅论文代码副本删除注释和文档字符串，本地源码不变。审计报告生成脚本 `report.py` 位于支撑材料的 `solve/` 目录；编译、检查和打包脚本位于 `tools/` 目录，均不改变模型求解逻辑。

B.1　物理参数与口径定义（params.py）

Listing 1:

```python
from __future__ import annotations
import math
from dataclasses import dataclass
from .io_data import DT_HOURS, SLOTS_PER_DAY
POWER_LIMIT_KW = 5000.0
ENERGY_LIMIT = POWER_LIMIT_KW * DT_HOURS
@dataclass(frozen=True)
class StorageParams:
    eta_ch: float = 0.9
    eta_dis: float = 0.9
    soc_min: float = 1200.0
```

```
    soc_max: float = 10800.0
    power_limit_kw: float = POWER_LIMIT_KW
    @property
    def energy_limit(self) -> float:
        return self.power_limit_kw * DT_HOURS
    @property
    def round_trip(self) -> float:
        return self.eta_ch * self.eta_dis
    def as_dict(self) -> dict:
        return {
            "eta_ch": self.eta_ch,
            "eta_dis": self.eta_dis,
            "round_trip_efficiency": self.round_trip,
            "soc_min": self.soc_min,
            "soc_max": self.soc_max,
            "power_limit_kw": self.power_limit_kw,
            "energy_limit_per_slot_kwh": self.energy_limit,
        }
DEFAULT_STORAGE = StorageParams()
SQRT_HALF = math.sqrt(0.9)
EFFICIENCY_VARIANTS: dict[str, StorageParams] = {
    "E1_eta_ch_0.90_eta_dis_0.90": StorageParams(0.9, 0.9),
    "E2_round_trip_0.90": StorageParams(SQRT_HALF, SQRT_HALF),
}
TERMINAL_POLICIES = ("daily_cycle", "free")
LOAD_MODES = ("perfect", "causal")
PRICE_MODES = ("perfect", "causal")
PV_RISK_MODES = ("asymmetric", "point")
SETTLEMENT_MODES = ("replacement", "additive")
```

B.2 数据读取与时间对齐（io_data.py）

Listing 2:

```
from __future__ import annotations
from dataclasses import dataclass
from datetime import date,datetime
from pathlib import Path
import numpy as np
import openpyxl
REPO_ROOT=Path(__file__).resolve().parents[2]
DEFAULT_DATA_ROOT=REPO_ROOT/"CUMCM2026Problems"/"C题"/"附件"
DT_HOURS=1.0/6.0; SLOTS_PER_DAY=144
@dataclass(frozen=True)
class DayData:
    labels:list[str]; price:np.ndarray; load_kw:np.ndarray; pv_kw:np.ndarray
```

```
@dataclass(frozen=True)
class AnnualData:
    dates:list[date]; load_kw:np.ndarray; pv_kw:np.ndarray
@dataclass(frozen=True)
class RollingForecastData:
    dates:list[date]; hourly_kw:np.ndarray
@dataclass(frozen=True)
class AnnualPriceData:
    dates:list[date]; price:np.ndarray
def _as_float_array(values:list[object],name:str)->np.ndarray:
    array=np.asarray(values,dtype=float)
    if array.shape!=(144,) or not np.isfinite(array).all(): raise ValueError(f"{name} invalid: {array.shape}")
    return array
def _coerce_date(value:object)->date:
    if isinstance(value,datetime): return value.date()
    if isinstance(value,date): return value
    text=str(value).strip()
    for fmt in ("%Y-%m-%d","%Y/%m/%d","%m/%d/%Y"):
        try: return datetime.strptime(text,fmt).date()
        except ValueError: pass
    raise ValueError(f"无法解析日期: {value!r}")
def read_attachment1(data_root:Path=DEFAULT_DATA_ROOT)->DayData:
    wb=openpyxl.load_workbook(Path(data_root)/"附件1.xlsx",data_only=True,read_only=True)
    rows=list(wb[wb.sheetnames[0]].iter_rows(min_row=2,max_row=145,min_col=1,max_col=4, values_only=True))
    labels=[str(r[0]) for r in rows]; price=_as_float_array([r[1] for r in rows],"price")
    load=_as_float_array([r[2] for r in rows],"load"); pv=_as_float_array([r[3] for r in rows],"pv")
    if (price<=0).any() or (load<0).any() or (pv<0).any(): raise ValueError("attachment1 bounds")
    return DayData(labels,price,load,pv)
def read_attachment2(data_root:Path=DEFAULT_DATA_ROOT)->AnnualData:
    wb=openpyxl.load_workbook(Path(data_root)/"附件2.xlsx",data_only=True,read_only=True)
    arrays=[]; date_lists=[]
    for sheet_name in wb.sheetnames[:2]:
        rows=list(wb[sheet_name].iter_rows(min_row=2,max_row=366,min_col=1,max_col=145, values_only=True))
        date_lists.append([_coerce_date(r[0]) for r in rows]); values=np.asarray([r[1:] for r in rows],dtype=float)
        if values.shape!=(365,144) or not np.isfinite(values).all() or (values<0).any(): raise ValueError(f"{sheet_name} invalid")
        arrays.append(values)
    if date_lists[0]!=date_lists[1]: raise ValueError("attachment2 date mismatch")
    return AnnualData(date_lists[0],arrays[0],arrays[1])
def read_attachment3(data_root:Path=DEFAULT_DATA_ROOT)->RollingForecastData:
    wb=openpyxl.load_workbook(Path(data_root)/"附件3.xlsx",data_only=True,read_only=True)
```

```
    rows=list(wb[wb.sheetnames[0]].iter_rows(min_row=2,max_row=1461,min_col=1,max_col=26,values_only=True))
    if len(rows)!=1460: raise ValueError("attachment3 must have 1460 forecast rows")
    dates=[]; blocks=[]; current_date=None
    expected=("0:00","6:00","12:00","18:00")
    for day in range(365):
        chunk=rows[day*4:(day+1)*4]
        current_date=_coerce_date(chunk[0][0])
        dates.append(current_date)
        issues=tuple(str(r[1]) for r in chunk)
        if issues!=expected: raise ValueError(f"attachment3 issue times invalid on {current_date}: {issues}")
        values=np.asarray([r[2:] for r in chunk],dtype=float)
        if values.shape!=(4,24) or not np.isfinite(values).all() or (values<0).any(): raise ValueError(f"attachment3 invalid {current_date}")
        blocks.append(values)
    return RollingForecastData(dates,np.stack(blocks))
def hourly_to_ten_min(hourly_kw:np.ndarray,method:str="step")->np.ndarray:
    hourly=np.asarray(hourly_kw,dtype=float)
    if hourly.shape!=(24,): raise ValueError("hourly forecast must have 24 values")
    if method=="step": return np.repeat(hourly,6)
    if method=="linear":
        return np.interp(np.arange(1,145)/6.0,np.arange(1,25),hourly,left=hourly[0],right=hourly[-1])
    raise ValueError(f"unknown downscale method: {method}")
def read_attachment4(data_root:Path=DEFAULT_DATA_ROOT)->AnnualPriceData:
    wb=openpyxl.load_workbook(Path(data_root)/"附件4.xlsx",data_only=True,read_only=True)
    rows=list(wb[wb.sheetnames[0]].iter_rows(min_row=2,max_row=366,min_col=1,max_col=145,values_only=True))
    dates=[_coerce_date(r[0]) for r in rows]; price=np.asarray([r[1:] for r in rows],dtype=float)
    if price.shape!=(365,144) or not np.isfinite(price).all() or (price<=0).any(): raise ValueError("attachment4 invalid")
    return AnnualPriceData(dates,price)
def template_path(name:str,data_root:Path=DEFAULT_DATA_ROOT)->Path:
    path=Path(data_root)/"附件5"/name
    if not path.exists(): raise FileNotFoundError(path)
    return path
```

B.3 公共线性规划内核（lp_core.py）

Listing 3:

```
from __future__ import annotations
from dataclasses import dataclass
from time import perf_counter
```

```
import numpy as np
from scipy.optimize import Bounds, LinearConstraint, linprog, milp
from .io_data import DT_HOURS
from .params import DEFAULT_STORAGE, ENERGY_LIMIT, StorageParams
ETA_CH = DEFAULT_STORAGE.eta_ch
ETA_DIS = DEFAULT_STORAGE.eta_dis
SOC_MIN = DEFAULT_STORAGE.soc_min
SOC_MAX = DEFAULT_STORAGE.soc_max
POWER_LIMIT_KW = DEFAULT_STORAGE.power_limit_kw
@dataclass
class DispatchResult:
    grid: np.ndarray
    charge: np.ndarray
    discharge: np.ndarray
    soc: np.ndarray
    curtail: np.ndarray
    objective: float
    primary_objective: float
    curtail_total: float
    throughput_total: float
    status: int
    message: str
    nit: int
    solve_seconds: float
    equality_marginals: np.ndarray
def _slices(t: int):
    return slice(0, t), slice(t, 2 * t), slice(2 * t, 3 * t), slice(3 * t, 4 * t), slice(4 *
        t, 5 * t)
def _solve(c, a_eq, b_eq, bounds, a_ub=None, b_ub=None):
    return linprog(
        c,
        A_ub=a_ub,
        b_ub=b_ub,
        A_eq=a_eq,
        b_eq=b_eq,
        bounds=bounds,
        method="highs",
        options={"primal_feasibility_tolerance": 1e-9, "dual_feasibility_tolerance": 1e-9},
    )
def _solve_milp(c, a_eq, b_eq, bounds, integrality, a_ub=None, b_ub=None):
    lower = np.asarray([(-np.inf if lo is None else lo) for lo, _ in bounds], dtype=float)
    upper = np.asarray([(np.inf if hi is None else hi) for _, hi in bounds], dtype=float)
    constraints = [LinearConstraint(np.asarray(a_eq), np.asarray(b_eq), np.asarray(b_eq))]
    if a_ub is not None and len(a_ub):
        constraints.append(
            LinearConstraint(
                np.asarray(a_ub, dtype=float),
```

```
                np.full(len(b_ub), -np.inf, dtype=float),
                np.asarray(b_ub, dtype=float),
            )
        )
    return milp(
        np.asarray(c, dtype=float),
        integrality=np.asarray(integrality, dtype=int),
        bounds=Bounds(lower, upper),
        constraints=constraints,
        options={"presolve": True, "mip_rel_gap": 1e-10},
    )
SLACK_ABS = 1e-5
SLACK_REL = 1e-10
def _lexicographic_slack(value: float, factor: float = 1.0) -> float:
    return factor * max(SLACK_ABS, SLACK_REL * max(1.0, abs(value)))
def _lexicographic(objectives, a_eq, b_eq, bounds, extra_rows=None, extra_rhs=None,
    slack_factor=1.0):
    base_rows = [np.asarray(r, dtype=float) for r in (extra_rows or [])]
    base_rhs = [float(v) for v in (extra_rhs or [])]
    rows = list(base_rows)
    rhs = list(base_rhs)
    x = None
    result = None
    degraded = False
    values: list[float] = []
    for index, c in enumerate(objectives):
        solution = None
        for factor, keep in ((1.0, None), (100.0, None), (1.0, 1)):
            if keep is None:
                sel_rows, sel_rhs = rows, rhs
            else:
                sel_rows, sel_rhs = rows[-keep:], rhs[-keep:]
            a_ub = np.asarray(sel_rows) if sel_rows else None
            b_ub = np.asarray(sel_rhs) if sel_rhs else None
            solution = _solve(c, a_eq, b_eq, bounds, a_ub, b_ub)
            if solution.success:
                if factor != 1.0 or (keep is not None and len(rows) > keep):
                    degraded = True
                break
        if solution is None or not solution.success:
            if x is None:
                raise RuntimeError(f"lexicographic LP stage {index} failed: {solution.
                    message}")
            degraded = True
            break
        x = solution.x
        result = solution
```

```
95          value = float(solution.fun)
96          values.append(value)
97          rows.append(np.asarray(c, dtype=float))
98          rhs.append(value + _lexicographic_slack(value, slack_factor))
99      return x, result, degraded, values
100 def _milp_lexicographic(objectives, a_eq, b_eq, bounds, integrality, extra_rows=None,
        extra_rhs=None):
101     rows = [np.asarray(r, dtype=float) for r in (extra_rows or [])]
102     rhs = [float(v) for v in (extra_rhs or [])]
103     values: list[float] = []
104     result = None
105     for index, objective in enumerate(objectives):
106         result = _solve_milp(
107             objective,
108             a_eq,
109             b_eq,
110             bounds,
111             integrality,
112             np.asarray(rows) if rows else None,
113             np.asarray(rhs) if rhs else None,
114         )
115         if not result.success:
116             raise RuntimeError(f"lexicographic MILP stage {index} failed: {result.message}")
117         value = float(np.dot(objective, result.x))
118         values.append(value)
119         rows.append(np.asarray(objective, dtype=float))
120         rhs.append(value + _lexicographic_slack(value, 10.0))
121     return result.x, result, values
122 def _validate(load_kw, pv_kw, price, storage: StorageParams, soc_initial, soc_terminal):
123     load_kw = np.asarray(load_kw, dtype=float)
124     pv_kw = np.asarray(pv_kw, dtype=float)
125     price = np.asarray(price, dtype=float)
126     if not (load_kw.shape == pv_kw.shape == price.shape):
127         raise ValueError("load/pv/price shape mismatch")
128     t = len(price)
129     if t == 0 or not np.isfinite(np.r_[load_kw, pv_kw, price]).all():
130         raise ValueError("invalid input")
131     if (load_kw < 0).any() or (pv_kw < 0).any() or (price <= 0).any():
132         raise ValueError("physical bounds violated")
133     if not storage.soc_min <= soc_initial <= storage.soc_max:
134         raise ValueError("initial SOC invalid")
135     if soc_terminal is not None and not storage.soc_min <= soc_terminal <= storage.soc_max:
136         raise ValueError("terminal SOC invalid")
137     return load_kw, pv_kw, price, t
138 def _energy_balance_rows(a_eq, b_eq, t, load_kw, pv_kw, storage: StorageParams, soc_initial)
    :
139     g, ch, dis, soc, spill = _slices(t)
```

```
    for i in range(t):
        a_eq[i, g.start + i] = 1
        a_eq[i, ch.start + i] = -1
        a_eq[i, dis.start + i] = 1
        a_eq[i, spill.start + i] = -1
        b_eq[i] = (load_kw[i] - pv_kw[i]) * DT_HOURS
        row = t + i
        a_eq[row, soc.start + i] = 1
        a_eq[row, ch.start + i] = -storage.eta_ch
        a_eq[row, dis.start + i] = 1.0 / storage.eta_dis
        if i == 0:
            b_eq[row] = soc_initial
        else:
            a_eq[row, soc.start + i - 1] = -1
def solve_dispatch(
    load_kw: np.ndarray,
    pv_kw: np.ndarray,
    price: np.ndarray,
    *,
    soc_initial: float,
    soc_terminal: float | None,
    storage: StorageParams = DEFAULT_STORAGE,
) -> DispatchResult:
    load_kw, pv_kw, price, t = _validate(load_kw, pv_kw, price, storage, soc_initial,
        soc_terminal)
    g, ch, dis, soc, spill = _slices(t)
    n = 5 * t
    a_eq = np.zeros((2 * t + (soc_terminal is not None), n))
    b_eq = np.zeros(a_eq.shape[0])
    _energy_balance_rows(a_eq, b_eq, t, load_kw, pv_kw, storage, soc_initial)
    if soc_terminal is not None:
        a_eq[-1, soc.stop - 1] = 1
        b_eq[-1] = soc_terminal
    limit = storage.energy_limit
    bounds = (
        [(0, None)] * t
        + [(0, limit)] * t
        + [(0, limit)] * t
        + [(storage.soc_min, storage.soc_max)] * t
        + [(0, float(x) * DT_HOURS) for x in pv_kw]
    )
    primary_c = np.zeros(n)
    primary_c[g] = price
    throughput_c = np.zeros(n)
    throughput_c[ch] = 1
    throughput_c[dis] = 1
    spill_c = np.zeros(n)
```

```
    spill_c[spill] = 1
    started = perf_counter()
    x, tertiary, fell_back, values = _lexicographic(
        [primary_c, throughput_c, spill_c], a_eq, b_eq, bounds
    )
    primary_value = values[0]
    return DispatchResult(
        x[g].copy(),
        x[ch].copy(),
        x[dis].copy(),
        x[soc].copy(),
        x[spill].copy(),
        float(np.dot(price, x[g])),
        primary_value,
        float(x[spill].sum()),
        float(x[ch].sum() + x[dis].sum()),
        int(tertiary.status),
        str(tertiary.message),
        int(getattr(tertiary, "nit", -1)),
        perf_counter() - started,
        np.asarray(tertiary.eqlin.marginals, dtype=float),
    )
@dataclass
class AdjustmentDispatchResult:
    dispatch: DispatchResult
    increase: np.ndarray
    decrease: np.ndarray
    settlement_objective: float
    variable_objective: float
    adjustment_mask: np.ndarray
    settlement_mode: str
    plan_purchase_constant: float
    lexicographic_fallback: bool = False
    throughput_objective: float = 0.0
def settlement_cost(plan_grid: np.ndarray, adjusted_grid: np.ndarray, price: np.ndarray,
    mode: str) -> float:
    plan = np.asarray(plan_grid, dtype=float)
    adjusted = np.asarray(adjusted_grid, dtype=float)
    p = np.asarray(price, dtype=float)
    increase = np.maximum(adjusted - plan, 0.0)
    decrease = np.maximum(plan - adjusted, 0.0)
    if mode == "replacement":
        return float(np.dot(p, np.minimum(plan, adjusted) + 0.5 * decrease + 1.5 * increase)
            )
    if mode == "additive":
        return float(np.dot(p, plan) + np.dot(p, 0.5 * decrease + 1.5 * increase))
    raise ValueError(f"unknown settlement mode: {mode}")
```

```python
def solve_adjustment_dispatch(
    load_kw: np.ndarray,
    pv_kw: np.ndarray,
    price: np.ndarray,
    baseline_grid: np.ndarray,
    adjustment_mask: np.ndarray,
    *,
    soc_initial: float,
    soc_terminal: float | None,
    settlement_mode: str = "replacement",
    storage: StorageParams = DEFAULT_STORAGE,
) -> AdjustmentDispatchResult:
    if settlement_mode not in ("replacement", "additive"):
        raise ValueError(f"unknown settlement mode: {settlement_mode}")
    load_kw, pv_kw, price, t = _validate(load_kw, pv_kw, price, storage, soc_initial,
        soc_terminal)
    baseline = np.asarray(baseline_grid, dtype=float)
    mask = np.asarray(adjustment_mask, dtype=bool)
    if not (baseline.shape == mask.shape == (t,)):
        raise ValueError("adjustment shape mismatch")
    g, ch, dis, soc, spill = _slices(t)
    inc = slice(5 * t, 6 * t)
    dec = slice(6 * t, 7 * t)
    use_milp = settlement_mode == "additive"
    z = slice(7 * t, 8 * t) if use_milp else slice(7 * t, 7 * t)
    n = 8 * t if use_milp else 7 * t
    masked = np.flatnonzero(mask)
    a_eq = np.zeros((2 * t + (soc_terminal is not None) + len(masked), n))
    b_eq = np.zeros(a_eq.shape[0])
    _energy_balance_rows(a_eq, b_eq, t, load_kw, pv_kw, storage, soc_initial)
    if soc_terminal is not None:
        last = a_eq.shape[0] - 1 - len(masked)
        a_eq[last, soc.stop - 1] = 1
        b_eq[last] = soc_terminal
    limit = storage.energy_limit
    bounds = (
        [(0, None)] * t
        + [(0, limit)] * t
        + [(0, limit)] * t
        + [(storage.soc_min, storage.soc_max)] * t
        + [(0, float(x) * DT_HOURS) for x in pv_kw]
        + [((0, None) if flag else (0, 0)) for flag in mask]
        + [((0, None) if flag else (0, 0)) for flag in mask]
        + ([(0, 1)] * t if use_milp else [])
    )
    for k, i in enumerate(masked):
        row = 2 * t + (1 if soc_terminal is not None else 0) + k
```

```
        a_eq[row, g.start + i] = 1.0
        a_eq[row, inc.start + i] = -1.0
        a_eq[row, dec.start + i] = 1.0
        b_eq[row] = baseline[i]
    adjustment_rows: list[np.ndarray] = []
    adjustment_rhs: list[float] = []
    primary_c = np.zeros(n)
    if settlement_mode == "replacement":
        primary_c[g] = price
        primary_c[inc] = np.where(mask, 0.5 * price, 0.0)
        primary_c[dec] = np.where(mask, 0.5 * price, 0.0)
        constant = 0.0
    else:
        primary_c[inc] = np.where(mask, 1.5 * price, 0.0)
        primary_c[dec] = np.where(mask, 0.5 * price, 0.0)
        constant = float(np.sum(price[mask] * baseline[mask]))
    throughput_c = np.zeros(n)
    throughput_c[ch] = 1
    throughput_c[dis] = 1
    throughput_c[inc] = 1
    throughput_c[dec] = 1
    spill_c = np.zeros(n)
    spill_c[spill] = 1
    started = perf_counter()
    if use_milp:
        mutual_rows = np.zeros((2 * t, n))
        mutual_rhs = np.zeros(2 * t)
        for i in range(t):
            mutual_rows[2 * i, ch.start + i] = 1.0
            mutual_rows[2 * i, z.start + i] = -limit
            mutual_rows[2 * i + 1, dis.start + i] = 1.0
            mutual_rows[2 * i + 1, z.start + i] = limit
            mutual_rhs[2 * i + 1] = limit
        integrality = np.zeros(n, dtype=int)
        integrality[z] = 1
        tertiary = _solve_milp(
            primary_c,
            a_eq,
            b_eq,
            bounds,
            integrality,
            mutual_rows,
            mutual_rhs,
        )
        if not tertiary.success:
            raise RuntimeError(f"adjustment MILP failed: {tertiary.message}")
        x = tertiary.x
```

```
        values = [
            float(np.dot(primary_c, x)),
            float(np.dot(throughput_c, x)),
            float(np.dot(spill_c, x)),
        ]
        fell_back = False
    else:
        x, tertiary, fell_back, values = _lexicographic(
            [primary_c, throughput_c, spill_c],
            a_eq,
            b_eq,
            bounds,
            adjustment_rows,
            adjustment_rhs,
        )
    primary_value = values[0]
    adjusted = x[g].copy()
    increase_vec = x[inc].copy()
    decrease_vec = x[dec].copy()
    if settlement_mode == "replacement":
        settlement = float(np.dot(primary_c, x))
    else:
        settlement = constant + float(np.dot(primary_c, x))
    dispatch = DispatchResult(
        adjusted,
        x[ch].copy(),
        x[dis].copy(),
        x[soc].copy(),
        x[spill].copy(),
        settlement,
        float(primary_value + constant),
        float(x[spill].sum()),
        float(x[ch].sum() + x[dis].sum()),
        int(tertiary.status),
        str(tertiary.message),
        int(getattr(tertiary, "nit", getattr(tertiary, "mip_node_count", -1)) or 0),
        perf_counter() - started,
        (
            np.zeros(a_eq.shape[0], dtype=float)
            if use_milp
            else np.asarray(tertiary.eqlin.marginals, dtype=float)
        ),
    )
    return AdjustmentDispatchResult(
        dispatch,
        increase_vec,
        decrease_vec,
```

```
        settlement,
        float(np.dot(primary_c, x)),
        mask.copy(),
        settlement_mode,
        constant,
        bool(fell_back),
        float(values[1]) if len(values) > 1 else float("nan"),
    )
```

B.4 光伏日前预测与风险定价（forecast.py）

Listing 4:

```
from __future__ import annotations
from dataclasses import dataclass
import numpy as np
from .io_data import DT_HOURS, SLOTS_PER_DAY
METHOD_WINDOWS = {"persistence": 1, "mean7": 7, "mean14": 14, "mean30": 30, "weighted14":
    14, "weighted30": 30}
QUANTILES = (0.80, 0.85, 0.90, 0.95)
@dataclass(frozen=True)
class ForecastDecision:
    day_index: int
    name: str
    method: str
    quantile: float
    margin_kw: float
    historical_score: float | None
    history_days_used: int
    forecast_kw: np.ndarray
    candidates: dict[str, np.ndarray]
    bases: dict[str, np.ndarray]
    margins: dict[str, float]
class OnlinePVForecaster:
    def __init__(
        self,
        price: np.ndarray,
        score_window: int = 30,
        residual_window: int = 30,
        risk_mode: str = "asymmetric",
        prior_pv: np.ndarray | None = None,
    ) -> None:
        if risk_mode not in ("asymmetric", "point"):
            raise ValueError(f"unknown risk_mode: {risk_mode}")
        self.price = np.asarray(price, dtype=float)
        if self.price.shape != (SLOTS_PER_DAY,):
```

```
            raise ValueError("price must contain 144 slots")
        self.prior_pv = None if prior_pv is None else np.asarray(prior_pv, dtype=float)
        self.score_window = score_window
        self.residual_window = residual_window
        self.risk_mode = risk_mode
        self.actual_history: list[np.ndarray] = []
        self.residual_days: dict[str, list[np.ndarray]] = {m: [] for m in METHOD_WINDOWS}
        self.score_days: dict[str, list[float]] = {self._name(m, q): [] for m in
            METHOD_WINDOWS for q in QUANTILES}
        if risk_mode == "point":
            self.score_days = {m: [] for m in METHOD_WINDOWS}
    @staticmethod
    def _name(method: str, q: float) -> str:
        return f"{method}_q{int(round(q * 100)):02d}"
    def _base(self, method: str) -> np.ndarray:
        if not self.actual_history:
            return np.zeros(SLOTS_PER_DAY) if self.prior_pv is None else np.clip(self.
                prior_pv, 0.0, None)
        n = min(METHOD_WINDOWS[method], len(self.actual_history))
        recent = np.stack(self.actual_history[-n:])
        if method == "persistence":
            return recent[-1].copy()
        if method.startswith("weighted"):
            return np.average(recent, axis=0, weights=np.arange(1, n + 1, dtype=float))
        return recent.mean(axis=0)
    def _margin(self, method: str, q: float) -> float:
        days = self.residual_days[method][-self.residual_window :]
        if not days:
            return 0.0
        residual = np.concatenate(days)
        return 0.0 if residual.size == 0 else max(0.0, float(np.quantile(residual, q)))
    def predict(self) -> ForecastDecision:
        d = len(self.actual_history)
        bases = {m: self._base(m) for m in METHOD_WINDOWS}
        candidates: dict[str, np.ndarray] = {}
        margins: dict[str, float] = {}
        if self.risk_mode == "asymmetric":
            for method, base in bases.items():
                for q in QUANTILES:
                    name = self._name(method, q)
                    margins[name] = self._margin(method, q)
                    candidates[name] = np.clip(base - margins[name], 0.0, None)
            default = self._name("persistence", 0.80)
            quantile = 0.80
        else:
            for method, base in bases.items():
                margins[method] = 0.0
```

```
78                  candidates[method] = np.clip(base, 0.0, None)
79              default = "persistence"
80              quantile = 0.0
81          eligible = [n for n, s in self.score_days.items() if s]
82          if d < 7 or not eligible:
83              selected = default
84              hist = None
85          else:
86              scores = {n: float(np.mean(self.score_days[n][-self.score_window :])) for n in
                    eligible}
87              selected = min(scores, key=lambda n: (scores[n], n))
88              hist = scores[selected]
89          if self.risk_mode == "asymmetric":
90              method, q_text = selected.rsplit("_q", 1)
91              q = int(q_text) / 100.0
92          else:
93              method = selected
94              q = 0.0
95          return ForecastDecision(
96              d,
97              selected,
98              method,
99              q if self.risk_mode == "asymmetric" else 0.0,
100             margins[selected],
101             hist,
102             d,
103             candidates[selected].copy(),
104             candidates,
105             bases,
106             margins,
107         )
108     def observe(self, decision: ForecastDecision, actual_kw: np.ndarray, price: np.ndarray |
          None = None) -> dict[str, float]:
109         actual = np.asarray(actual_kw, dtype=float)
110         if actual.shape != (SLOTS_PER_DAY,) or decision.day_index != len(self.actual_history
              ):
111             raise ValueError("forecast observation out of sequence")
112         score_price = self.price if price is None else np.asarray(price, dtype=float)
113         if score_price.shape != (SLOTS_PER_DAY,):
114             raise ValueError("score price must contain 144 slots")
115         result: dict[str, float] = {}
116         for name, forecast in decision.candidates.items():
117             error = forecast - actual
118             if self.risk_mode == "asymmetric":
119                 score = float(np.sum(score_price * (4 * np.maximum(error, 0) + np.maximum(-
                        error, 0))) * DT_HOURS)
120             else:
```

```
                score = float(np.mean(np.abs(error)))
            self.score_days[name].append(score)
            result[name] = score
        for method, base in decision.bases.items():
            mask = (actual > 1e-9) | (base > 1e-9)
            self.residual_days[method].append((base - actual)[mask].copy())
        self.actual_history.append(actual.copy())
        return result
```

B.5 因果负载预测（load_forecast.py）

Listing 5:

```
from __future__ import annotations
from dataclasses import dataclass
import numpy as np
from .io_data import DT_HOURS, SLOTS_PER_DAY
METHOD_WINDOWS: dict[str, int] = {
    "persistence": 1,
    "mean7": 7,
    "mean14": 14,
    "mean30": 30,
    "weighted14": 14,
    "weighted30": 30,
    "weekday_mean": 28,
}
BASE_METHODS = tuple(METHOD_WINDOWS)
NODE_SLOTS = (0, 36, 72, 108)
@dataclass(frozen=True)
class LoadForecastDecision:
    day_index: int
    node: int
    name: str
    method: str
    corrected: bool
    offset_kw: float
    historical_score: float | None
    history_days_used: int
    horizon: int
    forecast_kw: np.ndarray
    candidates: dict[str, np.ndarray]
    weekday_of_target: int
def _daily_window_shape(history: list[np.ndarray], method: str, weekday: int) -> np.ndarray:
    if not history:
        return np.zeros(SLOTS_PER_DAY, dtype=float)
    if method == "weekday_mean":
```

```
        same = [h for h, w in history if w == weekday]
        if len(same) >= 2:
            recent = np.stack(same[-4:])
            return recent.mean(axis=0)
        method = "mean7"
    n = min(METHOD_WINDOWS[method], len(history))
    recent = np.stack([h for h, _ in history[-n:]])
    if method == "persistence":
        return recent[-1].copy()
    if method.startswith("weighted"):
        weights = np.arange(1, n + 1, dtype=float)
        return np.average(recent, axis=0, weights=weights)
    return recent.mean(axis=0)
class OnlineLoadForecaster:
    def __init__(self, score_window: int = 30, prior_kw: np.ndarray | None = None) -> None:
        self.score_window = score_window
        self.history: list[tuple[np.ndarray, int]] = []
        self.prior_days = 0
        if prior_kw is not None:
            prior = np.asarray(prior_kw, dtype=float)
            if prior.shape != (SLOTS_PER_DAY,):
                raise ValueError("prior load must have 144 slots")
            self.history.append((prior.copy(), -1))
            self.prior_days = 1
        self.partial: np.ndarray = np.zeros(SLOTS_PER_DAY, dtype=float)
        self.partial_len: int = 0
        self.score_days: dict[tuple[int, str], list[float]] = {}
    @property
    def observed_days(self) -> int:
        return len(self.history) - self.prior_days
    def _target_weekday(self, day_index: int, day_offset: int) -> int:
        return (day_index + day_offset) % 7
    def _base_horizon(self, method: str, day_index: int, node: int, horizon: int) -> np.
        ndarray:
        wd_today = self._target_weekday(day_index, 0)
        shape_today = _daily_window_shape(self.history, method, wd_today)
        head = shape_today[node:]
        if len(head) >= horizon:
            return head[:horizon].copy()
        wd_next = self._target_weekday(day_index, 1)
        shape_next = _daily_window_shape(self.history, method, wd_next)
        tail = horizon - len(head)
        while len(shape_next) < tail:
            shape_next = np.concatenate([shape_next, shape_next])
        return np.concatenate([head, shape_next[:tail]])
    def _offset(self, method: str, day_index: int, node: int) -> float:
        if node <= 0 or self.partial_len <= 0:
```

```
80              return 0.0
81          k = min(36, node, self.partial_len)
82          if k < 6:
83              return 0.0
84          observed = self.partial[node - k : node]
85          base = _daily_window_shape(self.history, method, self._target_weekday(day_index, 0))
                [node - k : node]
86          return float(np.mean(observed - base))
87      def _candidate_names(self, node: int) -> tuple[str, ...]:
88          if node == 0:
89              return BASE_METHODS
90          return tuple(f"{m}_{suffix}" for m in BASE_METHODS for suffix in ("raw", "offset"))
91      def _build(self, name: str, day_index: int, node: int, horizon: int) -> tuple[np.ndarray
            , str, bool, float]:
92          if name.endswith("_offset"):
93              method, corrected = name[: -len("_offset")], True
94          elif name.endswith("_raw"):
95              method, corrected = name[: -len("_raw")], False
96          else:
97              method, corrected = name, False
98          base = self._base_horizon(method, day_index, node, horizon)
99          offset = self._offset(method, day_index, node) if corrected else 0.0
100         return np.clip(base + offset, 0.0, None), method, corrected, offset
101     def predict(self, day_index: int, node: int = 0, horizon: int = SLOTS_PER_DAY) ->
            LoadForecastDecision:
102         if node not in NODE_SLOTS:
103             raise ValueError(f"node must be one of {NODE_SLOTS}")
104         names = self._candidate_names(node)
105         candidates: dict[str, np.ndarray] = {}
106         meta: dict[str, tuple[str, bool, float]] = {}
107         for name in names:
108             forecast, method, corrected, offset = self._build(name, day_index, node, horizon
                    )
109             candidates[name] = forecast
110             meta[name] = (method, corrected, offset)
111         eligible = [n for n in names if self.score_days.get((node, n))]
112         history_days = self._score_history_days(node)
113         default = "persistence" if node == 0 else "persistence_raw"
114         if history_days < 7 or not eligible:
115             selected, hist = default, None
116         else:
117             scores = {
118                 n: float(np.mean(self.score_days[(node, n)][-self.score_window :])) for n in
                        eligible
119             }
120             selected = min(scores, key=lambda n: (scores[n], n))
121             hist = scores[selected]
```

```
122         method, corrected, offset = meta[selected]
123         return LoadForecastDecision(
124             day_index=day_index,
125             node=node,
126             name=selected,
127             method=method,
128             corrected=corrected,
129             offset_kw=offset,
130             historical_score=hist,
131             history_days_used=self.observed_days,
132             horizon=horizon,
133             forecast_kw=candidates[selected].copy(),
134             candidates=candidates,
135             weekday_of_target=self._target_weekday(day_index, 0),
136         )
137     def _score_history_days(self, node: int) -> int:
138         return max((len(v) for (n, _), v in self.score_days.items() if n == node), default
                =0)
139     def observe_block(self, decision: LoadForecastDecision, actual_block_kw: np.ndarray) ->
            dict[str, float]:
140         actual = np.asarray(actual_block_kw, dtype=float)
141         if actual.ndim != 1:
142             raise ValueError("actual block must be 1-D")
143         out: dict[str, float] = {}
144         for name, forecast in decision.candidates.items():
145             segment = forecast[: len(actual)]
146             mae = float(np.mean(np.abs(segment - actual)))
147             self.score_days.setdefault((decision.node, name), []).append(mae)
148             out[name] = mae
149         return out
150     def observe_partial(self, node: int, actual_slots: np.ndarray) -> None:
151         if actual_slots.shape != (node,) and node != 0:
152             raise ValueError("partial block shape mismatch")
153         self.partial[:node] = actual_slots
154         self.partial_len = node
155     def commit_day(self, actual_full_day_kw: np.ndarray, day_index: int) -> None:
156         actual = np.asarray(actual_full_day_kw, dtype=float)
157         if actual.shape != (SLOTS_PER_DAY,):
158             raise ValueError("daily load must have 144 slots")
159         self.history.append((actual.copy(), self._target_weekday(day_index, 0)))
160         self.partial = np.zeros(SLOTS_PER_DAY, dtype=float)
161         self.partial_len = 0
162     def log_row(self, decision: LoadForecastDecision, actual_block_kw: np.ndarray, date_text
            : str) -> dict:
163         actual = np.asarray(actual_block_kw, dtype=float)
164         forecast = decision.forecast_kw[: len(actual)]
165         error = forecast - actual
```

```
        slot = int(decision.node * 10) // 60
        minute = (decision.node * 10) % 60
        label = f"{slot:02d}:{minute:02d}"
        return {
            "date": date_text,
            "decision_time": label,
            "history_days_used": decision.history_days_used,
            "candidate": decision.name,
            "method": decision.method,
            "offset_corrected": decision.corrected,
            "offset_kw": decision.offset_kw,
            "mae_kw": float(np.mean(np.abs(error))),
            "rmse_kw": float(np.sqrt(np.mean(error**2))),
            "daily_energy_error_kwh": float(np.sum(error) * DT_HOURS),
            "block_energy_error_kwh": float(np.sum(error) * DT_HOURS),
            "historical_score": decision.historical_score,
        }
```

B.6 因果电价预测（price_forecast.py）

Listing 6:

```
from __future__ import annotations
from dataclasses import dataclass
import numpy as np
from .io_data import DT_HOURS, SLOTS_PER_DAY
from .load_forecast import NODE_SLOTS
METHOD_WINDOWS: dict[str, int] = {
    "persistence": 1,
    "mean7": 7,
    "mean14": 14,
    "mean30": 30,
    "weighted14": 14,
    "weighted30": 30,
    "weekday_mean": 28,
}
BASE_METHODS = tuple(METHOD_WINDOWS)
@dataclass(frozen=True)
class PriceForecastDecision:
    day_index: int
    node: int
    name: str
    method: str
    historical_score: float | None
    history_days_used: int
    horizon: int
```

```
    forecast_price: np.ndarray
    candidates: dict[str, np.ndarray]
def _window_shape(history: list[tuple[np.ndarray, int]], method: str, weekday: int) -> np.
    ndarray:
    if not history:
        return np.full(SLOTS_PER_DAY, np.nan, dtype=float)
    if method == "weekday_mean":
        same = [h for h, w in history if w == weekday]
        if len(same) >= 2:
            return np.stack(same[-4:]).mean(axis=0)
        method = "mean7"
    n = min(METHOD_WINDOWS[method], len(history))
    recent = np.stack([h for h, _ in history[-n:]])
    if method == "persistence":
        return recent[-1].copy()
    if method.startswith("weighted"):
        weights = np.arange(1, n + 1, dtype=float)
        return np.average(recent, axis=0, weights=weights)
    return recent.mean(axis=0)
class OnlinePriceForecaster:
    def __init__(self, score_window: int = 30, prior_price: np.ndarray | None = None) ->
        None:
        self.score_window = score_window
        self.history: list[tuple[np.ndarray, int]] = []
        self.prior_days = 0
        if prior_price is not None:
            prior = np.asarray(prior_price, dtype=float)
            if prior.shape != (SLOTS_PER_DAY,):
                raise ValueError("prior price must have 144 slots")
            self.history.append((prior.copy(), -1))
            self.prior_days = 1
        self.partial_len: int = 0
        self.score_days: dict[tuple[int, str], list[float]] = {}
    @property
    def observed_days(self) -> int:
        return len(self.history) - self.prior_days
    def _weekday(self, day_index: int, offset: int) -> int:
        return (day_index + offset) % 7
    def _base_horizon(self, method: str, day_index: int, node: int, horizon: int) -> np.
        ndarray:
        head = _window_shape(self.history, method, self._weekday(day_index, 0))[node:]
        if len(head) >= horizon:
            return head[:horizon].copy()
        tail_len = horizon - len(head)
        tail = _window_shape(self.history, method, self._weekday(day_index, 1))
        while len(tail) < tail_len:
            tail = np.concatenate([tail, tail])
```

```
69          return np.concatenate([head, tail[:tail_len]])
70      def _fallback(self) -> np.ndarray:
71          if not self.history:
72              return np.full(SLOTS_PER_DAY, 0.5, dtype=float)
73          return self.history[-1][0].copy()
74      def predict_horizon(self, day_index: int, node: int = 0, horizon: int = SLOTS_PER_DAY)
            -> PriceForecastDecision:
75          if node not in NODE_SLOTS:
76              raise ValueError(f"node must be one of {NODE_SLOTS}")
77          candidates: dict[str, np.ndarray] = {}
78          for method in BASE_METHODS:
79              horizon_values = self._base_horizon(method, day_index, node, horizon)
80              if not np.isfinite(horizon_values).all():
81                  horizon_values = np.where(np.isfinite(horizon_values), horizon_values, self.
                        _fallback()[node % SLOTS_PER_DAY])
82              candidates[method] = np.clip(horizon_values, 1e-6, None)
83          eligible = [n for n in BASE_METHODS if self.score_days.get((node, n))]
84          history_days = max((len(v) for (n, _), v in self.score_days.items() if n == node),
                default=0)
85          if history_days < 7 or not eligible:
86              selected, hist = "persistence", None
87          else:
88              scores = {n: float(np.mean(self.score_days[(node, n)][-self.score_window :]))
                    for n in eligible}
89              selected = min(scores, key=lambda n: (scores[n], n))
90              hist = scores[selected]
91          return PriceForecastDecision(
92              day_index=day_index,
93              node=node,
94              name=selected,
95              method=selected,
96              historical_score=hist,
97              history_days_used=self.observed_days,
98              horizon=horizon,
99              forecast_price=candidates[selected].copy(),
100             candidates=candidates,
101         )
102     def observe_block(self, decision: PriceForecastDecision, actual_block: np.ndarray) ->
            dict[str, float]:
103         actual = np.asarray(actual_block, dtype=float)
104         out: dict[str, float] = {}
105         for name, forecast in decision.candidates.items():
106             segment = forecast[: len(actual)]
107             mae = float(np.mean(np.abs(segment - actual)))
108             self.score_days.setdefault((decision.node, name), []).append(mae)
109             out[name] = mae
110         return out
```

```
    def observe_partial(self, node: int) -> None:
        self.partial_len = node
    def commit_day(self, actual_full_day: np.ndarray, day_index: int) -> None:
        actual = np.asarray(actual_full_day, dtype=float)
        if actual.shape != (SLOTS_PER_DAY,):
            raise ValueError("daily price must have 144 slots")
        self.history.append((actual.copy(), self._weekday(day_index, 0)))
        self.partial_len = 0
    def log_row(self, decision: PriceForecastDecision, actual_block: np.ndarray, date_text:
        str) -> dict:
        actual = np.asarray(actual_block, dtype=float)
        forecast = decision.forecast_price[: len(actual)]
        error = forecast - actual
        slot = int(decision.node * 10) // 60
        minute = (decision.node * 10) % 60
        return {
            "date": date_text,
            "decision_time": f"{slot:02d}:{minute:02d}",
            "history_days_used": decision.history_days_used,
            "method": decision.method,
            "price_mae": float(np.mean(np.abs(error))),
            "price_rmse": float(np.sqrt(np.mean(error**2))),
            "price_energy_weighted_error": float(
                np.sum(np.abs(error) * actual) / max(np.sum(actual), 1e-9)
            ),
            "daily_price_energy_weighted_error": float(
                np.sum(np.abs(error) * actual) * DT_HOURS
            ),
            "historical_score": decision.historical_score,
        }
```

B.7 实时储能反馈与执行（settle.py）

Listing 7:

```
from __future__ import annotations
from dataclasses import dataclass
import numpy as np
from .io_data import DT_HOURS
from .lp_core import DispatchResult
from .params import DEFAULT_STORAGE, StorageParams
TOL = 1e-10
@dataclass(frozen=True)
class ExecutionResult:
    grid: np.ndarray
    charge: np.ndarray
```

```
    discharge: np.ndarray
    soc: np.ndarray
    curtail: np.ndarray
    emergency: np.ndarray
def execute_with_realtime_storage_feedback(
    plan: DispatchResult,
    load_kw: np.ndarray,
    actual_pv_kw: np.ndarray,
    *,
    soc_initial: float,
    storage: StorageParams = DEFAULT_STORAGE,
) -> ExecutionResult:
    load = np.asarray(load_kw, dtype=float)
    pv = np.asarray(actual_pv_kw, dtype=float)
    n = len(load)
    if pv.shape != load.shape or len(plan.grid) != n:
        raise ValueError("execution shape mismatch")
    limit = storage.energy_limit
    c_out = np.zeros(n)
    h_out = np.zeros(n)
    soc = np.zeros(n)
    spill = np.zeros(n)
    emergency = np.zeros(n)
    state = float(soc_initial)
    for t in range(n):
        c = min(float(plan.charge[t]), limit, max(0.0, (storage.soc_max - state) / storage.
            eta_ch))
        h = min(float(plan.discharge[t]), limit, max(0.0, (state - storage.soc_min) *
            storage.eta_dis))
        net = float(plan.grid[t] + pv[t] * DT_HOURS + h - load[t] * DT_HOURS - c)
        if net < -TOL:
            deficit = -net
            reduce = min(c, deficit)
            c -= reduce
            deficit -= reduce
            max_h = max(0.0, (state + storage.eta_ch * c - storage.soc_min) * storage.
                eta_dis)
            add = min(deficit, limit - h, max(0.0, max_h - h))
            h += add
            deficit -= add
            emergency[t] = max(0.0, deficit)
        elif net > TOL:
            surplus = net
            reduce = min(h, surplus)
            h -= reduce
            surplus -= reduce
            max_c = max(0.0, (storage.soc_max - (state - h / storage.eta_dis)) / storage.
```

```
                eta_ch)
            add = min(surplus, limit - c, max(0.0, max_c - c))
            c += add
            surplus -= add
            spill[t] = max(0.0, surplus)
        state = state + storage.eta_ch * c - h / storage.eta_dis
        if not storage.soc_min - 1e-6 <= state <= storage.soc_max + 1e-6:
            raise RuntimeError(f"SOC invalid at {t}: {state}")
        c_out[t] = c
        h_out[t] = h
        soc[t] = min(storage.soc_max, max(storage.soc_min, state))
    return ExecutionResult(plan.grid.copy(), c_out, h_out, soc, spill, emergency)
def execute_fixed_storage_plan(
    plan: DispatchResult,
    load_kw: np.ndarray,
    actual_pv_kw: np.ndarray,
    *,
    soc_initial: float,
    storage: StorageParams = DEFAULT_STORAGE,
) -> ExecutionResult:
    load = np.asarray(load_kw, dtype=float)
    pv = np.asarray(actual_pv_kw, dtype=float)
    n = len(load)
    if pv.shape != load.shape or len(plan.grid) != n:
        raise ValueError("execution shape mismatch")
    limit = storage.energy_limit
    c_out = np.zeros(n)
    h_out = np.zeros(n)
    soc = np.zeros(n)
    spill = np.zeros(n)
    emergency = np.zeros(n)
    state = float(soc_initial)
    for t in range(n):
        c = min(float(plan.charge[t]), limit, max(0.0, (storage.soc_max - state) / storage.
            eta_ch))
        h = min(float(plan.discharge[t]), limit, max(0.0, (state - storage.soc_min) *
            storage.eta_dis))
        net = float(plan.grid[t] + pv[t] * DT_HOURS + h - load[t] * DT_HOURS - c)
        if net < 0:
            emergency[t] = -net
        elif net > 0:
            spill[t] = net
        state = state + storage.eta_ch * c - h / storage.eta_dis
        if not storage.soc_min - 1e-6 <= state <= storage.soc_max + 1e-6:
            raise RuntimeError(f"SOC invalid at {t}: {state}")
        c_out[t] = c
        h_out[t] = h
```

```
        soc[t] = min(storage.soc_max, max(storage.soc_min, state))
    return ExecutionResult(plan.grid.copy(), c_out, h_out, soc, spill, emergency)
def execute_plan(
    plan: DispatchResult,
    load_kw: np.ndarray,
    actual_pv_kw: np.ndarray,
    *,
    soc_initial: float,
    realtime_feedback: bool = True,
    storage: StorageParams = DEFAULT_STORAGE,
) -> ExecutionResult:
    if realtime_feedback:
        return execute_with_realtime_storage_feedback(
            plan, load_kw, actual_pv_kw, soc_initial=soc_initial, storage=storage
        )
    return execute_fixed_storage_plan(plan, load_kw, actual_pv_kw, soc_initial=soc_initial,
        storage=storage)
```

B.8 问题一求解（q1.py）

Listing 8:

```
from __future__ import annotations
import csv,json,platform,sys
from pathlib import Path
import numpy as np
import scipy
import openpyxl
from .export import export_result1
from .io_data import DEFAULT_DATA_ROOT,DT_HOURS,read_attachment1,template_path
from .lp_core import ETA_CH,ETA_DIS,SOC_MAX,SOC_MIN,solve_dispatch
ROOT=Path(__file__).resolve().parents[1]; OUTPUT=ROOT/"outputs"/"q1"
def run_q1(*,data_root:Path=DEFAULT_DATA_ROOT)->dict:
    data=read_attachment1(data_root)
    result=solve_dispatch(data.load_kw,data.pv_kw,data.price,soc_initial=6000,soc_terminal
        =6000)
    balance=result.grid+data.pv_kw*DT_HOURS+result.discharge-data.load_kw*DT_HOURS-result.
        charge-result.curtail
    previous=np.r_[6000.0,result.soc[:-1]]
    soc_res=result.soc-(previous+ETA_CH*result.charge-result.discharge/ETA_DIS)
    no_storage_grid=np.maximum((data.load_kw-data.pv_kw)*DT_HOURS,0)
    metrics={"cost_yuan":result.objective,"primary_cost_yuan":result.primary_objective,
        "primary_cost_gap_yuan":result.objective-result.primary_objective,
        "grid_energy_kwh":float(result.grid.sum()),"charge_energy_kwh":float(result.charge.
            sum()),
        "discharge_energy_kwh":float(result.discharge.sum()),"curtailment_energy_kwh":float(
```

```
            result.curtail.sum()),
        "no_storage_cost_yuan":float(np.dot(data.price,no_storage_grid)),
        "savings_yuan":float(np.dot(data.price,no_storage_grid)-result.objective),
        "savings_percent":float(100*(np.dot(data.price,no_storage_grid)-result.objective)/np.dot(data.price,no_storage_grid)),
        "initial_soc_kwh":6000.0,"terminal_soc_kwh":float(result.soc[-1]),
        "min_soc_kwh":float(result.soc.min()),"max_soc_kwh":float(result.soc.max()),
        "max_balance_residual":float(np.max(np.abs(balance))),"max_soc_residual":float(np.max(np.abs(soc_res))),
        "max_simultaneous_charge_discharge":float(np.max(np.minimum(result.charge,result.discharge))),
        "solver_status":result.status,"solver_message":result.message,"iterations":result.nit,
        "solve_seconds":result.solve_seconds}
    metrics["passed"]=bool(metrics["max_balance_residual"]<1e-7 and metrics["max_soc_residual"]<1e-7 and
        abs(result.soc[-1]-6000)<1e-7 and metrics["min_soc_kwh"]>=SOC_MIN-1e-7 and
        metrics["max_soc_kwh"]<=SOC_MAX+1e-7 and metrics["max_simultaneous_charge_discharge"]<1e-5)
    OUTPUT.mkdir(parents=True,exist_ok=True)
    with (OUTPUT/"solution.csv").open("w",newline="",encoding="utf-8-sig") as f:
        fields=["slot","label","price","load_kw","pv_kw","grid_kwh","charge_kwh","discharge_kwh","soc_kwh","curtail_kwh"]
        writer=csv.DictWriter(f,fieldnames=fields); writer.writeheader()
        for t in range(144):
            writer.writerow({"slot":t+1,"label":data.labels[t],"price":data.price[t],"load_kw":data.load_kw[t],
                "pv_kw":data.pv_kw[t],"grid_kwh":result.grid[t],"charge_kwh":result.charge[t],
                "discharge_kwh":result.discharge[t],"soc_kwh":result.soc[t],"curtail_kwh":result.curtail[t]})
    check=export_result1(template_path("result1.xlsx",data_root),OUTPUT/"result1.xlsx",result)
    metrics["xlsx_passed"]=check["passed"]
    (OUTPUT/"metrics.json").write_text(json.dumps(metrics,ensure_ascii=False,indent=2),encoding="utf-8")
    (OUTPUT/"xlsx_verification.json").write_text(json.dumps(check,ensure_ascii=False,indent=2),encoding="utf-8")
    env={"python":sys.version,"platform":platform.platform(),"numpy":np.__version__,"scipy":scipy.__version__,
         "openpyxl":openpyxl.__version__,"algorithm":"HiGHS lexicographic LP"}
    (OUTPUT/"environment.json").write_text(json.dumps(env,ensure_ascii=False,indent=2),encoding="utf-8")
    return {"metrics":metrics,"result":result}
if __name__=="__main__": print(json.dumps(run_q1()["metrics"],ensure_ascii=False,indent=2))
```

B.9 问题二求解（q2.py）

Listing 9:

```python
from __future__ import annotations
import csv
import json
import platform
import sys
from collections import Counter
from pathlib import Path
import numpy as np
import scipy
from .export import export_result2
from .forecast import QUANTILES, OnlinePVForecaster
from .io_data import DEFAULT_DATA_ROOT, DT_HOURS, read_attachment1, read_attachment2,
    template_path
from .load_forecast import OnlineLoadForecaster
from .lp_core import solve_dispatch
from .params import DEFAULT_STORAGE, StorageParams
from .price_forecast import OnlinePriceForecaster
from .settle import execute_plan
ROOT = Path(__file__).resolve().parents[1]
OUTPUT = ROOT / "outputs" / "q2"
def _verify_day(record: dict, storage: StorageParams) -> dict:
    execution = record["execution"]
    load = record["load_kw"]
    pv = record["actual_pv_kw"]
    balance = (
        execution.grid
        + pv * DT_HOURS
        + execution.discharge
        + execution.emergency
        - load * DT_HOURS
        - execution.charge
        - execution.curtail
    )
    previous = np.r_[record["soc_initial"], execution.soc[:-1]]
    soc_residual = execution.soc - (previous + storage.eta_ch * execution.charge - execution
        .discharge / storage.eta_dis)
    return {
        "max_balance_residual": float(np.max(np.abs(balance))),
        "max_soc_residual": float(np.max(np.abs(soc_residual))),
        "min_soc": float(execution.soc.min()),
        "max_soc": float(execution.soc.max()),
        "max_simultaneous": float(np.max(np.minimum(execution.charge, execution.discharge)))
            ,
```

```
    }
def _emergency_intervals(emergency: np.ndarray, threshold: float = 1e-7) -> int:
    active = emergency > threshold
    if not active.any():
        return 0
    return int(np.sum(active[1:] & ~active[:-1]) + (1 if active[0] else 0))
def run_q2(
    *,
    data_root: Path = DEFAULT_DATA_ROOT,
    max_days: int = 365,
    price_matrix: np.ndarray | None = None,
    output_dir: Path | None = None,
    template_name: str = "result2.xlsx",
    write_outputs: bool = True,
    load_mode: str = "causal",
    pv_risk_mode: str = "asymmetric",
    realtime_feedback: bool = True,
    terminal_policy: str = "daily_cycle",
    price_mode: str = "perfect",
    storage: StorageParams = DEFAULT_STORAGE,
    use_typical_prior: bool = True,
    tag: str = "",
) -> dict:
    if load_mode not in ("perfect", "causal"):
        raise ValueError(f"unknown load_mode: {load_mode}")
    if price_mode not in ("perfect", "causal"):
        raise ValueError(f"unknown price_mode: {price_mode}")
    if terminal_policy not in ("daily_cycle", "free"):
        raise ValueError(f"unknown terminal_policy: {terminal_policy}")
    day1 = read_attachment1(data_root)
    annual = read_attachment2(data_root)
    if max_days < 1 or max_days > 365:
        raise ValueError("max_days must be 1..365")
    prices = np.tile(day1.price, (365, 1)) if price_matrix is None else np.asarray(
        price_matrix, dtype=float)
    if prices.shape != (365, 144):
        raise ValueError("price matrix must have shape (365,144)")
    out_dir = OUTPUT if output_dir is None else Path(output_dir)
    prior_pv = day1.pv_kw if use_typical_prior else None
    prior_load = day1.load_kw if use_typical_prior else None
    prior_price = day1.price if use_typical_prior else None
    selector = OnlinePVForecaster(day1.price, risk_mode=pv_risk_mode, prior_pv=prior_pv)
    load_forecaster = OnlineLoadForecaster(prior_kw=prior_load)
    price_forecaster = OnlinePriceForecaster(prior_price=prior_price)
    state = 6000.0
    records: list[dict] = []
    forecast_rows: list[dict] = []
```

```
    load_rows: list[dict] = []
    price_rows: list[dict] = []
    verify_rows: list[dict] = []
    for d in range(max_days):
        initial = state
        load_decision = load_forecaster.predict(d, 0, 144)
        pv_decision = selector.predict()
        if pv_decision.history_days_used != d:
            raise RuntimeError("PV forecast causality counter mismatch")
        price_decision = price_forecaster.predict_horizon(d, 0, 144) if price_mode == "
            causal" else None
        load_plan = annual.load_kw[d] if load_mode == "perfect" else load_decision.
            forecast_kw
        price_plan = prices[d] if price_mode == "perfect" else price_decision.forecast_price
        soc_terminal = initial if terminal_policy == "daily_cycle" else None
        plan = solve_dispatch(
            load_plan,
            pv_decision.forecast_kw,
            price_plan,
            soc_initial=initial,
            soc_terminal=soc_terminal,
            storage=storage,
        )
        execution = execute_plan(
            plan,
            annual.load_kw[d],
            annual.pv_kw[d],
            soc_initial=initial,
            realtime_feedback=realtime_feedback,
            storage=storage,
        )
        plan_cost = float(np.dot(prices[d], plan.grid))
        emergency_cost = float(np.dot(5.0 * prices[d], execution.emergency))
        state = float(execution.soc[-1])
        record = {
            "day_index": d,
            "date": annual.dates[d],
            "soc_initial": initial,
            "plan": plan,
            "execution": execution,
            "load_kw": annual.load_kw[d],
            "actual_pv_kw": annual.pv_kw[d],
            "load_plan_kw": np.asarray(load_plan, dtype=float),
            "pv_plan_kw": pv_decision.forecast_kw,
            "price_plan": np.asarray(price_plan, dtype=float),
            "plan_cost": plan_cost,
            "emergency_cost": emergency_cost,
```

```
            "total_cost": plan_cost + emergency_cost,
            "load_decision": load_decision,
            "pv_decision": pv_decision,
            "adjusted_grid": execution.grid,
        }
        records.append(record)
        verify_rows.append(_verify_day(record, storage))
        forecast_row = {
                "day_index": d,
                "date": annual.dates[d].isoformat(),
                "history_days_used": pv_decision.history_days_used,
                "candidate": pv_decision.name,
                "method": pv_decision.method,
                "quantile": pv_decision.quantile,
                "margin_kw": pv_decision.margin_kw,
                "historical_score": pv_decision.historical_score,
                "pv_forecast_mae_kw": float(np.mean(np.abs(pv_decision.forecast_kw - annual.
                    pv_kw[d]))),
                "pv_forecast_rmse_kw": float(np.sqrt(np.mean((pv_decision.forecast_kw -
                    annual.pv_kw[d]) ** 2))),
                "pv_daily_energy_error_kwh": float(np.sum(pv_decision.forecast_kw - annual.
                    pv_kw[d]) * DT_HOURS),
            }
        forecast_rows.append(forecast_row)
        load_rows.append(load_forecaster.log_row(load_decision, annual.load_kw[d], annual.
            dates[d].isoformat()))
        if price_decision is not None:
            price_rows.append(
                price_forecaster.log_row(price_decision, prices[d], annual.dates[d].
                    isoformat())
            )
        candidate_scores = selector.observe(
            pv_decision, annual.pv_kw[d], price=np.asarray(price_plan, dtype=float)
        )
        if pv_risk_mode == "asymmetric":
            for quantile in QUANTILES:
                name = OnlinePVForecaster._name(pv_decision.method, quantile)
                forecast_row[f"risk_q{int(100 * quantile):02d}_yuan"] = candidate_scores[
                    name]
        load_forecaster.observe_block(load_decision, annual.load_kw[d])
        load_forecaster.commit_day(annual.load_kw[d], d)
        if price_decision is not None:
            price_forecaster.observe_block(price_decision, prices[d])
        price_forecaster.commit_day(prices[d], d)
    delivery = records[31:] if max_days == 365 else []
    min_soc = min(r["min_soc"] for r in verify_rows)
    max_soc = max(r["max_soc"] for r in verify_rows)
```

```
    max_balance = max(r["max_balance_residual"] for r in verify_rows)
    max_soc_res = max(r["max_soc_residual"] for r in verify_rows)
    max_sim = max(r["max_simultaneous"] for r in verify_rows)
    continuity = (
        max(abs(records[i]["soc_initial"] - records[i - 1]["execution"].soc[-1]) for i in
            range(1, len(records)))
        if len(records) > 1
        else 0.0
    )
    metrics = {
        "case_tag": tag,
        "days_simulated": max_days,
        "warmup_days": 31 if max_days == 365 else None,
        "delivery_days": len(delivery),
        "load_mode": load_mode,
        "pv_risk_mode": pv_risk_mode,
        "price_mode": price_mode,
        "realtime_feedback": bool(realtime_feedback),
        "terminal_policy": terminal_policy,
        "storage": storage.as_dict(),
        "price_source": "attachment1" if price_matrix is None else "attachment4",
        "load_information": (
            "Attachment 2 same-day true load curve (perfect-information benchmark)"
            if load_mode == "perfect"
            else "Online causal forecast from days strictly before d"
        ),
        "pv_information": (
            "Causal forecast from days strictly before d, "
            + ("historical residual quantile safety margin" if pv_risk_mode == "asymmetric"
                else "no safety margin (point)")
        ),
        "price_information": (
            "Attachment 4 same-day true price curve (perfect-information benchmark)"
            if price_mode == "perfect"
            else "Online causal forecast from days strictly before d"
        ),
        "realtime_storage_feedback": bool(realtime_feedback),
        "terminal_policy_definition": (
            "planned S_144 equals actual S_0 of the same day" if terminal_policy == "
                daily_cycle" else "no terminal SOC constraint"
        ),
        "max_balance_residual": max_balance,
        "max_soc_residual": max_soc_res,
        "max_day_boundary_soc_gap": continuity,
        "min_soc_kwh": min_soc,
        "max_soc_kwh": max_soc,
        "max_simultaneous_charge_discharge": max_sim,
```

```
        "forecast_selection_counts": dict(Counter(row["candidate"] for row in forecast_rows)),
        "load_selection_counts": dict(Counter(row["candidate"] for row in load_rows)),
        "price_selection_counts": dict(Counter(row["method"] for row in price_rows)) if price_rows else None,
    }
    if delivery:
        metrics.update(
            {
                "total_cost_yuan": float(sum(r["total_cost"] for r in delivery)),
                "plan_purchase_cost_yuan": float(sum(r["plan_cost"] for r in delivery)),
                "adjustment_cost_yuan": 0.0,
                "emergency_cost_yuan": float(sum(r["emergency_cost"] for r in delivery)),
                "plan_purchase_energy_kwh": float(sum(r["plan"].grid.sum() for r in delivery)),
                "adjusted_purchase_energy_kwh": float(sum(r["adjusted_grid"].sum() for r in delivery)),
                "emergency_energy_kwh": float(sum(r["execution"].emergency.sum() for r in delivery)),
                "curtailment_energy_kwh": float(sum(r["execution"].curtail.sum() for r in delivery)),
                "days_with_emergency": int(sum(r["execution"].emergency.sum() > 1e-7 for r in delivery)),
                "emergency_interval_count": int(
                    sum(_emergency_intervals(r["execution"].emergency) for r in delivery)
                ),
                "storage_charge_energy_kwh": float(sum(r["execution"].charge.sum() for r in delivery)),
                "storage_discharge_energy_kwh": float(sum(r["execution"].discharge.sum() for r in delivery)),
                "storage_throughput_kwh": float(
                    sum(r["execution"].charge.sum() + r["execution"].discharge.sum() for r in delivery)
                ),
                "ending_soc_kwh": float(delivery[-1]["execution"].soc[-1]),
                "load_forecast_mae_kw": float(np.mean([row["mae_kw"] for row in load_rows[31:]])),
                "load_forecast_rmse_kw": float(np.mean([row["rmse_kw"] for row in load_rows[31:]])),
                "pv_forecast_mae_kw": float(np.mean([row["pv_forecast_mae_kw"] for row in forecast_rows[31:]])),
                "pv_forecast_rmse_kw": float(np.mean([row["pv_forecast_rmse_kw"] for row in forecast_rows[31:]])),
                "price_forecast_mae": float(np.mean([row["price_mae"] for row in price_rows[31:]])) if price_rows else None,
                "price_forecast_rmse": float(np.mean([row["price_rmse"] for row in price_rows[31:]])) if price_rows else None,
```

```
                "runtime_seconds": float(sum(r["plan"].solve_seconds for r in delivery)),
            }
        )
        if pv_risk_mode == "asymmetric":
            metrics["pv_residual_quantile_evaluation"] = {
                f"Q{quantile:.2f}": {
                    "output_quantile": 1.0 - quantile,
                    "sample_out_risk_proxy_yuan": float(
                        sum(row[f"risk_q{int(100 * quantile):02d}_yuan"] for row in
                            forecast_rows[31:])
                    ),
                    "selected_days": int(
                        sum(abs(float(row["quantile"]) - quantile) < 1e-12 for row in
                            forecast_rows[31:])
                    ),
                }
                for quantile in QUANTILES
            }
    metrics["passed"] = bool(
        max_balance < 1e-7
        and max_soc_res < 1e-7
        and continuity < 1e-9
        and min_soc >= storage.soc_min - 1e-7
        and max_soc <= storage.soc_max + 1e-7
        and max_sim < 1e-5
    )
    if write_outputs:
        out_dir.mkdir(parents=True, exist_ok=True)
        (out_dir / "metrics.json").write_text(json.dumps(metrics, ensure_ascii=False, indent
            =2), encoding="utf-8")
        _write_csv(out_dir / "forecast_log.csv", forecast_rows)
        _write_csv(out_dir / "load_forecast_log.csv", load_rows)
        if price_rows:
            _write_csv(out_dir / "price_forecast_log.csv", price_rows)
        if delivery:
            fields = [
                "date", "slot", "load_kw", "actual_pv_kw", "load_plan_kw", "pv_plan_kw", "
                    price",
                "grid_plan_kwh", "grid_actual_kwh", "charge_actual_kwh", "
                    discharge_actual_kwh",
                "soc_actual_kwh", "curtail_actual_kwh", "emergency_kwh",
            ]
            with (out_dir / "solution.csv").open("w", newline="", encoding="utf-8-sig") as f
                :
                writer = csv.DictWriter(f, fieldnames=fields)
                writer.writeheader()
                for r in delivery:
```

```
                for t in range(144):
                    writer.writerow(
                        {
                            "date": r["date"].isoformat(),
                            "slot": t + 1,
                            "load_kw": r["load_kw"][t],
                            "actual_pv_kw": r["actual_pv_kw"][t],
                            "load_plan_kw": r["load_plan_kw"][t],
                            "pv_plan_kw": r["pv_plan_kw"][t],
                            "price": prices[r["day_index"], t],
                            "grid_plan_kwh": r["plan"].grid[t],
                            "grid_actual_kwh": r["adjusted_grid"][t],
                            "charge_actual_kwh": r["execution"].charge[t],
                            "discharge_actual_kwh": r["execution"].discharge[t],
                            "soc_actual_kwh": r["execution"].soc[t],
                            "curtail_actual_kwh": r["execution"].curtail[t],
                            "emergency_kwh": r["execution"].emergency[t],
                        }
                    )
            daily_fields = [
                "date", "soc_initial", "soc_terminal", "plan_cost", "emergency_cost", "
                    total_cost",
                "plan_energy", "emergency_energy", "curtailment_energy",
            ]
            _write_csv(
                out_dir / "daily_metrics.csv",
                [
                    {
                        "date": r["date"].isoformat(),
                        "soc_initial": r["soc_initial"],
                        "soc_terminal": r["execution"].soc[-1],
                        "plan_cost": r["plan_cost"],
                        "emergency_cost": r["emergency_cost"],
                        "total_cost": r["total_cost"],
                        "plan_energy": r["plan"].grid.sum(),
                        "emergency_energy": r["execution"].emergency.sum(),
                        "curtailment_energy": r["execution"].curtail.sum(),
                    }
                    for r in delivery
                ],
                daily_fields,
            )
            check = export_result2(template_path(template_name, data_root), out_dir /
                template_name, delivery)
            (out_dir / "xlsx_verification.json").write_text(
                json.dumps(check, ensure_ascii=False, indent=2), encoding="utf-8"
            )
```

```
            metrics["xlsx_passed"] = check["passed"]
            (out_dir / "metrics.json").write_text(json.dumps(metrics, ensure_ascii=False,
                indent=2), encoding="utf-8")
    environment = {
        "python": sys.version,
        "platform": platform.platform(),
        "numpy": np.__version__,
        "scipy": scipy.__version__,
        "algorithm": "scipy.optimize.linprog(method=highs), lexicographic 3-pass LP",
        "case_tag": tag,
    }
    if write_outputs:
        (out_dir / "environment.json").write_text(json.dumps(environment, ensure_ascii=False
            , indent=2), encoding="utf-8")
    return {"metrics": metrics, "records": records, "forecast_rows": forecast_rows, "
        load_rows": load_rows}
def _write_csv(path: Path, rows: list[dict], fields: list[str] | None = None) -> None:
    if not rows:
        return
    fieldnames = fields if fields is not None else list(rows[0])
    with path.open("w", newline="", encoding="utf-8-sig") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)
if __name__ == "__main__":
    print(json.dumps(run_q2()["metrics"], ensure_ascii=False, indent=2))
```

B.10 问题三求解（q3.py）

Listing 10:

```
from __future__ import annotations
import json
import platform
import sys
from pathlib import Path
import numpy as np
import scipy
from .export import export_result3
from .io_data import (
    DEFAULT_DATA_ROOT,
    DT_HOURS,
    hourly_to_ten_min,
    read_attachment1,
    read_attachment2,
    read_attachment3,
```

```
    template_path,
)
from .load_forecast import OnlineLoadForecaster
from .lp_core import DispatchResult, settlement_cost, solve_adjustment_dispatch,
    solve_dispatch
from .official_forecast import OnlineForecastBiasCalibrator
from .params import DEFAULT_STORAGE, StorageParams
from .price_forecast import OnlinePriceForecaster
from .q2 import _emergency_intervals, _write_csv
from .settle import ExecutionResult, execute_plan
ROOT = Path(__file__).resolve().parents[1]
OUTPUT = ROOT / "outputs" / "q3"
BLOCK = 36
ALL_NODES = (0, 36, 72, 108)
def _horizon(data: np.ndarray, day: int, start_slot: int, fallback: np.ndarray) -> np.
    ndarray:
    flat = data.reshape(-1)
    start = day * 144 + start_slot
    values = flat[start : min(start + 144, len(flat))]
    if len(values) < 144:
        pieces = [values]
        remaining = 144 - len(values)
        while remaining > 0:
            take = min(remaining, len(fallback))
            pieces.append(fallback[:take])
            remaining -= take
        values = np.concatenate(pieces)
    return np.asarray(values, dtype=float)
def _slice_dispatch(result: DispatchResult, start: int, stop: int) -> DispatchResult:
    return DispatchResult(
        result.grid[start:stop].copy(),
        result.charge[start:stop].copy(),
        result.discharge[start:stop].copy(),
        result.soc[start:stop].copy(),
        result.curtail[start:stop].copy(),
        result.objective,
        result.primary_objective,
        float(result.curtail[start:stop].sum()),
        float(result.charge[start:stop].sum() + result.discharge[start:stop].sum()),
        result.status,
        result.message,
        result.nit,
        result.solve_seconds,
        result.equality_marginals.copy(),
    )
def _replace_dispatch_tail(current: DispatchResult, horizon: DispatchResult, start: int) ->
    DispatchResult:
```

```
    count = 144 - start
    def merged(name: str) -> np.ndarray:
        values = np.asarray(getattr(current, name), dtype=float).copy()
        values[start:] = np.asarray(getattr(horizon, name), dtype=float)[:count]
        return values
    charge = merged("charge")
    discharge = merged("discharge")
    curtail = merged("curtail")
    return DispatchResult(
        merged("grid"),
        charge,
        discharge,
        merged("soc"),
        curtail,
        horizon.objective,
        horizon.primary_objective,
        float(curtail.sum()),
        float(charge.sum() + discharge.sum()),
        horizon.status,
        horizon.message,
        horizon.nit,
        horizon.solve_seconds,
        horizon.equality_marginals.copy(),
    )
def _reversal_count(revisions: np.ndarray) -> int:
    count = 0
    for slot in range(144):
        values = revisions[:, slot]
        values = values[np.isfinite(values)]
        signs = np.sign(np.diff(values))
        signs = signs[np.abs(signs) > 0]
        if len(signs) > 1 and np.any(signs[1:] * signs[:-1] < 0):
            count += 1
    return count
def _node_label(node: int) -> str:
    return f"{node // 6:02d}:00"
def run_q3(
    *,
    data_root: Path = DEFAULT_DATA_ROOT,
    max_days: int = 365,
    downscale: str = "linear",
    calibrate: bool = True,
    update_nodes: tuple[int, ...] = ALL_NODES,
    pv_update_nodes: tuple[int, ...] | None = None,
    settlement_mode: str = "additive",
    load_mode: str = "causal",
    price_mode: str = "perfect",
```

```
    realtime_feedback: bool = True,
    terminal_policy: str = "daily_cycle",
    price_matrix: np.ndarray | None = None,
    storage: StorageParams = DEFAULT_STORAGE,
    output_dir: Path | None = None,
    template_name: str = "result3.xlsx",
    write_outputs: bool = True,
    use_typical_prior: bool = True,
    tag: str = "",
) -> dict:
    if settlement_mode not in ("replacement", "additive"):
        raise ValueError(f"unknown settlement_mode: {settlement_mode}")
    if load_mode not in ("perfect", "causal") or price_mode not in ("perfect", "causal"):
        raise ValueError("unknown information mode")
    nodes = tuple(sorted(int(n) for n in update_nodes))
    if not nodes or nodes[0] != 0 or any(n not in ALL_NODES for n in nodes):
        raise ValueError(f"update_nodes must be a prefix of {ALL_NODES}, got {update_nodes}"
            )
    pv_nodes = nodes if pv_update_nodes is None else tuple(sorted(int(n) for n in
        pv_update_nodes))
    if not pv_nodes or pv_nodes[0] != 0 or any(n not in nodes for n in pv_nodes):
        raise ValueError(f"pv_update_nodes must contain 0 and be a subset of update_nodes,
            got {pv_nodes}")
    typical = read_attachment1(data_root)
    annual = read_attachment2(data_root)
    forecasts = read_attachment3(data_root)
    if annual.dates != forecasts.dates:
        raise ValueError("attachment2/3 dates do not align")
    if not 1 <= max_days <= 365:
        raise ValueError("max_days must be 1..365")
    prices = np.tile(typical.price, (365, 1)) if price_matrix is None else np.asarray(
        price_matrix, dtype=float)
    if prices.shape != (365, 144):
        raise ValueError("price matrix must have shape (365,144)")
    out_dir = OUTPUT if output_dir is None else Path(output_dir)
    state = 6000.0
    records: list[dict] = []
    verify: list[tuple] = []
    load_rows: list[dict] = []
    price_rows: list[dict] = []
    total_reversals = 0
    calibrator = OnlineForecastBiasCalibrator()
    load_forecaster = OnlineLoadForecaster(prior_kw=typical.load_kw if use_typical_prior
        else None)
    price_forecaster = OnlinePriceForecaster(prior_price=typical.price if use_typical_prior
        else None)
    for d in range(max_days):
```

```
148         initial = state
149         node_state = initial
150         soc_terminal = initial if terminal_policy == "daily_cycle" else None
151         load_decision0 = load_forecaster.predict(d, 0, 144)
152         price_decision0 = price_forecaster.predict_horizon(d, 0, 144) if price_mode == "
                causal" else None
153         load0 = annual.load_kw[d] if load_mode == "perfect" else load_decision0.forecast_kw
154         price0 = prices[d] if price_mode == "perfect" else price_decision0.forecast_price
155         raw_base = hourly_to_ten_min(forecasts.hourly_kw[d, 0], downscale)
156         calibration0 = calibrator.calibrate(0, raw_base)
157         base_pv = calibration0.forecast_kw if calibrate else raw_base
158         baseline = solve_dispatch(
159             load0, base_pv, price0, soc_initial=initial, soc_terminal=soc_terminal, storage=
                    storage
160         )
161         active_plan = baseline
162         active_pv_day = np.asarray(base_pv, dtype=float).copy()
163         active_source_node = 0
164         adjusted = np.zeros(144)
165         charge = np.zeros(144)
166         discharge = np.zeros(144)
167         soc = np.zeros(144)
168         curtail = np.zeros(144)
169         emergency = np.zeros(144)
170         inc_acc = np.zeros(144)
171         dec_acc = np.zeros(144)
172         revisions = np.full((4, 144), np.nan)
173         revisions[0] = baseline.grid
174         plan_source_nodes = np.zeros(144, dtype=int)
175         block_forecast_mae: list[float] = []
176         calibration_margins: list[float] = []
177         calibration_quantiles: list = []
178         load_plan_day = np.asarray(load0, dtype=float).copy()
179         load_rows.append(load_forecaster.log_row(load_decision0, annual.load_kw[d, :36],
                annual.dates[d].isoformat()))
180         if price_decision0 is not None:
181             price_rows.append(price_forecaster.log_row(price_decision0, prices[d, :36],
                    annual.dates[d].isoformat()))
182         node_load_decisions: dict[int, object] = {0: load_decision0}
183         for node_index, node in enumerate(ALL_NODES):
184             block_stop = node + BLOCK
185             if node_index == 0:
186                 calibration = calibration0
187                 block_plan = _slice_dispatch(baseline, 0, BLOCK)
188                 block_forecast = base_pv[0:BLOCK]
189                 block_inc = np.zeros(BLOCK)
190                 block_dec = np.zeros(BLOCK)
```

```
191             elif node in nodes:
192                 if node in pv_nodes:
193                     raw_horizon = hourly_to_ten_min(forecasts.hourly_kw[d, node_index],
                            downscale)
194                     calibration = calibrator.calibrate(node_index, raw_horizon)
195                     forecast_horizon = calibration.forecast_kw if calibrate else raw_horizon
196                 else:
197                     calibration = None
198                     forecast_horizon = np.r_[active_pv_day[node:], typical.pv_kw[:node]]
199                 if load_mode == "perfect":
200                     load_horizon = _horizon(annual.load_kw, d, node, typical.load_kw)
201                 else:
202                     load_decision = load_forecaster.predict(d, node, 144)
203                     load_horizon = load_decision.forecast_kw
204                     load_plan_day[node:] = load_horizon[: 144 - node]
205                     node_load_decisions[node] = load_decision
206                     load_rows.append(
207                         load_forecaster.log_row(load_decision, annual.load_kw[d, node:
                                block_stop], annual.dates[d].isoformat())
208                     )
209                 if price_mode == "perfect":
210                     price_horizon = _horizon(prices, d, node, typical.price)
211                 else:
212                     price_decision = price_forecaster.predict_horizon(d, node, 144)
213                     price_horizon = price_decision.forecast_price
214                     price_rows.append(
215                         price_forecaster.log_row(price_decision, prices[d, node:block_stop],
                                annual.dates[d].isoformat())
216                     )
217                 baseline_horizon = np.r_[baseline.grid[node:], np.zeros(node)]
218                 mask = np.r_[np.ones(144 - node, dtype=bool), np.zeros(node, dtype=bool)]
219                 rolling = solve_adjustment_dispatch(
220                     load_horizon,
221                     forecast_horizon,
222                     price_horizon,
223                     baseline_horizon,
224                     mask,
225                     soc_initial=node_state,
226                     soc_terminal=None if terminal_policy == "free" else node_state,
227                     settlement_mode=settlement_mode,
228                     storage=storage,
229                 )
230                 horizon_plan = rolling.dispatch
231                 active_plan = _replace_dispatch_tail(active_plan, horizon_plan, node)
232                 active_pv_day[node:] = forecast_horizon[: 144 - node]
233                 active_source_node = node
234                 block_plan = _slice_dispatch(active_plan, node, block_stop)
```

```
                revisions[node_index, node:] = active_plan.grid[node:]
                block_forecast = active_pv_day[node:block_stop]
                base_block = baseline.grid[node:block_stop]
                block_inc = np.maximum(block_plan.grid - base_block, 0.0)
                block_dec = np.maximum(base_block - block_plan.grid, 0.0)
            else:
                calibration = None
                block_plan = _slice_dispatch(active_plan, node, block_stop)
                block_forecast = active_pv_day[node:block_stop]
                base_block = baseline.grid[node:block_stop]
                block_inc = np.maximum(block_plan.grid - base_block, 0.0)
                block_dec = np.maximum(base_block - block_plan.grid, 0.0)
            plan_source_nodes[node:block_stop] = active_source_node
            execution = execute_plan(
                block_plan,
                annual.load_kw[d, node:block_stop],
                annual.pv_kw[d, node:block_stop],
                soc_initial=node_state,
                realtime_feedback=realtime_feedback,
                storage=storage,
            )
            adjusted[node:block_stop] = execution.grid
            charge[node:block_stop] = execution.charge
            discharge[node:block_stop] = execution.discharge
            soc[node:block_stop] = execution.soc
            curtail[node:block_stop] = execution.curtail
            emergency[node:block_stop] = execution.emergency
            inc_acc[node:block_stop] = block_inc
            dec_acc[node:block_stop] = block_dec
            if calibration is not None:
                calibrator.observe(calibration, annual.pv_kw[d, node:block_stop], prices[d,
                    node:block_stop])
                calibration_margins.append(calibration.margin_kw if calibrate else 0.0)
                calibration_quantiles.append(calibration.quantile if calibrate else None)
            block_forecast_mae.append(float(np.mean(np.abs(block_forecast - annual.pv_kw[d,
                node:block_stop]))))
            node_state = float(execution.soc[-1])
            if load_mode == "causal" and node in node_load_decisions:
                load_forecaster.observe_block(node_load_decisions[node], annual.load_kw[d,
                    node:block_stop])
            load_forecaster.observe_partial(block_stop, annual.load_kw[d, :block_stop])
        state = node_state
        actual = ExecutionResult(adjusted.copy(), charge, discharge, soc, curtail, emergency
            )
        gp = baseline.grid
        ga = adjusted
        increase = np.maximum(ga - gp, 0.0)
```

```
278         decrease = np.maximum(gp - ga, 0.0)
279         settlement = settlement_cost(gp, ga, prices[d], settlement_mode)
280         if settlement_mode == "replacement":
281             lp_side = float(np.dot(prices[d], ga + 0.5 * inc_acc + 0.5 * dec_acc))
282         else:
283             lp_side = float(np.dot(prices[d], gp) + np.dot(prices[d], 0.5 * dec_acc + 1.5 *
                    inc_acc))
284         emergency_cost = float(np.dot(5 * prices[d], emergency))
285         reversals = _reversal_count(revisions)
286         total_reversals += reversals
287         record = {
288             "day_index": d,
289             "date": annual.dates[d],
290             "soc_initial": initial,
291             "baseline_plan": baseline,
292             "adjusted_grid": adjusted,
293             "execution": actual,
294             "load_kw": annual.load_kw[d],
295             "actual_pv_kw": annual.pv_kw[d],
296             "load_plan_kw": load_plan_day,
297             "revisions": revisions,
298             "plan_source_nodes": plan_source_nodes.copy(),
299             "baseline_cost": float(np.dot(prices[d], gp)),
300             "settlement_cost": settlement,
301             "emergency_cost": emergency_cost,
302             "total_cost": settlement + emergency_cost,
303             "adjustment_cost_yuan": settlement - float(np.dot(prices[d], gp)) if
                    settlement_mode == "additive" else settlement - float(np.dot(prices[d], gp))
                    ,
304             "linearization_gap": abs(settlement - lp_side),
305             "up_energy": float(increase.sum()),
306             "down_energy": float(decrease.sum()),
307             "reversal_slots": reversals,
308             "forecast_mae": float(np.mean(block_forecast_mae)),
309             "calibration_margins": calibration_margins,
310             "calibration_quantiles": calibration_quantiles,
311         }
312         records.append(record)
313         balance = ga + annual.pv_kw[d] * DT_HOURS + discharge + emergency - annual.load_kw[d
                ] * DT_HOURS - charge - curtail
314         previous = np.r_[initial, soc[:-1]]
315         soc_res = soc - (previous + storage.eta_ch * charge - discharge / storage.eta_dis)
316         verify.append(
317             (
318                 float(np.max(np.abs(balance))),
319                 float(np.max(np.abs(soc_res))),
320                 float(soc.min()),
```

```
                float(soc.max()),
                float(np.max(np.minimum(charge, discharge))),
            )
        )
        if load_mode == "causal":
            load_forecaster.commit_day(annual.load_kw[d], d)
        if price_mode == "causal":
            price_forecaster.commit_day(prices[d], d)
        else:
            price_forecaster.commit_day(prices[d], d)
    delivery = records[31:] if max_days == 365 else []
    metrics = {
        "case_tag": tag,
        "days_simulated": max_days,
        "warmup_days": 31 if max_days == 365 else None,
        "delivery_days": len(delivery),
        "downscale_method": downscale,
        "calibration_enabled": calibrate,
        "update_nodes": list(nodes),
        "update_nodes_label": "S" + str(len(nodes) - 1),
        "pv_update_nodes": list(pv_nodes),
        "settlement_mode": settlement_mode,
        "load_mode": load_mode,
        "price_mode": price_mode,
        "realtime_feedback": bool(realtime_feedback),
        "terminal_policy": terminal_policy,
        "storage": storage.as_dict(),
        "price_source": "attachment1" if price_matrix is None else "attachment4",
        "load_information": (
            "Attachment 2 same-day true load curve (perfect-information benchmark)"
            if load_mode == "perfect"
            else "Online causal forecast from days strictly before d"
        ),
        "pv_information": (
            "Attachment 3 rolling forecasts with causal issue-specific calibration"
            if calibrate
            else "raw Attachment 3 forecasts"
        )
        + f"; newly issued forecasts used at nodes {list(pv_nodes)}; actual PV used only
            after block execution",
        "price_information": (
            "Attachment 4 same-day true price curve (perfect-information benchmark)"
            if price_mode == "perfect"
            else "Online causal price forecast from days strictly before d"
        ),
        "max_balance_residual": max(v[0] for v in verify),
        "max_soc_residual": max(v[1] for v in verify),
```

```
        "min_soc_kwh": min(v[2] for v in verify),
        "max_soc_kwh": max(v[3] for v in verify),
        "max_simultaneous_charge_discharge": max(v[4] for v in verify),
        "replacement_settlement_formula": "min(Gp,Ga) + 0.5*(Gp-Ga)^+ + 1.5*(Ga-Gp)^+",
        "additive_settlement_formula": "Gp + 0.5*(Gp-Ga)^+ + 1.5*(Ga-Gp)^+",
        "cross_midnight": "24h optimize, execute current 6h block only; carry SOC only",
    }
    if delivery:
        metrics.update(
            {
                "total_cost_yuan": float(sum(r["total_cost"] for r in delivery)),
                "baseline_purchase_cost_yuan": float(sum(r["baseline_cost"] for r in
                    delivery)),
                "adjustment_settlement_cost_yuan": float(sum(r["settlement_cost"] for r in
                    delivery)),
                "adjustment_cost_yuan": float(sum(r["adjustment_cost_yuan"] for r in
                    delivery)),
                "emergency_cost_yuan": float(sum(r["emergency_cost"] for r in delivery)),
                "plan_purchase_energy_kwh": float(sum(r["baseline_plan"].grid.sum() for r in
                     delivery)),
                "adjusted_purchase_energy_kwh": float(sum(r["adjusted_grid"].sum() for r in
                    delivery)),
                "emergency_energy_kwh": float(sum(r["execution"].emergency.sum() for r in
                    delivery)),
                "curtailment_energy_kwh": float(sum(r["execution"].curtail.sum() for r in
                    delivery)),
                "up_adjustment_energy_kwh": float(sum(r["up_energy"] for r in delivery)),
                "down_adjustment_energy_kwh": float(sum(r["down_energy"] for r in delivery))
                    ,
                "days_with_emergency": int(sum(r["execution"].emergency.sum() > 1e-7 for r
                    in delivery)),
                "emergency_interval_count": int(sum(_emergency_intervals(r["execution"].
                    emergency) for r in delivery)),
                "storage_charge_energy_kwh": float(sum(r["execution"].charge.sum() for r in
                    delivery)),
                "storage_discharge_energy_kwh": float(sum(r["execution"].discharge.sum() for
                     r in delivery)),
                "storage_throughput_kwh": float(
                    sum(r["execution"].charge.sum() + r["execution"].discharge.sum() for r
                        in delivery)
                ),
                "ending_soc_kwh": float(delivery[-1]["execution"].soc[-1]),
                "direction_reversal_slots": int(sum(r["reversal_slots"] for r in delivery)),
                "max_linearization_gap": float(max(r["linearization_gap"] for r in delivery)
                    ),
                "executed_block_forecast_mae_kw": float(np.mean([r["forecast_mae"] for r in
                    delivery])),
```

```
            "load_forecast_mae_kw": float(np.mean([row["mae_kw"] for row in load_rows[31:]])) if load_rows else None,
            "load_forecast_rmse_kw": float(np.mean([row["rmse_kw"] for row in load_rows[31:]])) if load_rows else None,
            "price_forecast_mae": float(np.mean([row["price_mae"] for row in price_rows[31:]])) if price_rows else None,
            "price_forecast_rmse": float(np.mean([row["price_rmse"] for row in price_rows[31:]])) if price_rows else None,
            "runtime_seconds": float(
                sum(r["baseline_plan"].solve_seconds for r in delivery)
            ),
        }
    )
metrics["passed"] = bool(
    metrics["max_balance_residual"] < 1e-7
    and metrics["max_soc_residual"] < 1e-7
    and metrics["min_soc_kwh"] >= storage.soc_min - 1e-7
    and metrics["max_soc_kwh"] <= storage.soc_max + 1e-7
    and metrics["max_simultaneous_charge_discharge"] < 1e-5
    and (not delivery or metrics["max_linearization_gap"] < 1e-6)
)
if write_outputs:
    out_dir.mkdir(parents=True, exist_ok=True)
    (out_dir / "metrics.json").write_text(json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8")
    _write_csv(out_dir / "load_forecast_log.csv", load_rows)
    if price_rows:
        _write_csv(out_dir / "price_forecast_log.csv", price_rows)
    if delivery:
        fields = [
            "date", "slot", "load_kw", "actual_pv_kw", "load_plan_kw", "price", "baseline_grid_kwh",
            "adjusted_grid_kwh", "charge_actual_kwh", "discharge_actual_kwh", "soc_actual_kwh",
            "curtail_actual_kwh", "emergency_kwh", "revision_0", "revision_6", "revision_12", "revision_18",
        ]
        with (out_dir / "solution.csv").open("w", newline="", encoding="utf-8-sig") as f:
            import csv as _csv
            writer = _csv.DictWriter(f, fieldnames=fields)
            writer.writeheader()
            for r in delivery:
                for t in range(144):
                    rev = r["revisions"][:, t]
                    writer.writerow(
                        {
```

```
                                "date": r["date"].isoformat(),
                                "slot": t + 1,
                                "load_kw": r["load_kw"][t],
                                "actual_pv_kw": r["actual_pv_kw"][t],
                                "load_plan_kw": r["load_plan_kw"][t],
                                "price": prices[r["day_index"], t],
                                "baseline_grid_kwh": r["baseline_plan"].grid[t],
                                "adjusted_grid_kwh": r["adjusted_grid"][t],
                                "charge_actual_kwh": r["execution"].charge[t],
                                "discharge_actual_kwh": r["execution"].discharge[t],
                                "soc_actual_kwh": r["execution"].soc[t],
                                "curtail_actual_kwh": r["execution"].curtail[t],
                                "emergency_kwh": r["execution"].emergency[t],
                                "revision_0": rev[0],
                                "revision_6": rev[1],
                                "revision_12": rev[2],
                                "revision_18": rev[3],
                            }
                        )
            fields_d = [
                "date", "soc_initial", "soc_terminal", "baseline_cost", "settlement_cost", "
                    emergency_cost",
                "total_cost", "up_energy", "down_energy", "reversal_slots", "forecast_mae",
            ]
            _write_csv(
                out_dir / "daily_metrics.csv",
                [
                    {
                        "date": r["date"].isoformat(),
                        "soc_initial": r["soc_initial"],
                        "soc_terminal": r["execution"].soc[-1],
                        **{k: r[k] for k in fields_d[3:]},
                    }
                    for r in delivery
                ],
                fields_d,
            )
            check = export_result3(template_path(template_name, data_root), out_dir /
                template_name, delivery)
            (out_dir / "xlsx_verification.json").write_text(json.dumps(check, ensure_ascii=
                False, indent=2), encoding="utf-8")
            metrics["xlsx_passed"] = check["passed"]
            (out_dir / "metrics.json").write_text(json.dumps(metrics, ensure_ascii=False,
                indent=2), encoding="utf-8")
        env = {
            "python": sys.version,
            "platform": platform.platform(),
```

```
            "numpy": np.__version__,
            "scipy": scipy.__version__,
            "algorithm": (
                "HiGHS MILP with binary charge/discharge exclusivity and explicit (inc, dec)
                     split"
                if settlement_mode == "additive"
                else "HiGHS LP with explicit (inc, dec) split"
            ),
            "case_tag": tag,
        }
        (out_dir / "environment.json").write_text(json.dumps(env, ensure_ascii=False, indent
            =2), encoding="utf-8")
    return {"metrics": metrics, "records": records}
if __name__ == "__main__":
    print(json.dumps(run_q3()["metrics"], ensure_ascii=False, indent=2))
```

B.11 问题四求解（q4.py）

Listing 11:

```
from __future__ import annotations
import json
from pathlib import Path
from .io_data import DEFAULT_DATA_ROOT, read_attachment4
from .q2 import run_q2
from .q3 import run_q3
ROOT = Path(__file__).resolve().parents[1]
OUTPUT = ROOT / "outputs"
def run_q4(
    *,
    data_root: Path = DEFAULT_DATA_ROOT,
    max_days: int = 365,
    price_mode_q2: str = "perfect",
    price_mode_q3: str = "perfect",
    load_mode: str = "causal",
    settlement_mode: str = "additive",
    update_nodes: tuple[int, ...] = (0, 36, 72, 108),
    realtime_feedback: bool = True,
    write_outputs: bool = True,
    tag_prefix: str = "",
) -> dict:
    annual_price = read_attachment4(data_root)
    prices = annual_price.price
    suffix = f"-{tag_prefix}" if tag_prefix else ""
    out2 = OUTPUT / f"q4-2{suffix}"
    out3 = OUTPUT / f"q4-3{suffix}"
```

```
    q4_2 = run_q2(
        data_root=data_root,
        max_days=max_days,
        price_matrix=prices,
        output_dir=out2,
        template_name="result4-2.xlsx",
        write_outputs=write_outputs,
        load_mode=load_mode,
        price_mode=price_mode_q2,
        realtime_feedback=realtime_feedback,
        tag=f"q4-2 {tag_prefix}".strip(),
    )
    q4_3 = run_q3(
        data_root=data_root,
        max_days=max_days,
        price_matrix=prices,
        output_dir=out3,
        template_name="result4-3.xlsx",
        write_outputs=write_outputs,
        load_mode=load_mode,
        price_mode=price_mode_q3,
        settlement_mode=settlement_mode,
        update_nodes=update_nodes,
        realtime_feedback=realtime_feedback,
        tag=f"q4-3 {tag_prefix}".strip(),
    )
    c2 = q4_2["metrics"].get("total_cost_yuan")
    c3 = q4_3["metrics"].get("total_cost_yuan")
    comparison = {
        "q4_2_total_cost_yuan": c2,
        "q4_3_total_cost_yuan": c3,
        "rolling_saving_yuan": None if (c2 is None or c3 is None) else c2 - c3,
        "rolling_saving_percent": None if (c2 is None or c3 is None) else 100 * (c2 - c3) /
            c2,
        "q4_2_emergency_energy_kwh": q4_2["metrics"].get("emergency_energy_kwh"),
        "q4_3_emergency_energy_kwh": q4_3["metrics"].get("emergency_energy_kwh"),
        "q4_2_price_mode": price_mode_q2,
        "q4_3_price_mode": price_mode_q3,
        "q4_2_price_mae": q4_2["metrics"].get("price_forecast_mae"),
        "q4_3_price_mae": q4_3["metrics"].get("price_forecast_mae"),
        "q4_2_passed": bool(q4_2["metrics"]["passed"]),
        "q4_3_passed": bool(q4_3["metrics"]["passed"]),
    }
    comparison["passed"] = comparison["q4_2_passed"] and comparison["q4_3_passed"]
    if write_outputs:
        out2.mkdir(parents=True, exist_ok=True)
        (out2 / "q4_comparison.json").write_text(
```

```
            json.dumps(comparison, ensure_ascii=False, indent=2), encoding="utf-8"
        )
    return {"q4_2": q4_2, "q4_3": q4_3, "comparison": comparison}
if __name__ == "__main__":
    print(json.dumps(run_q4()["comparison"], ensure_ascii=False, indent=2))
```

B.12 官方结果文件导出（export.py）

Listing 12:

```
from __future__ import annotations
from copy import copy
from datetime import datetime,time
from pathlib import Path
from shutil import copy2
import numpy as np
import openpyxl
from .lp_core import DispatchResult
def export_result1(template:Path,destination:Path,result:DispatchResult)->dict:
    destination.parent.mkdir(parents=True,exist_ok=True); copy2(template,destination)
    wb=openpyxl.load_workbook(destination); plan,storage=wb.worksheets[:2]
    for row,value in enumerate(result.grid,start=2):
        plan.cell(row,2,float(value)); plan.cell(row,2).number_format="0.0000"
    for block in range(6):
        lo,hi=block*24,(block+1)*24
        storage.cell(block+2,2,float(result.charge[lo:hi].sum()))
        storage.cell(block+2,3,float(result.discharge[lo:hi].sum()))
    storage.cell(2,5,6000.0); storage.cell(3,5,float(result.soc[-1])); wb.save(destination)
    check=openpyxl.load_workbook(destination,data_only=True,read_only=True)
    values=np.asarray([check.worksheets[0].cell(i,2).value for i in range(2,146)],dtype=
        float)
    diff=float(np.max(np.abs(values-result.grid)))
    return {"rows":len(values),"max_plan_roundtrip_diff":diff,"passed":bool(diff<1e-9)}
def _copy_row_style(ws,source_row:int,target_row:int,max_col:int)->None:
    for col in range(1,max_col+1):
        src=ws.cell(source_row,col); dst=ws.cell(target_row,col)
        if src.has_style: dst._style=copy(src._style)
        dst.alignment=copy(src.alignment); dst.protection=copy(src.protection)
    ws.row_dimensions[target_row].height=ws.row_dimensions[source_row].height
def _segments(values:np.ndarray,tolerance:float=1e-7)->list[tuple[int,int,float]]:
    result=[]; start=None
    for idx,flag in enumerate(np.r_[np.asarray(values)>tolerance,False]):
        if flag and start is None: start=idx
        elif not flag and start is not None:
            result.append((start,idx,float(np.sum(values[start:idx])))); start=None
    return result
```

```
def _slot_time(slot:int)->str:
    minutes=slot*10
    return "24:00" if minutes==1440 else f"{minutes//60}:{minutes%60:02d}"
def export_result2(template:Path,destination:Path,days:list[dict])->dict:
    if len(days)!=334: raise ValueError(f"result2 requires 334 days, got {len(days)}")
    destination.parent.mkdir(parents=True,exist_ok=True); copy2(template,destination)
    wb=openpyxl.load_workbook(destination); plan_ws,storage_ws,emergency_ws=wb.worksheets[:3]
    for idx,day in enumerate(days):
        row=idx+2; plan=day["plan"]
        plan_ws.cell(row,1,datetime.combine(day["date"],time()))
        for slot,value in enumerate(plan.grid,start=2):
            plan_ws.cell(row,slot,float(value)); plan_ws.cell(row,slot).number_format="0.0000"
        plan_ws.cell(row,146,float(np.sum(plan.grid))); plan_ws.cell(row,147,float(day["plan_cost"]))
        plan_ws.cell(row,146).number_format=plan_ws.cell(row,147).number_format="0.0000"
    target=1+6*len(days)
    if storage_ws.max_row<target: storage_ws.insert_rows(storage_ws.max_row+1,amount=target-storage_ws.max_row)
    periods=("0:00-4:00","4:00-8:00","8:00-12:00","12:00-16:00","16:00-20:00","20:00-24:00")
    for idx,day in enumerate(days):
        actual=day["execution"]
        for block in range(6):
            row=2+idx*6+block; _copy_row_style(storage_ws,2+block,row,6)
            storage_ws.cell(row,1,datetime.combine(day["date"],time()) if block==0 else None)
            storage_ws.cell(row,2,periods[block]); lo,hi=block*24,(block+1)*24
            storage_ws.cell(row,3,float(actual.charge[lo:hi].sum()))
            storage_ws.cell(row,4,float(actual.discharge[lo:hi].sum()))
            storage_ws.cell(row,5,time(0,0) if block==0 else ("24:00" if block==1 else None))
            storage_ws.cell(row,6,float(day["soc_initial"]) if block==0 else (float(actual.soc[-1]) if block==1 else None))
            for col in (3,4,6): storage_ws.cell(row,col).number_format="0.0000"
    if storage_ws.max_row>target: storage_ws.delete_rows(target+1,storage_ws.max_row-target)
    rows=[]
    for day in days:
        segments=_segments(day["execution"].emergency)
        if not segments: rows.append((datetime.combine(day["date"],time()),"无",0.0))
        else:
            for k,(start,end,total) in enumerate(segments):
                rows.append((datetime.combine(day["date"],time()) if k==0 else None,
                             f"{_slot_time(start)}-{_slot_time(end)}",total))
    target_e=1+len(rows)
    if emergency_ws.max_row<target_e: emergency_ws.insert_rows(emergency_ws.max_row+1,amount=target_e-emergency_ws.max_row)
```

```
    for row,(date_value,period,total) in enumerate(rows,start=2):
        _copy_row_style(emergency_ws,2,row,3)
        emergency_ws.cell(row,1,date_value); emergency_ws.cell(row,2,period)
        emergency_ws.cell(row,3,float(total)); emergency_ws.cell(row,3).number_format="
            0.0000"
    if emergency_ws.max_row>target_e: emergency_ws.delete_rows(target_e+1,emergency_ws.
        max_row-target_e)
    wb.save(destination)
    check=openpyxl.load_workbook(destination,data_only=True,read_only=True)
    plan_rows=list(check.worksheets[0].iter_rows(min_row=2,max_row=335,min_col=2,max_col
        =145,values_only=True))
    values=np.asarray(plan_rows,dtype=float)
    expected=np.stack([d["plan"].grid for d in days]); diff=float(np.max(np.abs(values-
        expected)))
    last_row=3+6*(len(days)-1)
    last_soc=float(next(check.worksheets[1].iter_rows(min_row=last_row,max_row=last_row,
        min_col=6,max_col=6,values_only=True))[0])
    return {"delivery_days":334,"plan_rows":values.shape[0],"plan_slots":values.shape[1],
            "storage_rows":check.worksheets[1].max_row-1,"emergency_rows":check.worksheets
                [2].max_row-1,
            "max_plan_roundtrip_diff":diff,"last_terminal_soc":last_soc,
            "passed":bool(values.shape==(334,144) and diff<1e-9 and check.worksheets[1].
                max_row-1==2004)}
def export_result3(template:Path,destination:Path,days:list[dict])->dict:
    if len(days)!=334: raise ValueError(f"result3 requires 334 days, got {len(days)}")
    destination.parent.mkdir(parents=True,exist_ok=True); copy2(template,destination)
    wb=openpyxl.load_workbook(destination); plan_ws,adjust_ws,storage_ws,emergency_ws=wb.
        worksheets[:4]
    for idx,day in enumerate(days):
        row=idx+2; plan=day["baseline_plan"]; adjusted=day["adjusted_grid"]
        for ws,values,cost in ((plan_ws,plan.grid,day["baseline_cost"]),(adjust_ws,adjusted,
            day["settlement_cost"])):
            ws.cell(row,1,datetime.combine(day["date"],time()))
            for slot,value in enumerate(values,start=2):
                ws.cell(row,slot,float(value)); ws.cell(row,slot).number_format="0.0000"
            ws.cell(row,146,float(np.sum(values))); ws.cell(row,147,float(cost))
            ws.cell(row,146).number_format=ws.cell(row,147).number_format="0.0000"
    target=1+6*len(days)
    if storage_ws.max_row<target: storage_ws.insert_rows(storage_ws.max_row+1,amount=target-
        storage_ws.max_row)
    periods=("0:00-4:00","4:00-8:00","8:00-12:00","12:00-16:00","16:00-20:00","20:00-24:00")
    for idx,day in enumerate(days):
        actual=day["execution"]
        for block in range(6):
            row=2+idx*6+block; _copy_row_style(storage_ws,2+block,row,6)
            storage_ws.cell(row,1,datetime.combine(day["date"],time()) if block==0 else None
                )
```

```
            storage_ws.cell(row,2,periods[block]); lo,hi=block*24,(block+1)*24
            storage_ws.cell(row,3,float(actual.charge[lo:hi].sum()))
            storage_ws.cell(row,4,float(actual.discharge[lo:hi].sum()))
            storage_ws.cell(row,5,time(0,0) if block==0 else ("24:00" if block==1 else None)
                )
            storage_ws.cell(row,6,float(day["soc_initial"]) if block==0 else (float(actual.
                soc[-1]) if block==1 else None))
            for col in (3,4,6): storage_ws.cell(row,col).number_format="0.0000"
    if storage_ws.max_row>target: storage_ws.delete_rows(target+1,storage_ws.max_row-target)
    rows=[]
    for day in days:
        segments=_segments(day["execution"].emergency)
        if not segments: rows.append((datetime.combine(day["date"],time()),"无",0.0))
        else:
            for k,(start,end,total) in enumerate(segments):
                rows.append((datetime.combine(day["date"],time()) if k==0 else None,
                             f"{_slot_time(start)}-{_slot_time(end)}",total))
    target_e=1+len(rows)
    if emergency_ws.max_row<target_e: emergency_ws.insert_rows(emergency_ws.max_row+1,amount
        =target_e-emergency_ws.max_row)
    for row,(date_value,period,total) in enumerate(rows,start=2):
        _copy_row_style(emergency_ws,2,row,3); emergency_ws.cell(row,1,date_value)
        emergency_ws.cell(row,2,period); emergency_ws.cell(row,3,float(total))
        emergency_ws.cell(row,3).number_format="0.0000"
    if emergency_ws.max_row>target_e: emergency_ws.delete_rows(target_e+1,emergency_ws.
        max_row-target_e)
    wb.save(destination)
    check=openpyxl.load_workbook(destination,data_only=True,read_only=True)
    plan_values=np.asarray(list(check.worksheets[0].iter_rows(min_row=2,max_row=335,min_col
        =2,max_col=145,values_only=True)),dtype=float)
    adjust_values=np.asarray(list(check.worksheets[1].iter_rows(min_row=2,max_row=335,
        min_col=2,max_col=145,values_only=True)),dtype=float)
    expected_plan=np.stack([d["baseline_plan"].grid for d in days]); expected_adjust=np.
        stack([d["adjusted_grid"] for d in days])
    return {"delivery_days":334,"plan_shape":list(plan_values.shape),"adjust_shape":list(
        adjust_values.shape),
            "storage_rows":check.worksheets[2].max_row-1,"emergency_rows":check.worksheets
                [3].max_row-1,
            "max_plan_roundtrip_diff":float(np.max(np.abs(plan_values-expected_plan))),
            "max_adjust_roundtrip_diff":float(np.max(np.abs(adjust_values-expected_adjust)))
                ,
            "passed":bool(plan_values.shape==(334,144) and adjust_values.shape==(334,144)
                and
                          check.worksheets[2].max_row-1==2004 and
                          np.max(np.abs(plan_values-expected_plan))<1e-9 and np.max(np.abs(
                              adjust_values-expected_adjust))<1e-9)}
```

B.13 模型审计与消融实验总控（ablation.py）

Listing 13:

```python
from __future__ import annotations
import json
import time
from pathlib import Path
import numpy as np
from .io_data import DEFAULT_DATA_ROOT, read_attachment1, read_attachment4
from .params import DEFAULT_STORAGE, EFFICIENCY_VARIANTS
from .q1 import run_q1
from .q2 import run_q2
from .q3 import run_q3
ROOT = Path(__file__).resolve().parents[1]
VARIANTS = ROOT / "outputs" / "variants" / "model_audit"
Q2_CASES = {
    "A_perfectload_risk_feedback": dict(load_mode="perfect", pv_risk_mode="asymmetric",
        realtime_feedback=True),
    "B_causalload_risk_feedback": dict(load_mode="causal", pv_risk_mode="asymmetric",
        realtime_feedback=True),
    "C_causalload_point_feedback": dict(load_mode="causal", pv_risk_mode="point",
        realtime_feedback=True),
    "D_causalload_risk_nofeedback": dict(load_mode="causal", pv_risk_mode="asymmetric",
        realtime_feedback=False),
}
Q3_MAIN_CASES = {
    "S0_0": dict(update_nodes=(0,)),
    "S1_0_6": dict(update_nodes=(0, 36)),
    "S2_0_6_12": dict(update_nodes=(0, 36, 72)),
    "S3_0_6_12_18": dict(update_nodes=(0, 36, 72, 108)),
}
Q3_CONTROL_CASES = {
    "C1_load6": dict(update_nodes=(0, 36), pv_update_nodes=(0,)),
    "C2_load12": dict(update_nodes=(0, 36, 72), pv_update_nodes=(0, 36)),
    "C3_load18": dict(update_nodes=(0, 36, 72, 108), pv_update_nodes=(0, 36, 72)),
}
Q3_CASES = {**Q3_MAIN_CASES, **Q3_CONTROL_CASES}
Q4_CASES = {
    "q42_perfect_price": ("q2", dict(price_mode="perfect")),
    "q42_causal_price": ("q2", dict(price_mode="causal")),
    "q43_perfect_price": ("q3", dict(price_mode="perfect")),
    "q43_causal_price": ("q3", dict(price_mode="causal")),
}
SEASONS = {
    "spring": (3, 4, 5),
    "summer": (6, 7, 8),
```

```
    "autumn": (9, 10, 11),
    "winter": (12, 1, 2),
}
def _season(month: int) -> str:
    for name, months in SEASONS.items():
        if month in months:
            return name
    raise ValueError(month)
def run_q2_ablation(*, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT) -> dict:
    out = VARIANTS / "q2"
    summary = {}
    for name, kwargs in Q2_CASES.items():
        started = time.perf_counter()
        result = run_q2(
            data_root=data_root,
            max_days=max_days,
            output_dir=out / name,
            template_name="result2.xlsx",
            write_outputs=True,
            tag=name,
            **kwargs,
        )
        metrics = result["metrics"]
        metrics["wall_seconds"] = time.perf_counter() - started
        (out / name / "metrics.json").write_text(
            json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
        )
        summary[name] = metrics
        print(f"[Q2] {name}: total={metrics.get('total_cost_yuan')} emergency={metrics.get('
            emergency_energy_kwh')}")
    _dump(out / "summary.json", summary)
    return summary
def run_q3_ablation(
    *, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT, settlement_mode: str = "
        additive"
) -> dict:
    out = VARIANTS / f"q3_{settlement_mode}"
    summary = {}
    for name, kwargs in Q3_CASES.items():
        started = time.perf_counter()
        result = run_q3(
            data_root=data_root,
            max_days=max_days,
            output_dir=out / name,
            template_name="result3.xlsx",
            write_outputs=True,
            settlement_mode=settlement_mode,
```

```
            tag=name,
            **kwargs,
        )
        metrics = result["metrics"]
        metrics["wall_seconds"] = time.perf_counter() - started
        (out / name / "metrics.json").write_text(
            json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
        )
        (out / name / "by_season.csv").write_text(_season_table(result["records"]), encoding
            ="utf-8-sig")
        summary[name] = metrics
        print(f"[Q3-{settlement_mode}] {name}: total={metrics.get('total_cost_yuan')}")
    _dump(out / "summary.json", summary)
    return summary
def run_q3_controls(
    *, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT, settlement_mode: str = "
        additive"
) -> dict:
    out = VARIANTS / f"q3_{settlement_mode}"
    summary_path = out / "summary.json"
    summary = json.loads(summary_path.read_text(encoding="utf-8")) if summary_path.exists()
        else {}
    for name, kwargs in Q3_CONTROL_CASES.items():
        started = time.perf_counter()
        result = run_q3(
            data_root=data_root,
            max_days=max_days,
            output_dir=out / name,
            template_name="result3.xlsx",
            write_outputs=True,
            settlement_mode=settlement_mode,
            tag=name,
            **kwargs,
        )
        metrics = result["metrics"]
        metrics["wall_seconds"] = time.perf_counter() - started
        (out / name / "metrics.json").write_text(
            json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
        )
        (out / name / "by_season.csv").write_text(
            _season_table(result["records"]), encoding="utf-8-sig"
        )
        summary[name] = metrics
        print(f"[Q3-{settlement_mode}-control] {name}: total={metrics.get('total_cost_yuan')
            }")
    _dump(summary_path, summary)
    return summary
```

```python
def _season_table(records: list[dict]) -> str:
    delivery = records[31:]
    rows = ["season,days,total_cost_yuan,baseline_cost_yuan,emergency_energy_kwh,
        up_energy_kwh,down_energy_kwh,curtailment_kwh"]
    for season in ("spring", "summer", "autumn", "winter"):
        subset = [r for r in delivery if _season(r["date"].month) == season]
        if not subset:
            continue
        rows.append(
            ",".join(
                str(x)
                for x in [
                    season,
                    len(subset),
                    round(sum(r["total_cost"] for r in subset), 4),
                    round(sum(r["baseline_cost"] for r in subset), 4),
                    round(sum(r["execution"].emergency.sum() for r in subset), 4),
                    round(sum(r["up_energy"] for r in subset), 4),
                    round(sum(r["down_energy"] for r in subset), 4),
                    round(sum(r["execution"].curtail.sum() for r in subset), 4),
                ]
            )
        )
    return "\n".join(rows) + "\n"
def run_q4_ablation(*, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT) -> dict:
    out = VARIANTS / "q4"
    prices = read_attachment4(data_root).price
    summary = {}
    for name, (kind, kwargs) in Q4_CASES.items():
        started = time.perf_counter()
        if kind == "q2":
            result = run_q2(
                data_root=data_root,
                max_days=max_days,
                price_matrix=prices,
                output_dir=out / name,
                template_name="result4-2.xlsx",
                write_outputs=True,
                load_mode="causal",
                tag=name,
                **kwargs,
            )
        else:
            result = run_q3(
                data_root=data_root,
                max_days=max_days,
                price_matrix=prices,
```

```
                output_dir=out / name,
                template_name="result4-3.xlsx",
                write_outputs=True,
                load_mode="causal",
                settlement_mode="additive",
                update_nodes=(0, 36, 72, 108),
                tag=name,
                **kwargs,
            )
        metrics = result["metrics"]
        metrics["wall_seconds"] = time.perf_counter() - started
        (out / name / "metrics.json").write_text(
            json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
        )
        summary[name] = metrics
        print(f"[Q4] {name}: total={metrics.get('total_cost_yuan')} emergency={metrics.get('
            emergency_energy_kwh')}")
    _dump(out / "summary.json", summary)
    return summary
def run_sensitivity(*, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT) -> dict:
    out = VARIANTS / "sensitivity"
    summary = {"terminal": {}, "efficiency": {}}
    for policy in ("daily_cycle", "free"):
        started = time.perf_counter()
        result = run_q2(
            data_root=data_root,
            max_days=max_days,
            output_dir=out / "terminal" / policy,
            template_name="result2.xlsx",
            write_outputs=True,
            load_mode="causal",
            pv_risk_mode="asymmetric",
            realtime_feedback=True,
            terminal_policy=policy,
            tag=f"terminal_{policy}",
        )
        metrics = result["metrics"]
        metrics["wall_seconds"] = time.perf_counter() - started
        daily_soc = [float(r["execution"].soc[-1]) for r in result["records"][31:]]
        metrics["daily_boundary_soc"] = {
            "min": float(np.min(daily_soc)),
            "max": float(np.max(daily_soc)),
            "mean": float(np.mean(daily_soc)),
            "std": float(np.std(daily_soc)),
        }
        (out / "terminal" / policy / "metrics.json").write_text(
            json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
```

```
        )
        summary["terminal"][policy] = metrics
        print(f"[P1-terminal] {policy}: total={metrics.get('total_cost_yuan')} endSOC={
            metrics.get('ending_soc_kwh')}")
    for name, storage in EFFICIENCY_VARIANTS.items():
        started = time.perf_counter()
        result = run_q2(
            data_root=data_root,
            max_days=max_days,
            output_dir=out / "efficiency" / name,
            template_name="result2.xlsx",
            write_outputs=True,
            load_mode="causal",
            pv_risk_mode="asymmetric",
            realtime_feedback=True,
            storage=storage,
            tag=f"efficiency_{name}",
        )
        metrics = result["metrics"]
        metrics["wall_seconds"] = time.perf_counter() - started
        (out / "efficiency" / name / "metrics.json").write_text(
            json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
        )
        summary["efficiency"][name] = metrics
        print(f"[P1-efficiency] {name}: total={metrics.get('total_cost_yuan')}")
    _dump(out / "summary.json", summary)
    return summary
def _dump(path: Path, payload: dict) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(payload, ensure_ascii=False, indent=2, default=str), encoding
        ="utf-8")
def main() -> None:
    run_q1()
    run_q2_ablation()
    run_q3_ablation(settlement_mode="additive")
    run_q3_ablation(settlement_mode="replacement")
    run_q4_ablation()
    run_sensitivity()
if __name__ == "__main__":
    main()
```

B.14 官方光伏预报处理（official_forecast.py）

Listing 14:

```
from __future__ import annotations
```

```
from dataclasses import dataclass
import numpy as np
from .io_data import DT_HOURS
QUANTILES=(0.80,0.85,0.90,0.95)
@dataclass(frozen=True)
class CalibrationDecision:
    issue_index:int; quantile:float; margin_kw:float; history_days_used:int
    raw_forecast_kw:np.ndarray; forecast_kw:np.ndarray; candidates:dict[float,np.ndarray]
    historical_score:float|None
class OnlineForecastBiasCalibrator:
    def __init__(self,residual_window:int=30,score_window:int=30):
        self.residual_window=residual_window; self.score_window=score_window
        self.residual_days=[[] for _ in range(4)]
        self.score_days=[{q:[] for q in QUANTILES} for _ in range(4)]
    def calibrate(self,issue_index:int,raw_forecast_kw:np.ndarray)->CalibrationDecision:
        raw=np.asarray(raw_forecast_kw,dtype=float)
        days=self.residual_days[issue_index][-self.residual_window:]
        residual=np.concatenate(days) if days else np.array([])
        margins={q:(max(0.0,float(np.quantile(residual,q))) if residual.size else 0.0) for q
             in QUANTILES}
        candidates={q:np.clip(raw-margins[q],0.0,None) for q in QUANTILES}
        history_count=len(self.residual_days[issue_index])
        if history_count<7: q=0.80; hist=None
        else:
            scores={x:float(np.mean(self.score_days[issue_index][x][-self.score_window:]))
                for x in QUANTILES}
            q=min(scores,key=lambda x:(scores[x],x)); hist=scores[q]
        return CalibrationDecision(issue_index,q,margins[q],history_count,raw.copy(),
             candidates[q],candidates,hist)
    def observe(self,decision:CalibrationDecision,actual_block_kw:np.ndarray,price_block:np.
         ndarray)->None:
        actual=np.asarray(actual_block_kw,dtype=float); price=np.asarray(price_block,dtype=
             float)
        if actual.shape!=(36,) or price.shape!=(36,): raise ValueError("calibration block
             must contain 36 slots")
        issue=decision.issue_index
        if decision.history_days_used!=len(self.residual_days[issue]): raise ValueError("
             stale calibration decision")
        for q,forecast in decision.candidates.items():
            error=forecast[:36]-actual
            score=float(np.sum(price*(4*np.maximum(error,0)+np.maximum(-error,0)))*DT_HOURS)
            self.score_days[issue][q].append(score)
        raw_error=decision.raw_forecast_kw[:36]-actual
        mask=(decision.raw_forecast_kw[:36]>1e-9)|(actual>1e-9)
        self.residual_days[issue].append(raw_error[mask].copy())
```

B.15 指定日期论文表格生成（paper_tables.py）

Listing 15:

```python
from __future__ import annotations
import csv
from collections import defaultdict
from pathlib import Path
import numpy as np
import openpyxl
ROOT = Path(__file__).resolve().parents[1]
VARIANTS = ROOT / "outputs" / "variants" / "model_audit"
GENERATED = ROOT / "paper" / "generated"
TEMPLATE = ROOT.parent / "CUMCM2026Problems" / "C题" / "附件" / "附件5" / "result2.xlsx"
DATES = ("2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21")
INTERVALS = ("10:00-10:10", "12:00-12:10", "14:00-14:10", "16:00-16:10", "18:00-18:10", "20:00-20:10")
PERIODS = ("0:00--4:00", "4:00--8:00", "8:00--12:00", "12:00--16:00", "16:00--20:00", "20:00--24:00")
SOURCES = {
    "q2": VARIANTS / "q2" / "B_causalload_risk_feedback",
    "q3": VARIANTS / "q3_additive" / "S3_0_6_12_18",
    "q42": VARIANTS / "q4" / "q42_perfect_price",
    "q43": VARIANTS / "q4" / "q43_perfect_price",
}
def _rows(path: Path) -> list[dict[str, str]]:
    with path.open(encoding="utf-8-sig", newline="") as handle:
        return list(csv.DictReader(handle))
def _slot_indices() -> dict[str, int]:
    workbook = openpyxl.load_workbook(TEMPLATE, data_only=True, read_only=True)
    headers = list(next(workbook.worksheets[0].iter_rows(min_row=1, max_row=1, values_only=True)))
    workbook.close()
    return {label: headers.index(label) - 1 for label in INTERVALS}
def emergency_segments(values: np.ndarray, tol: float = 1e-7) -> list[tuple[str, float]]:
    output: list[tuple[str, float]] = []
    start = None
    def stamp(slot: int) -> str:
        minutes = slot * 10
        return "24:00" if minutes == 1440 else f"{minutes // 60}:{minutes % 60:02d}"
    for index, active in enumerate(np.r_[np.asarray(values, dtype=float) > tol, False]):
        if active and start is None:
            start = index
        elif not active and start is not None:
            output.append((f"{stamp(start)}--{stamp(index)}", float(np.sum(values[start:index]))))
            start = None
```

```
    return output
def _dataset(key: str) -> tuple[dict[str, list[dict[str, str]]], dict[str, dict[str, str]]]:
    grouped: dict[str, list[dict[str, str]]] = defaultdict(list)
    for row in _rows(SOURCES[key] / "solution.csv"):
        grouped[row["date"]].append(row)
    daily = {row["date"]: row for row in _rows(SOURCES[key] / "daily_metrics.csv")}
    missing = [date for date in DATES if date not in grouped or date not in daily]
    if missing:
        raise ValueError(f"{key} 缺少指定日期: {missing}")
    return grouped, daily
def _purchase_table(key: str, caption: str, label: str, adjusted: bool) -> str:
    grouped, daily = _dataset(key)
    indices = _slot_indices()
    lines = [
        r"\begin{table}[htbp]",
        r"\centering\zihao{-5}",
        rf"\caption{{{caption}}}",
        rf"\label{{{label}}}",
        r"\resizebox{\textwidth}{!}{%",
    ]
    if adjusted:
        lines += [
            r"\begin{tabular}{c c r r r r r r r r}",
            r"\toprule",
            r"日期 & 口径 & 10:00 & 12:00 & 14:00 & 16:00 & 18:00 & 20:00 & 全天/kWh & 费用/
                元 \\",
            r"\midrule",
        ]
    else:
        lines += [
            r"\begin{tabular}{c r r r r r r r r}",
            r"\toprule",
            r"日期 & 10:00 & 12:00 & 14:00 & 16:00 & 18:00 & 20:00 & 全天/kWh & 计划费/元 \\
                ",
            r"\midrule",
        ]
    for date in DATES:
        day = grouped[date]
        drow = daily[date]
        if adjusted:
            for row_index, (name, column, cost_field) in enumerate(
                (("计划", "baseline_grid_kwh", "baseline_cost"), ("调整", "adjusted_grid_kwh
                    ", "settlement_cost"))
            ):
                values = [float(day[indices[item]][column]) for item in INTERVALS]
                total = sum(float(row[column]) for row in day)
                date_cell = rf"\multirow{{2}}{{*}}{{{date}}}" if row_index == 0 else ""
```

```
                lines.append(
                    f"{date_cell} & {name} & " + " & ".join(f"{value:.2f}" for value in
                        values)
                    + f" & {total:.2f} & {float(drow[cost_field]):.2f} \\\\"
                )
        else:
            values = [float(day[indices[item]]["grid_plan_kwh"]) for item in INTERVALS]
            total = sum(float(row["grid_plan_kwh"]) for row in day)
            lines.append(
                f"{date} & " + " & ".join(f"{value:.2f}" for value in values)
                + f" & {total:.2f} & {float(drow['plan_cost']):.2f} \\\\"
            )
    lines += [r"\bottomrule", r"\end{tabular}}", r"\end{table}"]
    return "\n".join(lines)
def _storage_table(key: str, caption: str, label: str) -> str:
    grouped, daily = _dataset(key)
    lines = [
        r"\begin{table}[htbp]",
        r"\centering\zihao{-5}",
        rf"\caption{{{caption}}}",
        rf"\label{{{label}}}",
        r"\resizebox{\textwidth}{!}{%",
        r"\begin{tabular}{c r r r r r r r}",
        r"\toprule",
        r"日期 & \multicolumn{6}{c}{各 4 h 时段充电量/放电量（kWh）} & $S_{0:00}$ & $S_
            {24:00}$ \\",
        r"\cmidrule(lr){2-7}",
        "日期 & " + " & ".join(PERIODS) + r" & kWh & kWh \\",
        r"\midrule",
    ]
    for date in DATES:
        day = grouped[date]
        charge = np.asarray([float(row["charge_actual_kwh"]) for row in day])
        discharge = np.asarray([float(row["discharge_actual_kwh"]) for row in day])
        pairs = [f"{charge[i * 24:(i + 1) * 24].sum():.2f}/{discharge[i * 24:(i + 1) * 24].
            sum():.2f}" for i in range(6)]
        lines.append(
            f"{date} & " + " & ".join(pairs)
            + f" & {float(daily[date]['soc_initial']):.2f} & {float(daily[date]['
                soc_terminal']):.2f} \\\\"
        )
    lines += [r"\bottomrule", r"\end{tabular}}", r"\end{table}"]
    return "\n".join(lines)
def _emergency_table(key: str, caption: str, label: str) -> str:
    grouped, daily = _dataset(key)
    lines = [
        r"\begin{table}[htbp]",
```

```
        r"\centering\zihao{-5}",
        rf"\caption{{{caption}}}",
        rf"\label{{{label}}}",
        r"\begin{tabularx}{0.98\textwidth}{c >{\raggedright\arraybackslash}X r r}",
        r"\toprule",
        r"日期 & 紧急购电时段及对应电量/kWh & 合计/kWh & 当日总费用/元 \\",
        r"\midrule",
    ]
    for date in DATES:
        day = grouped[date]
        segments = emergency_segments(np.asarray([float(row["emergency_kwh"]) for row in day
            ]))
        description = "无" if not segments else "; ".join(f"{period}（{value:.2f}）" for
            period, value in segments)
        total = sum(value for _, value in segments)
        lines.append(f"{date} & {description} & {total:.2f} & {float(daily[date]['total_cost
            ']):.2f} \\\\ ")
    lines += [r"\bottomrule", r"\end{tabularx}", r"\end{table}"]
    return "\n".join(lines)
def _bundle(key: str, title: str, prefix: str, adjusted: bool) -> str:
    return "\n\n".join(
        (
            _purchase_table(key, f"{title}指定日期购电结果", f"tab:{prefix}-purchase",
                adjusted),
            _storage_table(key, f"{title}指定日期储能结果", f"tab:{prefix}-storage"),
            _emergency_table(key, f"{title}指定日期紧急购电结果", f"tab:{prefix}-emergency")
                ,
        )
    ) + "\n"
def build() -> None:
    GENERATED.mkdir(parents=True, exist_ok=True)
    outputs = {
        "q2_required_tables.tex": _bundle("q2", "问题二", "q2-required", False),
        "q3_required_tables.tex": _bundle("q3", "问题三", "q3-required", True),
        "q4_required_tables.tex": (
            _bundle("q42", "问题四对应问题二", "q42-required", False)
            + "\n"
            + _bundle("q43", "问题四对应问题三", "q43-required", True)
        ),
    }
    for name, content in outputs.items():
        path = GENERATED / name
        path.write_text("% 由 MathModel.solve.paper_tables 自动生成，勿手工修改。\n" +
            content, encoding="utf-8")
        print("[table]", path)
if __name__ == "__main__":
    build()
```

B.16 问题二结果验证（validate_q2.py）

Listing 16:

```
from __future__ import annotations
import csv,json
from pathlib import Path
import numpy as np
import openpyxl
ROOT=Path(__file__).resolve().parents[1]
OUT=ROOT/"outputs"/"q2"
def validate()->dict:
    with (OUT/"solution.csv").open(encoding="utf-8-sig",newline="") as f:
        rows=list(csv.DictReader(f))
    if len(rows)!=334*144: raise ValueError(f"solution rows: {len(rows)}")
    grid=np.asarray([float(r["grid_plan_kwh"]) for r in rows]).reshape(334,144)
    charge=np.asarray([float(r["charge_actual_kwh"]) for r in rows]).reshape(334,144)
    discharge=np.asarray([float(r["discharge_actual_kwh"]) for r in rows]).reshape(334,144)
    emergency=np.asarray([float(r["emergency_kwh"]) for r in rows]).reshape(334,144)
    with (OUT/"daily_metrics.csv").open(encoding="utf-8-sig",newline="") as f:
        daily=list(csv.DictReader(f))
    wb=openpyxl.load_workbook(OUT/"result2.xlsx",data_only=True,read_only=True)
    template=openpyxl.load_workbook(ROOT.parent/"CUMCM2026Problems"/"C题"/"附件"/"附件5"/"
        result2.xlsx",
                                    data_only=True,read_only=True)
    plan_rows=list(wb.worksheets[0].iter_rows(min_row=2,max_row=335,min_col=2,max_col=147,
        values_only=True))
    plan_values=np.asarray([r[:144] for r in plan_rows],dtype=float)
    total_values=np.asarray([r[144] for r in plan_rows],dtype=float)
    cost_values=np.asarray([r[145] for r in plan_rows],dtype=float)
    storage_rows=list(wb.worksheets[1].iter_rows(min_row=2,max_row=2005,min_col=1,max_col=6,
        values_only=True))
    storage_charge=np.asarray([float(r[2]) for r in storage_rows]).reshape(334,6)
    storage_discharge=np.asarray([float(r[3]) for r in storage_rows]).reshape(334,6)
    storage_initial=np.asarray([float(storage_rows[d*6][5]) for d in range(334)])
    storage_terminal=np.asarray([float(storage_rows[d*6+1][5]) for d in range(334)])
    expected_charge=charge.reshape(334,6,24).sum(axis=2)
    expected_discharge=discharge.reshape(334,6,24).sum(axis=2)
    emergency_rows=list(wb.worksheets[2].iter_rows(min_row=2,min_col=1,max_col=3,values_only
        =True))
    emergency_sheet_total=float(sum(float(r[2] or 0) for r in emergency_rows))
    expected_initial=np.asarray([float(r["soc_initial"]) for r in daily])
    expected_terminal=np.asarray([float(r["soc_terminal"]) for r in daily])
    expected_cost=np.asarray([float(r["plan_cost"]) for r in daily])
```

```
    result={
        "sheet_names_preserved":wb.sheetnames==template.sheetnames,
        "solution_rows":len(rows),"plan_shape":list(plan_values.shape),"storage_rows":len(
            storage_rows),
        "max_plan_diff":float(np.max(np.abs(plan_values-grid))),
        "max_plan_total_diff":float(np.max(np.abs(total_values-grid.sum(axis=1)))),
        "max_plan_cost_diff":float(np.max(np.abs(cost_values-expected_cost))),
        "max_storage_charge_diff":float(np.max(np.abs(storage_charge-expected_charge))),
        "max_storage_discharge_diff":float(np.max(np.abs(storage_discharge-
            expected_discharge))),
        "max_storage_initial_soc_diff":float(np.max(np.abs(storage_initial-expected_initial)
            )),
        "max_storage_terminal_soc_diff":float(np.max(np.abs(storage_terminal-
            expected_terminal))),
        "emergency_sheet_total_kwh":emergency_sheet_total,
        "emergency_csv_total_kwh":float(emergency.sum()),
        "emergency_total_diff":abs(emergency_sheet_total-float(emergency.sum())),
    }
    result["passed"]=bool(result["sheet_names_preserved"] and result["plan_shape"
        ]==[334,144] and
        len(storage_rows)==2004 and max(result[k] for k in result if k.startswith("max_"))<1
            e-7 and
        result["emergency_total_diff"]<1e-7)
    (OUT/"xlsx_verification.json").write_text(json.dumps(result,ensure_ascii=False,indent=2)
        ,encoding="utf-8")
    metrics=json.loads((OUT/"metrics.json").read_text(encoding="utf-8"))
    metrics["xlsx_passed"]=result["passed"]
    (OUT/"metrics.json").write_text(json.dumps(metrics,ensure_ascii=False,indent=2),encoding
        ="utf-8")
    return result
if __name__=="__main__": print(json.dumps(validate(),ensure_ascii=False,indent=2))
```

B.17 全部方案物理验证（validate_all.py）

Listing 17:

```
from __future__ import annotations
import csv,json
from pathlib import Path
import numpy as np
import openpyxl
ROOT=Path(__file__).resolve().parents[1]
DATA=ROOT.parent/"CUMCM2026Problems"/"C题"/"附件"/"附件5"
def _csv(path:Path)->list[dict]:
    with path.open(encoding="utf-8-sig",newline="") as f: return list(csv.DictReader(f))
def _maxdiff(a,b)->float: return float(np.max(np.abs(np.asarray(a,dtype=float)-np.asarray(b,
```

```
    dtype=float))))
def validate_q1()->dict:
    folder=ROOT/"outputs"/"q1"; rows=_csv(folder/"solution.csv")
    wb=openpyxl.load_workbook(folder/"result1.xlsx",data_only=True,read_only=True)
    template=openpyxl.load_workbook(DATA/"result1.xlsx",data_only=True,read_only=True)
    grid=np.asarray([float(r["grid_kwh"]) for r in rows])
    xgrid=np.asarray(list(wb.worksheets[0].iter_rows(min_row=2,max_row=145,min_col=2,max_col
        =2,values_only=True)),dtype=float).ravel()
    result={"rows":len(rows),"sheet_names_preserved":wb.sheetnames==template.sheetnames,
            "max_grid_diff":_maxdiff(grid,xgrid)}
    result["passed"]=bool(len(rows)==144 and result["sheet_names_preserved"] and result["
        max_grid_diff"]<1e-7)
    return result
def validate_annual(folder:Path,xlsx_name:str,kind:str)->dict:
    rows=_csv(folder/"solution.csv"); daily=_csv(folder/"daily_metrics.csv")
    if len(rows)!=334*144 or len(daily)!=334: raise ValueError(f"{folder}: row count
        mismatch")
    wb=openpyxl.load_workbook(folder/xlsx_name,data_only=True,read_only=True)
    template=openpyxl.load_workbook(DATA/xlsx_name,data_only=True,read_only=True)
    has_adjust=kind=="q3"; storage_index=2 if has_adjust else 1; emergency_index=3 if
        has_adjust else 2
    plan_field="baseline_grid_kwh" if has_adjust else "grid_plan_kwh"
    plan=np.asarray([float(r[plan_field]) for r in rows]).reshape(334,144)
    charge=np.asarray([float(r["charge_actual_kwh"]) for r in rows]).reshape(334,144)
    discharge=np.asarray([float(r["discharge_actual_kwh"]) for r in rows]).reshape(334,144)
    emergency=np.asarray([float(r["emergency_kwh"]) for r in rows]).reshape(334,144)
    plan_rows=list(wb.worksheets[0].iter_rows(min_row=2,max_row=335,min_col=2,max_col=147,
        values_only=True))
    xplan=np.asarray([r[:144] for r in plan_rows],dtype=float)
    xtotal=np.asarray([r[144] for r in plan_rows],dtype=float)
    xcost=np.asarray([r[145] for r in plan_rows],dtype=float)
    expected_plan_cost=np.asarray([float(r["baseline_cost"] if has_adjust else r["plan_cost"
        ]) for r in daily])
    result={"solution_rows":len(rows),"daily_rows":len(daily),"sheet_names_preserved":wb.
        sheetnames==template.sheetnames,
            "max_plan_diff":_maxdiff(xplan,plan),"max_plan_total_diff":_maxdiff(xtotal,plan.
                sum(axis=1)),
            "max_plan_cost_diff":_maxdiff(xcost,expected_plan_cost)}
    if has_adjust:
        adjusted=np.asarray([float(r["adjusted_grid_kwh"]) for r in rows]).reshape(334,144)
        adjust_rows=list(wb.worksheets[1].iter_rows(min_row=2,max_row=335,min_col=2,max_col
            =147,values_only=True))
        xadjust=np.asarray([r[:144] for r in adjust_rows],dtype=float)
        xa_total=np.asarray([r[144] for r in adjust_rows],dtype=float)
        xa_cost=np.asarray([r[145] for r in adjust_rows],dtype=float)
        expected_cost=np.asarray([float(r["settlement_cost"]) for r in daily])
        result.update({"max_adjust_diff":_maxdiff(xadjust,adjusted),
```

```
                        "max_adjust_total_diff":_maxdiff(xa_total,adjusted.sum(axis=1)),
                        "max_adjust_cost_diff":_maxdiff(xa_cost,expected_cost)})
    storage_rows=list(wb.worksheets[storage_index].iter_rows(min_row=2,max_row=2005,min_col
        =1,max_col=6,values_only=True))
    xcharge=np.asarray([float(r[2]) for r in storage_rows]).reshape(334,6)
    xdischarge=np.asarray([float(r[3]) for r in storage_rows]).reshape(334,6)
    xs0=np.asarray([float(storage_rows[d*6][5]) for d in range(334)])
    xs1=np.asarray([float(storage_rows[d*6+1][5]) for d in range(334)])
    result.update({"storage_rows":len(storage_rows),
        "max_storage_charge_diff":_maxdiff(xcharge,charge.reshape(334,6,24).sum(axis=2)),
        "max_storage_discharge_diff":_maxdiff(xdischarge,discharge.reshape(334,6,24).sum(
            axis=2)),
        "max_storage_initial_diff":_maxdiff(xs0,[float(r["soc_initial"]) for r in daily]),
        "max_storage_terminal_diff":_maxdiff(xs1,[float(r["soc_terminal"]) for r in daily])
            })
    erows=list(wb.worksheets[emergency_index].iter_rows(min_row=2,min_col=1,max_col=3,
        values_only=True))
    x_emergency=sum(float(r[2] or 0) for r in erows)
    result.update({"emergency_rows":len(erows),"emergency_sheet_total":x_emergency,
                   "emergency_csv_total":float(emergency.sum()),
                   "emergency_total_diff":abs(x_emergency-float(emergency.sum()))})
    numerical=[v for k,v in result.items() if k.startswith("max_") or k=="
        emergency_total_diff"]
    result["passed"]=bool(result["sheet_names_preserved"] and len(storage_rows)==2004 and
        max(numerical)<1e-7)
    return result
def validate_all()->dict:
    cases={
        "q1":validate_q1(),
        "q2":validate_annual(ROOT/"outputs"/"q2","result2.xlsx","q2"),
        "q3_main":validate_annual(ROOT/"outputs"/"q3","result3.xlsx","q3"),
        "q3_raw_step":validate_annual(ROOT/"outputs"/"q3"/"variants"/"raw_step","result3.
            xlsx","q3"),
        "q3_calibrated_step":validate_annual(ROOT/"outputs"/"q3"/"variants"/"calibrated_step
            ","result3.xlsx","q3"),
        "q4_2":validate_annual(ROOT/"outputs"/"q4-2","result4-2.xlsx","q2"),
        "q4_3":validate_annual(ROOT/"outputs"/"q4-3","result4-3.xlsx","q3"),
    }
    result={"cases":cases,"passed":all(v["passed"] for v in cases.values())}
    (ROOT/"outputs"/"verification_summary.json").write_text(json.dumps(result,ensure_ascii=
        False,indent=2),encoding="utf-8")
    return result
if __name__=="__main__": print(json.dumps(validate_all(),ensure_ascii=False,indent=2))
```

B.18 复现清单生成（build_manifest.py）

Listing 18:

```
from __future__ import annotations
import hashlib,json
from datetime import datetime
from pathlib import Path
ROOT=Path(__file__).resolve().parents[1]
OUTPUTS=ROOT/"outputs"
VARIANTS={
    "q1":{"path":"q1","parameters":{"question":1}},
    "q2_fixed_online":{"path":"q2","parameters":{"question":2,"price":"attachment1","forecast":"strictly_online"}},
    "q3_main_linear_calibrated":{"path":"q3","parameters":{"question":3,"downscale":"linear","calibration":True,"price":"attachment1"}},
    "q3_raw_step":{"path":"q3/variants/raw_step","parameters":{"question":3,"downscale":"step","calibration":False,"price":"attachment1"}},
    "q3_calibrated_step":{"path":"q3/variants/calibrated_step","parameters":{"question":3,"downscale":"step","calibration":True,"price":"attachment1"}},
    "q4_2_dynamic":{"path":"q4-2","parameters":{"question":"4-2","price":"attachment4","forecast":"strictly_online"}},
    "q4_3_dynamic":{"path":"q4-3","parameters":{"question":"4-3","price":"attachment4","downscale":"linear","calibration":True}},
}
def digest(path:Path)->str:
    h=hashlib.sha256()
    with path.open("rb") as f:
        for block in iter(lambda:f.read(1024*1024),b""): h.update(block)
    return h.hexdigest()
def build()->dict:
    result={"generated_at":datetime.now().astimezone().isoformat(),"authority_root":str(ROOT),
            "main":{"q1":"q1","q2":"q2_fixed_online","q3":"q3_main_linear_calibrated",
                    "q4_2":"q4_2_dynamic","q4_3":"q4_3_dynamic"},"variants":{},"source_files":{}}
    for name,spec in VARIANTS.items():
        folder=OUTPUTS/spec["path"]; files={}
        for path in sorted(folder.glob("*")):
            if path.is_file():
                files[path.name]={"bytes":path.stat().st_size,"sha256":digest(path)}
        metrics_path=folder/"metrics.json"
        metrics=json.loads(metrics_path.read_text(encoding="utf-8")) if metrics_path.exists() else None
        result["variants"][name]={"relative_path":spec["path"],"parameters":spec["parameters"],
                                  "metrics":metrics,"files":files}
    for path in sorted((ROOT/"solve").glob("*.py")):
        result["source_files"][str(path.relative_to(ROOT))]={"bytes":path.stat().st_size,"
```

```
            sha256":digest(path)}
    for pattern in ("reports/*.md","tests/*.py","README.md","requirements.txt","run_all.ps1"):
        for path in sorted(ROOT.glob(pattern)):
            if path.is_file(): result["source_files"][str(path.relative_to(ROOT))]={"bytes":path.stat().st_size,"sha256":digest(path)}
    result["figures"]={}
    for path in sorted((ROOT/"figures").rglob("*.pdf")):
        result["figures"][str(path.relative_to(ROOT))]={"bytes":path.stat().st_size,"sha256":digest(path)}
    result["governance_outputs"]={}
    for name in ("verification_summary.json","q4_comparison.json"):
        path=OUTPUTS/name
        if path.exists(): result["governance_outputs"][name]={"bytes":path.stat().st_size,"sha256":digest(path)}
    (OUTPUTS/"MANIFEST.json").write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding="utf-8")
    return result
if __name__=="__main__":
    manifest=build(); print(json.dumps({"variants":list(manifest["variants"]),"source_files":len(manifest["source_files"]),"manifest":str(OUTPUTS/"MANIFEST.json")},ensure_ascii=False,indent=2))
```

B.19 科研绘图公共样式（plot_style.py）

Listing 19:

```
from __future__ import annotations
from pathlib import Path
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
FIGURES = Path(__file__).resolve().parents[1] / "paper" / "figures"
FIGURES.mkdir(parents=True, exist_ok=True)
PALETTE = {
    "navy": "#1F3B73",
    "blue": "#3E6FB0",
    "sky": "#7FB2E5",
    "teal": "#2E8B84",
    "green": "#5FA05A",
    "olive": "#9BAF3B",
    "gold": "#D8A22B",
    "orange": "#E07B39",
    "red": "#C0392B",
    "rose": "#D46A8C",
    "purple": "#7A5BA6",
```

```
    "gray": "#7F8C99",
    "lightgray": "#D5DBE1",
    "ink": "#1A2028",
}
CYCLE = [PALETTE[k] for k in ("navy", "orange", "teal", "red", "purple", "gold", "sky", "
    green")]
SCENARIO_COLORS = {
    "Q1": PALETTE["navy"],
    "Q2": PALETTE["orange"],
    "Q3": PALETTE["teal"],
    "Q4-2": PALETTE["red"],
    "Q4-3": PALETTE["purple"],
    "legacy": PALETTE["gray"],
}
def apply_style() -> None:
    plt.rcParams.update(
        {
            "font.family": "sans-serif",
            "font.sans-serif": ["SimSun", "Times New Roman", "DejaVu Sans"],
            "font.serif": ["SimSun", "Times New Roman"],
            "axes.unicode_minus": False,
            "font.size": 11,
            "axes.titlesize": 12.5,
            "axes.labelsize": 11.5,
            "axes.titleweight": "bold",
            "legend.fontsize": 10,
            "xtick.labelsize": 10,
            "ytick.labelsize": 10,
            "axes.linewidth": 0.9,
            "axes.edgecolor": PALETTE["ink"],
            "axes.labelcolor": PALETTE["ink"],
            "text.color": PALETTE["ink"],
            "xtick.color": PALETTE["ink"],
            "ytick.color": PALETTE["ink"],
            "axes.grid": True,
            "grid.color": PALETTE["lightgray"],
            "grid.linewidth": 0.6,
            "grid.alpha": 0.75,
            "axes.axisbelow": True,
            "figure.dpi": 130,
            "savefig.dpi": 330,
            "savefig.bbox": "tight",
            "savefig.facecolor": "white",
            "figure.facecolor": "white",
            "axes.spines.top": False,
            "axes.spines.right": False,
            "legend.frameon": False,
```

```
            "lines.linewidth": 1.8,
            "lines.markersize": 4.5,
        }
    )
def panel_label(ax, text: str, dx: float = -0.085, dy: float = 1.06) -> None:
    ax.text(dx, dy, text, transform=ax.transAxes, fontsize=12.5, fontweight="bold",
            va="bottom", ha="left", color=PALETTE["ink"])
def save(fig, name: str, *, pdf: bool = True) -> Path:
    png = FIGURES / f"{name}.png"
    fig.savefig(png)
    if pdf:
        fig.savefig(FIGURES / f"{name}.pdf")
    plt.close(fig)
    return png
def wan(value: float) -> float:
    return value / 1e4
```

B.20 数据总览绘图（fig_data.py）

Listing 20:

```
from __future__ import annotations
import sys
from pathlib import Path
import numpy as np
sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
from MathModel.solve.io_data import (
    DEFAULT_DATA_ROOT,
    DT_HOURS,
    hourly_to_ten_min,
    read_attachment1,
    read_attachment2,
    read_attachment3,
    read_attachment4,
)
from MathModel.solve.plot_style import PALETTE, apply_style, panel_label, save
apply_style()
import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.colors import TwoSlopeNorm
MONTH_EDGES = np.cumsum([0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31])
MONTH_LABELS = [f"{m}月" for m in range(1, 13)]
DAYS_PER_MONTH = np.diff(MONTH_EDGES)
def fig_data_overview(root: Path = DEFAULT_DATA_ROOT) -> None:
    day1 = read_attachment1(root)
```

```
    annual = read_attachment2(root)
    price = read_attachment4(root)
    fig = plt.figure(figsize=(13.6, 8.2))
    gs = fig.add_gridspec(2, 2, hspace=0.42, wspace=0.22)
    ax = fig.add_subplot(gs[0, 0])
    hours = (np.arange(144) + 1) / 6.0
    ax.plot(hours, day1.load_kw / 1e3, color=PALETTE["navy"], label="小区负载")
    ax.plot(hours, day1.pv_kw / 1e3, color=PALETTE["gold"], label="光伏发电预测")
    ax.set_xlabel("时刻 / h")
    ax.set_ylabel("功率 / MW")
    ax.set_title("典型日负载与光伏出力的错配")
    ax.set_xlim(0, 24)
    ax.set_xticks(range(0, 25, 4))
    ax2 = ax.twinx()
    ax2.plot(hours, day1.price, color=PALETTE["red"], linestyle="--", linewidth=1.5, label="
        外网电价")
    ax2.set_ylabel("电价 /（元·kWh$^{-1}$）", color=PALETTE["red"])
    ax2.tick_params(axis="y", colors=PALETTE["red"])
    ax2.grid(False)
    ax2.spines["right"].set_visible(True)
    handles1, labels1 = ax.get_legend_handles_labels()
    handles2, labels2 = ax2.get_legend_handles_labels()
    ax.legend(handles1 + handles2, labels1 + labels2, loc="upper left", ncol=1, fontsize=9)
    panel_label(ax, "(a)")
    for idx, (data, title, cmap, unit) in enumerate(
        [
            (annual.load_kw, "全年负载分布", "YlGnBu", "kW"),
            (annual.pv_kw, "全年光伏出力分布", "YlOrBr", "kW"),
            (price.price, "全年实时电价分布", "RdYlBu_r", "元/kWh"),
        ]
    ):
        ax = fig.add_subplot(gs[(1 + idx) // 2, (1 + idx) % 2])
        im = ax.imshow(
            data / (1e3 if unit == "kW" else 1),
            aspect="auto",
            cmap=cmap,
            interpolation="nearest",
            extent=[0, 24, 365, 1],
        )
        ax.set_xlabel("时刻 / h")
        ax.set_ylabel("日期（第 n 天）")
        ax.set_title(title)
        ax.set_xticks(range(0, 25, 4))
        ax.grid(False)
        cb = fig.colorbar(im, ax=ax, pad=0.02)
        cb.set_label("MW" if unit == "kW" else "元/kWh", fontsize=9)
        cb.ax.tick_params(labelsize=8.5)
```

```
        panel_label(ax, "(%s)" % "bcd"[idx])
    save(fig, "data_overview")
def fig_netload_season(root: Path = DEFAULT_DATA_ROOT) -> None:
    annual = read_attachment2(root)
    net = annual.load_kw - annual.pv_kw
    month_of_day = np.repeat(np.arange(12), DAYS_PER_MONTH)
    fig = plt.figure(figsize=(13.6, 4.4))
    gs = fig.add_gridspec(1, 3, wspace=0.28)
    ax = fig.add_subplot(gs[0, 0])
    data = [net[month_of_day == m].ravel() / 1e3 for m in range(12)]
    bp = ax.boxplot(data, patch_artist=True, widths=0.62, showfliers=False,
                    medianprops=dict(color=PALETTE["ink"], linewidth=1.3),
                    whiskerprops=dict(color=PALETTE["gray"]),
                    capprops=dict(color=PALETTE["gray"]),
                    boxprops=dict(facecolor=PALETTE["sky"], edgecolor=PALETTE["navy"],
                        linewidth=0.9))
    ax.axhline(0, color=PALETTE["red"], linewidth=1.1, linestyle="--")
    ax.set_xticks(range(1, 13))
    ax.set_xticklabels([f"{m}" for m in range(1, 13)], fontsize=9)
    ax.set_xlabel("月份")
    ax.set_ylabel("净负荷 / MW")
    ax.set_title("净负荷的月度分布")
    panel_label(ax, "(a)")
    ax = fig.add_subplot(gs[0, 1])
    penetration = annual.pv_kw / np.maximum(annual.load_kw, 1e-9)
    data = [np.clip(penetration[month_of_day == m].ravel(), 0, 3) for m in range(12)]
    parts = ax.violinplot(data, showmedians=True, widths=0.85)
    for body in parts["bodies"]:
        body.set_facecolor(PALETTE["gold"])
        body.set_edgecolor(PALETTE["orange"])
        body.set_alpha(0.75)
    for key in ("cmedians", "cbars", "cmins", "cmaxes"):
        parts[key].set_color(PALETTE["ink"])
        parts[key].set_linewidth(1.1)
    ax.axhline(1.0, color=PALETTE["red"], linestyle="--", linewidth=1.1)
    ax.set_xticks(range(1, 13))
    ax.set_xticklabels([f"{m}" for m in range(1, 13)], fontsize=9)
    ax.set_xlabel("月份")
    ax.set_ylabel("光伏/负载 比值")
    ax.set_title("光伏渗透率的月度分布")
    ax.set_ylim(0, 3)
    panel_label(ax, "(b)")
    ax = fig.add_subplot(gs[0, 2])
    surplus_slots = (annual.pv_kw - annual.load_kw > 0).sum(axis=1)
    big_surplus = (annual.pv_kw - annual.load_kw > 5000).sum(axis=1)
    hours_surplus = np.array([surplus_slots[month_of_day == m].mean() * 10 / 60 for m in
        range(12)])
```

```
    hours_big = np.array([big_surplus[month_of_day == m].mean() * 10 / 60 for m in range(12)])
    x = np.arange(12)
    ax.bar(x - 0.2, hours_surplus, width=0.4, color=PALETTE["green"], label="光伏富余时段")
    ax.bar(x + 0.2, hours_big, width=0.4, color=PALETTE["red"], label="富余超 5000 kW 时段")
    ax.set_xticks(x)
    ax.set_xticklabels([f"{m}" for m in range(1, 13)], fontsize=9)
    ax.set_xlabel("月份")
    ax.set_ylabel("日均时长 / h")
    ax.set_title("光伏富余的月度强度")
    ax.legend(fontsize=9)
    panel_label(ax, "(c)")
    save(fig, "netload_season")
def fig_forecast_diagnosis(root: Path = DEFAULT_DATA_ROOT) -> None:
    annual = read_attachment2(root)
    forecasts = read_attachment3(root)
    month_of_day = np.repeat(np.arange(12), DAYS_PER_MONTH)
    err = np.zeros_like(forecasts.hourly_kw)
    for node in range(4):
        for hour in range(24):
            target_slot = (node * 36 + (hour + 1) * 6 - 1) % 144
            target_day = np.arange(365) + (node * 36 + (hour + 1) * 6 - 1) // 144
            valid = target_day < 365
            err[valid, node, hour] = forecasts.hourly_kw[valid, node, hour] - annual.pv_kw[
                target_day[valid], target_slot]
    fig = plt.figure(figsize=(13.6, 4.6))
    gs = fig.add_gridspec(1, 3, wspace=0.28)
    ax = fig.add_subplot(gs[0, 0])
    labels = ["0:00", "6:00", "12:00", "18:00"]
    colors = [PALETTE["navy"], PALETTE["orange"], PALETTE["teal"], PALETTE["purple"]]
    for n, (label, color) in enumerate(zip(labels, colors)):
        values = np.sort(np.abs(err[:, n, :].ravel()))
        values = np.maximum(values, 1e-3)
        share = np.arange(1, values.size + 1) / values.size
        step = max(1, values.size // 4000)
        ax.plot(values[::step], share[::step], color=color, linewidth=2.1, label=label)
        p95 = np.percentile(values, 95)
        ax.plot([p95], [0.95], marker="o", color=color, markersize=6,
                markeredgecolor="white", markeredgewidth=1.0, zorder=5)
    ax.axhline(0.95, color=PALETTE["gray"], linestyle=":", linewidth=1.0)
    ax.text(1.6, 0.955, "$P_{95}$", fontsize=9, color=PALETTE["gray"])
    ax.set_xscale("log")
    ax.set_xlim(1, 6000)
    ax.set_ylim(0, 1.0)
    ax.set_xlabel("绝对预报误差 / kW（对数轴）")
    ax.set_ylabel("经验累积分布 ECDF")
    ax.set_title("分发布时刻的预报误差累积分布")
```

```
    ax.legend(fontsize=9, loc="lower right", title="发布时刻", title_fontsize=9)
    panel_label(ax, "(a)")
    ax = fig.add_subplot(gs[0, 1])
    mae = [np.mean(np.abs(err[month_of_day == m])) for m in range(12)]
    bias = [np.mean(err[month_of_day == m]) for m in range(12)]
    x = np.arange(12)
    ax.bar(x, mae, color=PALETTE["sky"], edgecolor=PALETTE["navy"], linewidth=0.7, label="
        MAE")
    ax.plot(x, bias, color=PALETTE["red"], marker="o", markersize=4.5, label="平均偏差")
    ax.axhline(0, color=PALETTE["ink"], linewidth=0.8)
    ax.set_xticks(x)
    ax.set_xticklabels([f"{m}" for m in range(1, 13)], fontsize=9)
    ax.set_xlabel("月份")
    ax.set_ylabel("误差 / kW")
    ax.set_title("预报误差的月度演变")
    ax.legend(fontsize=9)
    panel_label(ax, "(b)")
    ax = fig.add_subplot(gs[0, 2])
    raw_step, raw_linear = [], []
    for d in range(365):
        for n in range(4):
            lo = n * 36
            hi = lo + 36
            actual = annual.pv_kw[d, lo:hi]
            raw_step.append(np.mean(np.abs(hourly_to_ten_min(forecasts.hourly_kw[d, n], "
                step")[:36] - actual)))
            raw_linear.append(np.mean(np.abs(hourly_to_ten_min(forecasts.hourly_kw[d, n], "
                linear")[:36] - actual)))
    mae_step, mae_linear = float(np.mean(raw_step)), float(np.mean(raw_linear))
    bars = ax.bar(["阶梯保持", "线性插值"], [mae_step, mae_linear],
                  color=[PALETTE["gray"], PALETTE["teal"]], width=0.5,
                  edgecolor=PALETTE["ink"], linewidth=0.8)
    for bar, value in zip(bars, [mae_step, mae_linear]):
        ax.text(bar.get_x() + bar.get_width() / 2, value * 1.02, f"{value:.1f} kW",
                ha="center", va="bottom", fontsize=10, fontweight="bold")
    ax.set_ylabel("执行块 MAE / kW")
    ax.set_title("小时预报降尺度方法的精度对比")
    ax.set_ylim(0, max(mae_step, mae_linear) * 1.25)
    panel_label(ax, "(c)")
    save(fig, "forecast_diagnosis")
    print(f"[fig] 降尺度执行块 MAE: 阶梯={mae_step:.4f} kW, 线性={mae_linear:.4f} kW")
def fig_correlation(root: Path = DEFAULT_DATA_ROOT) -> None:
    annual = read_attachment2(root)
    price = read_attachment4(root)
    load = annual.load_kw.ravel()
    pv = annual.pv_kw.ravel()
    net = load - pv
```

```
    p = price.price.ravel()
    slot_phase = np.tile(np.arange(144) / 144.0, 365)
    frame = {
        "小区负载": load, "光伏出力": pv, "净负荷": net,
        "实时电价": p, "日内相位": slot_phase,
    }
    keys = list(frame)
    matrix = np.corrcoef(np.stack([frame[k] for k in keys]))
    matrix = matrix / np.sqrt(np.outer(np.diag(matrix), np.diag(matrix)))
    fig = plt.figure(figsize=(13.0, 4.8))
    gs = fig.add_gridspec(1, 2, width_ratios=[1.0, 1.15], wspace=0.3)
    ax = fig.add_subplot(gs[0, 0])
    im = ax.imshow(matrix, cmap="RdBu_r", vmin=-1, vmax=1)
    ax.set_xticks(range(len(keys)))
    ax.set_xticklabels(keys, rotation=32, ha="right", fontsize=9)
    ax.set_yticks(range(len(keys)))
    ax.set_yticklabels(keys, fontsize=9)
    for i in range(len(keys)):
        for j in range(len(keys)):
            ax.text(j, i, f"{matrix[i, j]:.2f}", ha="center", va="center", fontsize=9.5,
                    color="white" if abs(matrix[i, j]) > 0.55 else PALETTE["ink"])
    ax.grid(False)
    ax.set_title("关键变量的相关系数矩阵")
    cb = fig.colorbar(im, ax=ax, pad=0.02, shrink=0.9)
    cb.set_label("Pearson 相关系数", fontsize=9)
    panel_label(ax, "(a)")
    ax = fig.add_subplot(gs[0, 1])
    sample = np.random.default_rng(0).choice(load.size, size=min(50000, load.size), replace=
        False)
    hb = ax.hexbin(load[sample] / 1e3, net[sample] / 1e3, gridsize=48, cmap="viridis",
                   bins="log", mincnt=1, linewidths=0)
    ax.axhline(0, color=PALETTE["red"], linestyle="--", linewidth=1.2)
    ax.set_xlabel("小区负载 / MW")
    ax.set_ylabel("净负荷 / MW")
    ax.set_title("负载—净负荷联合分布（含零线）")
    cb = fig.colorbar(hb, ax=ax, pad=0.02, shrink=0.9)
    cb.set_label("时段数（对数）", fontsize=9)
    panel_label(ax, "(b)")
    save(fig, "correlation_structure")
def main() -> None:
    root = DEFAULT_DATA_ROOT
    fig_data_overview(root)
    fig_netload_season(root)
    fig_forecast_diagnosis(root)
    fig_correlation(root)
    print("[fig] 数据层配图完成 →", (Path(__file__).resolve().parents[1] / "paper" / "
        figures"))
```

```
if __name__ == "__main__":
    main()
```

B.21 问题一绘图（fig_q1.py）

Listing 21:

```
from __future__ import annotations
import sys
from pathlib import Path
import numpy as np
import pandas as pd
sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
from MathModel.solve.io_data import DT_HOURS, read_attachment1
from MathModel.solve.plot_style import PALETTE, apply_style, panel_label, save
apply_style()
import matplotlib.pyplot as plt
ROOT = Path(__file__).resolve().parents[1]
def main() -> None:
    frame = pd.read_csv(ROOT / "outputs" / "q1" / "solution.csv", encoding="utf-8-sig")
    day1 = read_attachment1()
    hours = frame["slot"].to_numpy() / 6.0
    fig = plt.figure(figsize=(13.6, 7.6))
    gs = fig.add_gridspec(2, 2, hspace=0.42, wspace=0.24)
    ax = fig.add_subplot(gs[0, 0])
    ax.plot(hours, day1.load_kw / 1e3, color=PALETTE["navy"], linewidth=1.9, label="小区负载
        ")
    ax.plot(hours, day1.pv_kw / 1e3, color=PALETTE["gold"], linewidth=1.9, label="光伏预测出
        力")
    ax.fill_between(hours, day1.pv_kw / 1e3, day1.load_kw / 1e3,
                    where=(day1.pv_kw < day1.load_kw), color=PALETTE["sky"], alpha=0.35,
                        label="净负荷缺口")
    ax.fill_between(hours, day1.pv_kw / 1e3, day1.load_kw / 1e3,
                    where=(day1.pv_kw >= day1.load_kw), color=PALETTE["green"], alpha=0.32,
                        label="光伏富余")
    ax.set_xlabel("时刻 / h")
    ax.set_ylabel("功率 / MW")
    ax.set_title("典型日净负荷结构")
    ax.set_xlim(0, 24)
    ax.legend(fontsize=8.5, ncol=2)
    panel_label(ax, "(a)")
    ax = fig.add_subplot(gs[0, 1])
    no_storage = np.maximum((day1.load_kw - day1.pv_kw) * DT_HOURS, 0)
    ax.plot(hours, frame["grid_kwh"] * 6 / 1e3, color=PALETTE["navy"], linewidth=1.9, label=
        "含储能最优购电")
    ax.plot(hours, no_storage * 6 / 1e3, color=PALETTE["gray"], linewidth=1.6, linestyle="--
```

```
        ", label="无储能基准购电")
    ax.fill_between(hours, frame["grid_kwh"] * 6 / 1e3, no_storage * 6 / 1e3,
                    where=(no_storage > frame["grid_kwh"]), color=PALETTE["teal"], alpha
                        =0.25, label="储能削减的购电")
    ax.set_xlabel("时刻 / h")
    ax.set_ylabel("购电功率 / MW")
    ax.set_title("储能对购电曲线的削峰填谷")
    ax.set_xlim(0, 24)
    ax.legend(fontsize=8.5)
    panel_label(ax, "(b)")
    ax = fig.add_subplot(gs[1, 0])
    ax.bar(hours, frame["charge_kwh"], width=0.13, color=PALETTE["purple"],
           edgecolor=PALETTE["ink"], linewidth=0.5, label="充电量")
    ax.bar(hours, -frame["discharge_kwh"], width=0.13, color=PALETTE["teal"],
           edgecolor=PALETTE["ink"], linewidth=0.5, label="放电量")
    ax.axhline(0, color=PALETTE["ink"], linewidth=0.8)
    ax.axhline(833.33, color=PALETTE["red"], linestyle=":", linewidth=1.0)
    ax.axhline(-833.33, color=PALETTE["red"], linestyle=":", linewidth=1.0)
    ax.text(0.2, 860, "单时段功率上限 833.33 kWh", fontsize=8, color=PALETTE["red"])
    ax.set_xlabel("时刻 / h")
    ax.set_ylabel("电量 / kWh（10 min）")
    ax.set_title("储能充放电计划")
    ax.set_xlim(0, 24)
    ax.legend(fontsize=9)
    panel_label(ax, "(c)")
    ax = fig.add_subplot(gs[1, 1])
    ax.plot(hours, frame["soc_kwh"] / 1e3, color=PALETTE["navy"], linewidth=2.2)
    ax.fill_between(hours, frame["soc_kwh"] / 1e3, 1.2, color=PALETTE["sky"], alpha=0.3)
    ax.axhline(1.2, color=PALETTE["red"], linestyle="--", linewidth=1.1)
    ax.axhline(10.8, color=PALETTE["red"], linestyle="--", linewidth=1.1)
    ax.axhline(6.0, color=PALETTE["gray"], linestyle=":", linewidth=1.0)
    ax.text(0.2, 1.35, "下限 1200 kWh", fontsize=8.5, color=PALETTE["red"])
    ax.text(0.2, 10.95, "上限 10800 kWh", fontsize=8.5, color=PALETTE["red"])
    ax.text(0.2, 6.15, "首末储电量 6000 kWh", fontsize=8.5, color=PALETTE["gray"])
    ax.set_xlabel("时刻 / h")
    ax.set_ylabel("储电量 / MWh")
    ax.set_title("储电量日内轨迹（首末闭合）")
    ax.set_xlim(0, 24)
    ax.set_ylim(0.5, 12.2)
    panel_label(ax, "(d)")
    save(fig, "q1_dispatch")
    print("[fig] q1_dispatch 完成")
if __name__ == "__main__":
    main()
```

B.22 结果与消融绘图（fig_results.py）

Listing 22:

```python
from __future__ import annotations
import json
import sys
from pathlib import Path
import numpy as np
import pandas as pd
sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
from MathModel.solve.plot_style import (
    CYCLE,
    PALETTE,
    SCENARIO_COLORS,
    apply_style,
    panel_label,
    save,
)
apply_style()
import matplotlib.pyplot as plt
ROOT = Path(__file__).resolve().parents[1]
VARIANTS = ROOT / "outputs" / "variants" / "model_audit"
FIGS = ROOT / "paper" / "figures"
Q2_ORDER = ["A_perfectload_risk_feedback", "B_causalload_risk_feedback",
            "C_causalload_point_feedback", "D_causalload_risk_nofeedback"]
Q2_LABEL = {
    "A_perfectload_risk_feedback": "A 完美负载\n风险边际•有反馈",
    "B_causalload_risk_feedback": "B 因果负载\n风险边际•有反馈",
    "C_causalload_point_feedback": "C 因果负载\n点预测•有反馈",
    "D_causalload_risk_nofeedback": "D 因果负载\n风险边际•无反馈",
}
Q3_ORDER = ["S0_0", "S1_0_6", "S2_0_6_12", "S3_0_6_12_18"]
Q3_LABEL = {"S0_0": "S0\n仅0:00", "S1_0_6": "S1\n+6:00", "S2_0_6_12": "S2\n+12:00", "
    S3_0_6_12_18": "S3\n+18:00"}
Q4_ORDER = ["q42_perfect_price", "q42_causal_price", "q43_perfect_price", "q43_causal_price"
    ]
Q4_LABEL = {"q42_perfect_price": "Q4-2\n附件4直用", "q42_causal_price": "Q4-2\n因果扩展",
            "q43_perfect_price": "Q4-3\n附件4直用", "q43_causal_price": "Q4-3\n因果扩展"}
def load(path: Path) -> dict:
    return json.loads(path.read_text(encoding="utf-8"))
def load_group(*parts: str) -> dict:
    *dirs, stem = parts
    return load(VARIANTS.joinpath(*dirs) / f"{stem}.json")
def _bar_labels(ax, bars, values, fmt="{:.2f}", dy=0.012, fontsize=9.5):
    span = ax.get_ylim()[1] - ax.get_ylim()[0]
    for bar, value in zip(bars, values):
```

```
        ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + span * dy,
                fmt.format(value), ha="center", va="bottom", fontsize=fontsize, fontweight="bold")
def fig_q2_attribution() -> None:
    summary = load_group("q2", "summary")
    totals = [summary[k]["total_cost_yuan"] / 1e4 for k in Q2_ORDER]
    emergencies = [summary[k]["emergency_energy_kwh"] / 1e3 for k in Q2_ORDER]
    curtails = [summary[k]["curtailment_energy_kwh"] / 1e3 for k in Q2_ORDER]
    c = {k: summary[k]["total_cost_yuan"] / 1e4 for k in Q2_ORDER}
    fig = plt.figure(figsize=(13.8, 8.6))
    gs = fig.add_gridspec(2, 2, hspace=0.48, wspace=0.26)
    ax = fig.add_subplot(gs[0, 0])
    colors = [PALETTE["blue"], PALETTE["teal"], PALETTE["yellow"] if "yellow" in PALETTE else PALETTE["gold"], PALETTE["red"]]
    bars = ax.bar(range(4), totals, color=colors, edgecolor=PALETTE["ink"], linewidth=0.8, width=0.62)
    ax.set_xticks(range(4))
    ax.set_xticklabels([Q2_LABEL[k] for k in Q2_ORDER], fontsize=9)
    ax.set_ylabel("全年总费用 / 万元")
    ax.set_title("问题二四种信息集/机制组合的总费用")
    ax.set_ylim(0, max(totals) * 1.16)
    _bar_labels(ax, bars, totals, "{:.1f}")
    panel_label(ax, "(a)")
    ax = fig.add_subplot(gs[0, 1])
    x = np.arange(4)
    bars = ax.bar(x - 0.19, emergencies, width=0.38, color=PALETTE["orange"],
                  edgecolor=PALETTE["ink"], linewidth=0.7, label="紧急购电")
    bars2 = ax.bar(x + 0.19, curtails, width=0.38, color=PALETTE["sky"],
                   edgecolor=PALETTE["ink"], linewidth=0.7, label="弃光")
    ax.set_xticks(x)
    ax.set_xticklabels([Q2_LABEL[k] for k in Q2_ORDER], fontsize=9)
    ax.set_ylabel("电量 / MWh")
    ax.set_title("紧急购电与弃光电量")
    ax.legend(fontsize=9)
    _bar_labels(ax, bars, emergencies, "{:.0f}")
    panel_label(ax, "(b)")
    ax = fig.add_subplot(gs[1, 0])
    delta_load = c["B_causalload_risk_feedback"] - c["A_perfectload_risk_feedback"]
    delta_pv = c["C_causalload_point_feedback"] - c["B_causalload_risk_feedback"]
    delta_fb = c["D_causalload_risk_nofeedback"] - c["B_causalload_risk_feedback"]
    labels = ["完美负载信息\n$\\Delta C_{load}$", "去掉光伏风险边际\n$\\Delta C_{PVrisk}$",
              "去掉实时储能反馈\n$\\Delta C_{feedback}$"]
    values = [delta_load, delta_pv, delta_fb]
    bars = ax.bar(range(3), values, width=0.55,
                  color=[PALETTE["red"], PALETTE["orange"], PALETTE["gold"]],
                  edgecolor=PALETTE["ink"], linewidth=0.8)
    _bar_labels(ax, bars, values, "{:+.1f}")
```

```
    for i, value in enumerate(values):
        share = 100 * value / c["B_causalload_risk_feedback"]
        ax.text(i, value * 0.5, f"占因果基准\n{share:.1f}%", ha="center", va="center",
                fontsize=9, color="white", fontweight="bold")
    ax.set_xticks(range(3))
    ax.set_xticklabels(labels, fontsize=8.8)
    ax.set_ylabel("费用增量 / 万元")
    ax.set_title("三项机制各自的费用贡献（万元）")
    ax.set_ylim(0, max(values) * 1.24)
    panel_label(ax, "(c)")
    ax = fig.add_subplot(gs[1, 1])
    rows = []
    for key in Q2_ORDER:
        m = summary[key]
        rows.append([m["plan_purchase_cost_yuan"] / 1e4, m["emergency_cost_yuan"] / 1e4])
    rows = np.asarray(rows)
    ax.bar(range(4), rows[:, 0], width=0.62, color=PALETTE["navy"], label="计划购电费",
           edgecolor=PALETTE["ink"], linewidth=0.8)
    ax.bar(range(4), rows[:, 1], bottom=rows[:, 0], width=0.62, color=PALETTE["orange"],
           label="紧急购电费", edgecolor=PALETTE["ink"], linewidth=0.8)
    for i, (plan, em) in enumerate(rows):
        ax.text(i, plan / 2, f"{plan:.0f}", ha="center", va="center", color="white",
                fontsize=9.5, fontweight="bold")
        ax.text(i, plan + em + max(rows[:, 0]) * 0.012, f"{plan + em:.1f}", ha="center", va=
                "bottom",
                fontsize=9.5, fontweight="bold")
    ax.set_xticks(range(4))
    ax.set_xticklabels([Q2_LABEL[k] for k in Q2_ORDER], fontsize=9)
    ax.set_ylabel("费用 / 万元")
    ax.set_title("费用构成：计划购电与紧急购电")
    ax.set_ylim(0, (rows.sum(axis=1).max()) * 1.16)
    ax.legend(fontsize=9)
    panel_label(ax, "(d)")
    save(fig, "q2_attribution")
def fig_q3_forecast_value() -> None:
    summary = load_group("q3_additive", "summary")
    costs = np.array([summary[k]["total_cost_yuan"] / 1e4 for k in Q3_ORDER])
    fig = plt.figure(figsize=(13.8, 8.6))
    gs = fig.add_gridspec(2, 2, hspace=0.46, wspace=0.26)
    ax = fig.add_subplot(gs[0, 0])
    ax.plot(range(4), costs, marker="o", color=PALETTE["navy"], markersize=7, linewidth=2.2)
    for i, value in enumerate(costs):
        ax.annotate(f"{value:.1f}", (i, value), textcoords="offset points", xytext=(0, 10),
                    ha="center", fontsize=10, fontweight="bold")
    ax.set_xticks(range(4))
    ax.set_xticklabels([Q3_LABEL[k] for k in Q3_ORDER], fontsize=9)
    ax.set_xlabel("可用的决策更新节点")
```

```
    ax.set_ylabel("全年总费用 / 万元")
    ax.set_title("新增决策节点带来的费用下降")
    ax.set_ylim(costs.min() * 0.965, costs.max() * 1.022)
    panel_label(ax, "(a)")
    ax = fig.add_subplot(gs[0, 1])
    controls = [summary["C1_load6"]["total_cost_yuan"], summary["C2_load12"]["
        total_cost_yuan"],
                summary["C3_load18"]["total_cost_yuan"]]
    load_value = np.array([summary[Q3_ORDER[i]]["total_cost_yuan"] - controls[i] for i in
        range(3)]) / 1e4
    pv_value = np.array([controls[i] - summary[Q3_ORDER[i + 1]]["total_cost_yuan"] for i in
        range(3)]) / 1e4
    labels = ["$V_6$", "$V_{12}$", "$V_{18}$"]
    bars = ax.bar(labels, load_value, color=PALETTE["orange"],
                  edgecolor=PALETTE["ink"], linewidth=0.8, width=0.55, label="负载观测刷新")
    ax.bar(labels, pv_value, bottom=load_value, color=PALETTE["teal"],
           edgecolor=PALETTE["ink"], linewidth=0.8, width=0.55, label="光伏预报刷新")
    values = load_value + pv_value
    ax.axhline(0, color=PALETTE["ink"], linewidth=0.9)
    for index, value in enumerate(values):
        ax.text(index, value + max(values) * 0.025, f"{value:+.1f}", ha="center", va="bottom
            ",
                fontsize=9.5, fontweight="bold")
    ax.set_ylabel("增量价值 / 万元")
    ax.set_title("节点价值的配对分解")
    ax.set_ylim(min(0, min(values)) - 12, max(values) * 1.24)
    ax.legend(fontsize=8.5)
    panel_label(ax, "(b)")
    ax = fig.add_subplot(gs[1, 0])
    by_season = []
    for key in Q3_ORDER:
        frame = pd.read_csv(VARIANTS / "q3_additive" / key / "by_season.csv", encoding="utf
            -8-sig")
        frame = frame.set_index("season").reindex(["spring", "summer", "autumn", "winter"])
        by_season.append(frame)
    seasons = ["spring", "summer", "autumn", "winter"]
    names = ["春季", "夏季", "秋季", "冬季"]
    matrix = np.array([[f.loc[s, "total_cost_yuan"] / 1e4 for s in seasons] for f in
        by_season])
    im = ax.imshow(matrix, cmap="YlGnBu", aspect="auto")
    ax.set_xticks(range(4))
    ax.set_xticklabels(names)
    ax.set_yticks(range(4))
    ax.set_yticklabels([Q3_LABEL[k].replace("\n", " ") for k in Q3_ORDER], fontsize=9)
    for i in range(4):
        for j in range(4):
            ax.text(j, i, f"{matrix[i, j]:.0f}", ha="center", va="center", fontsize=9.5,
```

```
                    color="white" if matrix[i, j] > matrix.mean() else PALETTE["ink"])
    ax.grid(False)
    ax.set_title("季节维度的费用下降（万元）")
    cb = fig.colorbar(im, ax=ax, pad=0.02, shrink=0.92)
    cb.set_label("万元", fontsize=9)
    panel_label(ax, "(c)")
    ax = fig.add_subplot(gs[1, 1])
    up = [summary[k]["up_adjustment_energy_kwh"] / 1e3 for k in Q3_ORDER]
    down = [summary[k]["down_adjustment_energy_kwh"] / 1e3 for k in Q3_ORDER]
    em = [summary[k]["emergency_energy_kwh"] / 1e3 for k in Q3_ORDER]
    x = np.arange(4)
    ax.bar(x - 0.21, up, width=0.4, color=PALETTE["red"], label="向上调整",
           edgecolor=PALETTE["ink"], linewidth=0.7)
    ax.bar(x + 0.21, np.array(down) * -1, width=0.4, color=PALETTE["green"], label="向下调整
        ",
           edgecolor=PALETTE["ink"], linewidth=0.7)
    ax.plot(x, em, color=PALETTE["orange"], marker="s", markersize=6, linewidth=2.0, label="
        紧急购电")
    ax.axhline(0, color=PALETTE["ink"], linewidth=0.9)
    ax.set_xticks(x)
    ax.set_xticklabels([Q3_LABEL[k] for k in Q3_ORDER], fontsize=9)
    ax.set_ylabel("电量 / MWh")
    ax.set_title("调整电量与紧急购电")
    ax.legend(fontsize=9, ncol=1)
    panel_label(ax, "(d)")
    save(fig, "q3_forecast_value")
def fig_q3_settlement() -> None:
    repl = load_group("q3_replacement", "summary")
    addi = load_group("q3_additive", "summary")
    fig, axes = plt.subplots(1, 3, figsize=(13.8, 4.4))
    plt.subplots_adjust(wspace=0.3)
    ax = axes[0]
    x = np.arange(4)
    r = [repl[k]["total_cost_yuan"] / 1e4 for k in Q3_ORDER]
    a = [addi[k]["total_cost_yuan"] / 1e4 for k in Q3_ORDER]
    ax.bar(x - 0.2, r, width=0.4, color=PALETTE["navy"], label="replacement 口径",
           edgecolor=PALETTE["ink"], linewidth=0.8)
    ax.bar(x + 0.2, a, width=0.4, color=PALETTE["gold"], label="additive 口径",
           edgecolor=PALETTE["ink"], linewidth=0.8)
    ax.set_xticks(x)
    ax.set_xticklabels([Q3_LABEL[k] for k in Q3_ORDER], fontsize=9)
    ax.set_ylabel("全年总费用 / 万元")
    ax.set_title("两种结算口径分别重优化")
    ax.set_ylim(0, max(a) * 1.16)
    ax.legend(fontsize=8.5)
    panel_label(ax, "(a)")
    ax = axes[1]
```

```
    gap = [a[i] - r[i] for i in range(4)]
    bars = ax.bar(x, gap, width=0.55, color=PALETTE["rose"], edgecolor=PALETTE["ink"],
        linewidth=0.8)
    _bar_labels(ax, bars, gap, "{:+.2f}")
    ax.set_xticks(x)
    ax.set_xticklabels([Q3_LABEL[k] for k in Q3_ORDER], fontsize=9)
    ax.set_ylabel("additive - replacement / 万元")
    ax.set_title("口径差异（同一优化问题的两种目标）")
    ax.set_ylim(0, max(gap) * 1.25)
    panel_label(ax, "(b)")
    ax = axes[2]
    for label, data, color in (("replacement", r, PALETTE["navy"]), ("additive", a, PALETTE[
        "gold"])):
        ax.plot(range(4), data, marker="o", color=color, linewidth=2.0, markersize=6.5,
            label=label)
    ax.set_xticks(range(4))
    ax.set_xticklabels([Q3_LABEL[k] for k in Q3_ORDER], fontsize=9)
    ax.set_ylabel("全年总费用 / 万元")
    ax.set_title("预报时刻价值的口径稳健性")
    ax.legend(fontsize=9)
    panel_label(ax, "(c)")
    save(fig, "q3_settlement")
def fig_q4_price_value() -> None:
    summary = load_group("q4", "summary")
    costs = np.array([summary[k]["total_cost_yuan"] / 1e4 for k in Q4_ORDER])
    fig = plt.figure(figsize=(13.8, 4.6))
    gs = fig.add_gridspec(1, 3, wspace=0.3)
    ax = fig.add_subplot(gs[0, 0])
    colors = [PALETTE["sky"], PALETTE["red"], PALETTE["teal"], PALETTE["purple"]]
    bars = ax.bar(range(4), costs, color=colors, edgecolor=PALETTE["ink"], linewidth=0.8,
        width=0.6)
    ax.set_xticks(range(4))
    ax.set_xticklabels([Q4_LABEL[k] for k in Q4_ORDER], fontsize=8.5)
    ax.set_ylabel("全年总费用 / 万元")
    ax.set_title("波动电价下的四组结果")
    ax.set_ylim(0, costs.max() * 1.16)
    _bar_labels(ax, bars, costs, "{:.1f}")
    panel_label(ax, "(a)")
    ax = fig.add_subplot(gs[0, 1])
    value_q2 = costs[1] - costs[0]
    value_q3 = costs[3] - costs[2]
    value_roll_perfect = costs[0] - costs[2]
    value_roll_causal = costs[1] - costs[3]
    labels = ["Q4-2\n发布信息差额", "Q4-3\n发布信息差额", "附件4直用下\n滚动调整价值", "因果
        扩展下\n滚动调整价值"]
    values = [value_q2, value_q3, value_roll_perfect, value_roll_causal]
    bars = ax.bar(range(4), values,
```

```
                  color=[PALETTE["red"], PALETTE["purple"], PALETTE["teal"], PALETTE["green"]],
                  edgecolor=PALETTE["ink"], linewidth=0.8, width=0.6)
    _bar_labels(ax, bars, values, "{:+.1f}")
    ax.set_xticks(range(4))
    ax.set_xticklabels(labels, fontsize=8.5)
    ax.set_ylabel("费用差额 / 万元")
    ax.set_title("价格信息与滚动调整的价值")
    ax.set_ylim(0, max(values) * 1.28)
    panel_label(ax, "(b)")
    ax = fig.add_subplot(gs[0, 2])
    frame = pd.read_csv(VARIANTS / "q4" / "q43_causal_price" / "price_forecast_log.csv", encoding="utf-8-sig")
    frame["hour"] = frame["decision_time"].str.slice(0, 2).astype(int)
    grouped = frame.groupby("hour")["price_mae"].mean()
    hours = grouped.index.values
    ax.bar(hours, grouped.values, color=PALETTE["purple"], edgecolor=PALETTE["ink"], linewidth=0.7, width=0.6)
    ax.set_xticks(hours)
    ax.set_xticklabels([f"{h}" for h in hours])
    ax.set_xlabel("决策时刻 / h")
    ax.set_ylabel("电价预测 MAE /（元·kWh$^{-1}$）")
    ax.set_title("因果电价预测的分时精度")
    panel_label(ax, "(c)")
    save(fig, "q4_price_value")
def fig_sensitivity() -> None:
    sens = load_group("sensitivity", "summary")
    terminal = sens["terminal"]
    efficiency = sens["efficiency"]
    fig = plt.figure(figsize=(13.8, 4.6))
    gs = fig.add_gridspec(1, 3, wspace=0.3)
    ax = fig.add_subplot(gs[0, 0])
    policies = ["daily_cycle", "free"]
    names = ["daily_cycle\n(计划 S$_1$$_4$$_4$=S$_0$)", "free\n(无终端约束)"]
    totals = [terminal[p]["total_cost_yuan"] / 1e4 for p in policies]
    bars = ax.bar(range(2), totals, color=[PALETTE["navy"], PALETTE["gold"]],
                  edgecolor=PALETTE["ink"], linewidth=0.8, width=0.5)
    _bar_labels(ax, bars, totals, "{:.1f}")
    ax.set_xticks(range(2))
    ax.set_xticklabels(names, fontsize=9)
    ax.set_ylabel("全年总费用 / 万元")
    ax.set_title("终端储电量策略敏感性")
    ax.set_ylim(0, max(totals) * 1.16)
    panel_label(ax, "(a)")
    ax = fig.add_subplot(gs[0, 1])
    keys = list(efficiency)
    short = ["E1\n$\\eta_c=\\eta_d=0.90$", "E2\n$\\eta_c=\\eta_d=\\sqrt{0.9}$"]
```

```
    totals = [efficiency[k]["total_cost_yuan"] / 1e4 for k in keys]
    throughput = [efficiency[k]["storage_throughput_kwh"] / 1e3 for k in keys]
    bars = ax.bar(range(len(keys)), totals, color=[PALETTE["teal"], PALETTE["rose"]],
                  edgecolor=PALETTE["ink"], linewidth=0.8, width=0.5)
    _bar_labels(ax, bars, totals, "{:.1f}")
    for i, value in enumerate(throughput):
        ax.text(i, totals[i] * 0.5, f"吞吐\n{value:.0f} MWh", ha="center", va="center",
                fontsize=9, color="white", fontweight="bold")
    ax.set_xticks(range(len(keys)))
    ax.set_xticklabels(short, fontsize=9)
    ax.set_ylabel("全年总费用 / 万元")
    ax.set_title("储能效率口径敏感性")
    ax.set_ylim(0, max(totals) * 1.16)
    panel_label(ax, "(b)")
    ax = fig.add_subplot(gs[0, 2])
    data = [terminal[p]["daily_boundary_soc"]["mean"] for p in policies]
    x = np.arange(2)
    ax.bar(x, data, color=[PALETTE["navy"], PALETTE["gold"]], edgecolor=PALETTE["ink"],
           linewidth=0.8, width=0.5, label="日均日末储电量")
    for i, p in enumerate(policies):
        info = terminal[p]["daily_boundary_soc"]
        ax.errorbar(i, info["mean"], yerr=[[info["mean"] - info["min"]], [info["max"] - info
            ["mean"]]],
                    fmt="none", ecolor=PALETTE["ink"], elinewidth=1.4, capsize=7)
        ax.text(i, info["max"] + 200, f"std={info['std']:.0f}", ha="center", fontsize=9)
    ax.set_xticks(x)
    ax.set_xticklabels(names, fontsize=9)
    ax.set_ylabel("日末储电量 / kWh")
    ax.set_title("日边界储电量的分布")
    ax.set_ylim(0, 12500)
    panel_label(ax, "(c)")
    save(fig, "sensitivity")
def fig_dispatch_overview(day_label: str = "2025-06-21", scenario: str = "q3_additive/
    S3_0_6_12_18") -> None:
    path = VARIANTS / scenario / "solution.csv"
    frame = pd.read_csv(path, encoding="utf-8-sig")
    frame = frame[frame["date"] == day_label].reset_index(drop=True)
    if frame.empty:
        raise ValueError(f"{day_label} not found in {path}")
    hours = (frame["slot"].to_numpy()) / 6.0
    fig = plt.figure(figsize=(13.8, 8.4))
    gs = fig.add_gridspec(2, 2, hspace=0.42, wspace=0.24)
    ax = fig.add_subplot(gs[0, 0])
    ax.fill_between(hours, 0, frame["load_kw"] / 1e3, color=PALETTE["sky"], alpha=0.55,
        label="小区负载")
    ax.plot(hours, frame["actual_pv_kw"] / 1e3, color=PALETTE["gold"], linewidth=1.9, label=
        "实际光伏")
```

```
    ax.plot(hours, frame["discharge_actual_kwh"] * 6 / 1e3, color=PALETTE["teal"], linewidth=1.5, label="储能放电功率")
    ax.plot(hours, -frame["charge_actual_kwh"] * 6 / 1e3, color=PALETTE["purple"], linewidth=1.5, label="储能充电功率")
    ax.axhline(0, color=PALETTE["ink"], linewidth=0.8)
    ax.set_xlabel("时刻 / h")
    ax.set_ylabel("功率 / MW")
    ax.set_title(f"{day_label} 功率平衡与储能充放电")
    ax.set_xlim(0, 24)
    ax.legend(fontsize=8.5, ncol=2)
    panel_label(ax, "(a)")
    ax = fig.add_subplot(gs[0, 1])
    ax.plot(hours, frame["baseline_grid_kwh"] * 6 / 1e3, color=PALETTE["gray"], linewidth=1.6,
            linestyle="--", label="0:00 基准计划 $G^{p}$")
    ax.plot(hours, frame["adjusted_grid_kwh"] * 6 / 1e3, color=PALETTE["navy"], linewidth=1.9,
            label="最终调整购电 $G^{a}$")
    ax.fill_between(hours, frame["baseline_grid_kwh"] * 6 / 1e3, frame["adjusted_grid_kwh"] * 6 / 1e3,
                    where=(frame["adjusted_grid_kwh"] > frame["baseline_grid_kwh"]),
                    color=PALETTE["red"], alpha=0.28, label="向上调整")
    ax.fill_between(hours, frame["baseline_grid_kwh"] * 6 / 1e3, frame["adjusted_grid_kwh"] * 6 / 1e3,
                    where=(frame["adjusted_grid_kwh"] <= frame["baseline_grid_kwh"]),
                    color=PALETTE["green"], alpha=0.28, label="向下调整")
    ax.set_xlabel("时刻 / h")
    ax.set_ylabel("购电功率 / MW")
    ax.set_title("计划与调整购电策略")
    ax.set_xlim(0, 24)
    ax.legend(fontsize=8.5)
    panel_label(ax, "(b)")
    ax = fig.add_subplot(gs[1, 0])
    ax.plot(hours, frame["soc_actual_kwh"] / 1e3, color=PALETTE["navy"], linewidth=2.0)
    ax.fill_between(hours, frame["soc_actual_kwh"] / 1e3, 1.2, color=PALETTE["sky"], alpha=0.28)
    ax.axhline(1.2, color=PALETTE["red"], linestyle="--", linewidth=1.1)
    ax.axhline(10.8, color=PALETTE["red"], linestyle="--", linewidth=1.1)
    ax.text(0.2, 1.35, "SOC 下限 1200 kWh", fontsize=8.5, color=PALETTE["red"])
    ax.text(0.2, 10.95, "SOC 上限 10800 kWh", fontsize=8.5, color=PALETTE["red"])
    ax.set_xlabel("时刻 / h")
    ax.set_ylabel("储电量 / MWh")
    ax.set_title("储能荷电状态的日内轨迹")
    ax.set_xlim(0, 24)
    ax.set_ylim(0.5, 12.2)
    panel_label(ax, "(c)")
    ax = fig.add_subplot(gs[1, 1])
```

```
    ax.bar(hours, frame["emergency_kwh"] * 6, width=0.13, color=PALETTE["red"],
           edgecolor=PALETTE["ink"], linewidth=0.5, label="紧急购电")
    ax.bar(hours, frame["curtail_actual_kwh"] * 6, width=0.13, color=PALETTE["olive"],
           edgecolor=PALETTE["ink"], linewidth=0.5, label="弃光")
    ax.set_xlabel("时刻 / h")
    ax.set_ylabel("功率 / kW")
    ax.set_title("紧急购电与弃光的功率分布")
    ax.set_xlim(0, 24)
    ax.legend(fontsize=9)
    panel_label(ax, "(d)")
    save(fig, f"dispatch_{day_label.replace('-', '')}")
def main() -> None:
    fig_q2_attribution()
    fig_q3_forecast_value()
    fig_q4_price_value()
    fig_sensitivity()
    for day in ("2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"):
        try:
            fig_dispatch_overview(day)
        except ValueError as exc:
            print("[warn]", exc)
    print("[fig] 结果层配图完成 →", FIGS)
if __name__ == "__main__":
    main()
```

B.23 技术路线图生成（fig_route.py）

Listing 23:

```
from __future__ import annotations
import sys
from pathlib import Path
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch
ROOT = Path(__file__).resolve().parents[1]
FIGS = ROOT / "paper" / "figures"
INK = "#1A2028"
NAVY = "#1F3B73"
BANDS = [
    ("数据层", "#E8F0FA", "#3E6FB0"),
    ("统一内核", "#E6F4F1", "#2E8B84"),
    ("四问模型", "#FBF0DE", "#D8A22B"),
    ("求解算法", "#EDE9F6", "#7A5BA6"),
    ("验证交付", "#EAF3E8", "#5FA05A"),
```

```
]
CONTENT = [
    ["附件 1\n典型日电价•负载•光伏", "附件 2\n全年 144 时段负载与光伏实际值",
     "附件 3\n每日 4 次未来 24 h 光伏预报", "附件 4\n全年 144 时段实时电价"],
    ["母线能量平衡\n式 (1)", "储能 SOC 状态转移\n$S_t=S_{t-1}+\\eta_c C_t-H_t/\\eta_d$\n式
        (2) 核心状态方程",
     "容量与功率限值\n式 (3) (4)", "购电非负与弃光上界\n式 (5) (6)"],
    ["问题一\n确定性日前 LP\n首末 SOC 闭环",
     "问题二\n因果预测•报童定价\n$5$ 倍电价 $\\Rightarrow$ $0.2$ 分位点\n实时储能反馈",
     "问题三\n四时刻滚动优化\n两种结算口径\n分别重优化",
     "问题四\n波动电价重算\nperfect / causal\n价格信息边界"],
    ["HiGHS 线性规划\n原始—对偶容差 $10^{-9}$", "三层词典序目标\n费用 → 吞吐量 → 弃光",
     "滚动在线选型\n仅用已发生误差", "因果偏差校正\n分发布节点分位数"],
    ["物理残差校验\n平衡/SOC 残差 $<10^{-12}$", "未来信息泄漏\nmutation 测试",
     "结算口径单元测试\n与 legacy 回归", "官方 result1–4\nExcel 回填与回读"],
]
def main() -> None:
    plt.rcParams.update({
        "font.family": "sans-serif",
        "font.sans-serif": ["SimSun", "Times New Roman", "DejaVu Sans"],
        "axes.unicode_minus": False,
        "mathtext.fontset": "dejavusans",
    })
    band_h = 1.0
    gap = 0.34
    n_bands = len(BANDS)
    fig_h = n_bands * band_h + (n_bands - 1) * gap + 0.5
    fig, ax = plt.subplots(figsize=(13.2, fig_h * 1.35))
    ax.set_xlim(0, 13.2)
    ax.set_ylim(0, fig_h)
    ax.axis("off")
    left = 1.42
    box_w = 2.79
    box_gap = 0.155
    for b, (band_name, fill, edge) in enumerate(BANDS):
        y = fig_h - 0.25 - (b + 1) * band_h - b * gap
        ax.add_patch(FancyBboxPatch(
            (0.10, y), 13.00, band_h,
            boxstyle="round,pad=0.008,rounding_size=0.10",
            facecolor=fill, edgecolor=edge, linewidth=1.5, alpha=0.55, zorder=1,
        ))
        ax.text(0.72, y + band_h / 2, band_name, ha="center", va="center",
                fontsize=11.0, fontweight="bold", color=edge, rotation=90, zorder=3)
        for j, text in enumerate(CONTENT[b]):
            x = left + j * (box_w + box_gap)
            emphasized = b == 1 and j == 1
            ax.add_patch(FancyBboxPatch(
```

```
                (x, y + 0.09), box_w, band_h - 0.18,
                boxstyle="round,pad=0.01,rounding_size=0.09",
                facecolor="white", edgecolor=edge,
                linewidth=2.0 if emphasized else 1.1, zorder=2,
            ))
            ax.text(x + box_w / 2, y + band_h / 2, text, ha="center", va="center",
                    fontsize=9.8 if emphasized else 10.2, color=INK, zorder=3, linespacing
                        =1.55)
    for b in range(n_bands - 1):
        y_top = fig_h - 0.25 - (b + 1) * band_h - b * gap
        y_bottom = y_top - gap
        for j in range(4):
            x = left + j * (box_w + box_gap) + box_w / 2
            ax.add_patch(FancyArrowPatch(
                (x, y_top), (x, y_bottom + 0.001),
                arrowstyle="-|>", mutation_scale=13,
                linewidth=1.5, color=NAVY, alpha=0.75, zorder=4,
                shrinkA=0, shrinkB=0,
            ))
    FIGS.mkdir(parents=True, exist_ok=True)
    fig.savefig(FIGS / "technical_route.png", dpi=320, bbox_inches="tight", facecolor="white
        ")
    fig.savefig(FIGS / "technical_route.pdf", bbox_inches="tight", facecolor="white")
    plt.close(fig)
    print("[fig] technical_route 完成 →", FIGS / "technical_route.png")
if __name__ == "__main__":
    main()
```

B.24 绘图总控（plot_results.py）

Listing 24:

```
from __future__ import annotations
import csv,json
from datetime import datetime
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.font_manager import FontProperties
ROOT=Path(__file__).resolve().parents[1]
FONT=Path(r"C:\Windows\Fonts\msyh.ttc")
fp=FontProperties(fname=str(FONT)) if FONT.exists() else None
plt.rcParams["axes.unicode_minus"]=False
COLORS={"Q2":"#557A95","Q3":"#1B998B","Q4-2":"#D9A441","Q4-3":"#C8553D"}
def j(path): return json.loads(path.read_text(encoding="utf-8"))
def csvrows(path):
```

```
    with path.open(encoding="utf-8-sig",newline="") as f: return list(csv.DictReader(f))
def label(ax,x,y,fmt="{:.2f}"):
    for xi,yi in zip(x,y): ax.text(xi,yi,fmt.format(yi),ha="center",va="bottom",fontsize=9,fontproperties=fp)
def save(fig,path):
    path.parent.mkdir(parents=True,exist_ok=True); fig.tight_layout(); fig.savefig(path,bbox_inches="tight"); plt.close(fig)
def main():
    rows=csvrows(ROOT/"outputs"/"q1"/"solution.csv"); x=np.arange(144)/6
    load=np.array([float(r["load_kw"]) for r in rows]); pv=np.array([float(r["pv_kw"]) for r in rows])
    grid=np.array([float(r["grid_kwh"]) for r in rows])*6
    ch=np.array([float(r["charge_kwh"]) for r in rows])*6; dis=np.array([float(r["discharge_kwh"]) for r in rows])*6
    soc=np.array([float(r["soc_kwh"]) for r in rows]); price=np.array([float(r["price"]) for r in rows])
    fig,(ax1,ax2)=plt.subplots(2,1,figsize=(11,7),sharex=True)
    ax1.plot(x,load,label="负载",lw=1.8,color="#263238"); ax1.plot(x,pv,label="光伏",lw=1.8,color="#E6A700")
    ax1.plot(x,grid,label="外网购电",lw=1.6,color="#386FA4")
    ax1.fill_between(x,0,ch,alpha=.25,color="#1B998B",label="充电功率")
    ax1.fill_between(x,0,-dis,alpha=.25,color="#C8553D",label="放电功率")
    ax1.set_ylabel("功率 / kW",fontproperties=fp); ax1.grid(alpha=.2); ax1.legend(ncol=5,prop=fp,loc="upper center")
    ax2.plot(x,soc,color="#6A4C93",lw=2,label="SOC"); ax2.set_ylabel("储电量 / kWh",fontproperties=fp)
    ax2.set_xlabel("时刻 / h",fontproperties=fp); ax2.grid(alpha=.2)
    axp=ax2.twinx(); axp.step(x,price,where="post",color="#8D6E63",alpha=.75,label="电价")
    axp.set_ylabel("电价 / 元/kWh",fontproperties=fp)
    fig.suptitle("问题1：典型日购电—储能联合调度",fontproperties=fp,fontsize=15)
    save(fig,ROOT/"figures"/"q1"/"q1_dispatch.pdf")
    metrics={name:j(ROOT/"outputs"/folder/"metrics.json") for name,folder in
             [("Q2","q2"),("Q3","q3"),("Q4-2","q4-2"),("Q4-3","q4-3")]}
    names=list(metrics); xpos=np.arange(4)
    costs=np.array([metrics[n]["total_cost_yuan"]/1e4 for n in names])
    fig,ax=plt.subplots(figsize=(8,5)); bars=ax.bar(xpos,costs,color=[COLORS[n] for n in names],width=.62)
    ax.set_xticks(xpos,names); ax.set_ylabel("全年总费用 / 万元",fontproperties=fp); ax.grid(axis="y",alpha=.2)
    ax.set_title("固定与波动电价下的全年费用对比",fontproperties=fp,fontsize=14); label(ax,xpos,costs)
    save(fig,ROOT/"figures"/"comparison"/"annual_cost_comparison.pdf")
    emergency=np.array([metrics[n]["emergency_energy_kwh"]/1e4 for n in names])
    fig,ax=plt.subplots(figsize=(8,5)); ax.bar(xpos,emergency,color=[COLORS[n] for n in names],width=.62)
    ax.set_xticks(xpos,names); ax.set_ylabel("紧急购电量 / 万kWh",fontproperties=fp); ax.grid(axis="y",alpha=.2)
```

```
    ax.set_title("滚动预报对 5 倍紧急购电的抑制",fontproperties=fp,fontsize=14); label(ax,xpos,emergency,fmt="{:.3f}")
    save(fig,ROOT/"figures"/"comparison"/"emergency_energy_comparison.pdf")
    variants=[("原始阶梯","q3/variants/raw_step"),("校正阶梯","q3/variants/calibrated_step"),("校正线性","q3")]
    vm=[j(ROOT/"outputs"/p/"metrics.json") for _,p in variants]
    vcost=np.array([m["total_cost_yuan"]/1e4 for m in vm]); vem=np.array([m["emergency_energy_kwh"]/1e4 for m in vm])
    fig,ax=plt.subplots(figsize=(9,5.5)); xx=np.arange(3); width=.34
    b1=ax.bar(xx-width/2,vcost,width,color="#557A95",label="总费用（万元）")
    ax.set_ylabel("总费用 / 万元",fontproperties=fp); ax.set_xticks(xx,[v[0] for v in variants],fontproperties=fp)
    ax2=ax.twinx(); b2=ax2.bar(xx+width/2,vem,width,color="#C8553D",label="紧急购电（万kWh）")
    ax2.set_ylabel("紧急购电量 / 万kWh",fontproperties=fp); ax.grid(axis="y",alpha=.2)
    ax.set_title("Q3 策略变更的完整对照",fontproperties=fp,fontsize=14)
    ax.bar_label(b1,fmt="%.2f",padding=3,fontsize=8)
    ax2.bar_label(b2,fmt="%.3f",padding=3,fontsize=8)
    ax.legend(handles=[b1,b2],labels=["总费用","紧急购电"],prop=fp,loc="upper right")
    save(fig,ROOT/"figures"/"comparison"/"q3_variant_comparison.pdf")
    fig,ax=plt.subplots(figsize=(11,5.5))
    for name,folder in [("Q2","q2"),("Q3","q3"),("Q4-2","q4-2"),("Q4-3","q4-3")]:
        rows=csvrows(ROOT/"outputs"/folder/"daily_metrics.csv")
        dates=np.array([datetime.fromisoformat(r["date"]) for r in rows])
        daily=np.array([float(r["total_cost"]) for r in rows])
        smooth=np.convolve(daily,np.ones(7)/7,mode="same")
        ax.plot(dates[3:-3],smooth[3:-3]/1e4,label=name,color=COLORS[name],lw=1.5)
    ax.set_ylabel("7日移动平均费用 / 万元/日",fontproperties=fp); ax.grid(alpha=.2)
    ax.set_title("全年日费用变化（7日移动平均）",fontproperties=fp,fontsize=14); ax.legend(prop=fp,ncol=4)
    save(fig,ROOT/"figures"/"comparison"/"daily_cost_timeseries.pdf")
    print("generated 5 PDF figures")
if __name__=="__main__": main()
```

B.25 mutation 测试辅助函数（mutate.py）

Listing 25:

```
from __future__ import annotations
import shutil
from pathlib import Path
import openpyxl
from ..solve.io_data import DEFAULT_DATA_ROOT
ATTACHMENTS = ("附件1.xlsx", "附件2.xlsx", "附件3.xlsx", "附件4.xlsx")
def make_root(tmp: Path, source: Path = DEFAULT_DATA_ROOT) -> Path:
    tmp.mkdir(parents=True, exist_ok=True)
```

```
    for name in ATTACHMENTS:
        target = tmp / name
        if not target.exists():
            shutil.copy2(source / name, target)
    return tmp
def _mutate_sheet(path: Path, sheet: str, day_index: int, from_slot: int, factor: float) ->
    None:
    workbook = openpyxl.load_workbook(path)
    worksheet = workbook[sheet] if sheet in workbook.sheetnames else workbook.worksheets[0]
    row = day_index + 2
    for col in range(from_slot + 2, worksheet.max_column + 1):
        cell = worksheet.cell(row, col)
        if cell.value is None:
            continue
        cell.value = float(cell.value) * factor
    workbook.save(path)
    workbook.close()
def mutate_attachment2_load(root: Path, day_index: int, from_slot: int = 0, factor: float =
    10.0) -> None:
    _mutate_sheet(root / "附件2.xlsx", "小区负载", day_index, from_slot, factor)
def mutate_attachment2_pv(root: Path, day_index: int, from_slot: int = 0, factor: float =
    10.0) -> None:
    _mutate_sheet(root / "附件2.xlsx", "光伏发电实际功率", day_index, from_slot, factor)
def mutate_attachment4_price(root: Path, day_index: int, from_slot: int = 0, factor: float =
     10.0) -> None:
    _mutate_sheet(root / "附件4.xlsx", "Sheet1", day_index, from_slot, factor)
def max_abs_diff(left, right) -> float:
    import numpy as np
    return float(np.max(np.abs(np.asarray(left, dtype=float) - np.asarray(right, dtype=float
        ))))
```

B.26 未来信息泄漏 mutation 测试（test_causality.py）

Listing 26:

```
from __future__ import annotations
import shutil
import tempfile
import unittest
from pathlib import Path
import numpy as np
from ..solve.io_data import read_attachment4
from ..solve.q2 import run_q2
from ..solve.q3 import run_q3
from .mutate import (
    make_root,
```

```
    mutate_attachment2_load,
    mutate_attachment2_pv,
    mutate_attachment4_price,
)
TARGET_DAY = 35
DAYS = 36
TOL = 1e-10
class CausalityTestBase(unittest.TestCase):
    @classmethod
    def setUpClass(cls) -> None:
        cls._tmpdir = tempfile.mkdtemp(prefix="mathmodel_causal_")
        cls.tmp = Path(cls._tmpdir)
        cls.base = make_root(cls.tmp / "base")
        cls.mut_load = make_root(cls.tmp / "mut_load")
        mutate_attachment2_load(cls.mut_load, TARGET_DAY, 0, 10.0)
        cls.mut_pv = make_root(cls.tmp / "mut_pv")
        mutate_attachment2_pv(cls.mut_pv, TARGET_DAY, 0, 10.0)
        cls.mut_both_after_6 = make_root(cls.tmp / "mut_both_after6")
        mutate_attachment2_load(cls.mut_both_after_6, TARGET_DAY, 36, 10.0)
        mutate_attachment2_pv(cls.mut_both_after_6, TARGET_DAY, 36, 10.0)
        cls.mut_price_after_6 = make_root(cls.tmp / "mut_price_after6")
        mutate_attachment4_price(cls.mut_price_after_6, TARGET_DAY, 36, 10.0)
    @classmethod
    def tearDownClass(cls) -> None:
        shutil.rmtree(cls._tmpdir, ignore_errors=True)
    @staticmethod
    def _run_q2(root: Path, **kwargs):
        return run_q2(data_root=root, max_days=DAYS, write_outputs=False, load_mode="causal"
            , **kwargs)
    @staticmethod
    def _run_q3(root: Path, **kwargs):
        kwargs.setdefault("settlement_mode", "replacement")
        return run_q3(data_root=root, max_days=DAYS, write_outputs=False, load_mode="causal"
            , **kwargs)
class Q2LoadCausalityTest(CausalityTestBase):
    def test_future_load_cannot_change_plan(self):
        base = self._run_q2(self.base)
        mutated = self._run_q2(self.mut_load)
        b = base["records"][TARGET_DAY]["load_plan_kw"]
        m = mutated["records"][TARGET_DAY]["load_plan_kw"]
        self.assertLess(float(np.max(np.abs(b - m))), TOL, "第 d 天真实负载被篡改后 0:00 计
            划输入发生变化")
    def test_future_load_changes_execution_only(self):
        base = self._run_q2(self.base)
        mutated = self._run_q2(self.mut_load)
        b = base["records"][TARGET_DAY]["execution"].emergency.sum()
        m = mutated["records"][TARGET_DAY]["execution"].emergency.sum()
```

```
        self.assertNotAlmostEqual(b, m, places=3)
class Q2PVCausalityTest(CausalityTestBase):
    def test_future_pv_cannot_change_plan(self):
        base = self._run_q2(self.base)
        mutated = self._run_q2(self.mut_pv)
        b = base["records"][TARGET_DAY]["pv_plan_kw"]
        m = mutated["records"][TARGET_DAY]["pv_plan_kw"]
        self.assertLess(float(np.max(np.abs(b - m))), TOL, "第 d 天真实光伏被篡改后 0:00 计
            划输入发生变化")
class Q3NodeCausalityTest(CausalityTestBase):
    def test_node_inputs_do_not_see_future_load_or_pv(self):
        base = self._run_q3(self.base)
        mutated = self._run_q3(self.mut_both_after_6)
        for day in (TARGET_DAY,):
            b_load = base["records"][day]["load_plan_kw"][36:72]
            m_load = mutated["records"][day]["load_plan_kw"][36:72]
            self.assertLess(float(np.max(np.abs(b_load - m_load))), TOL, "6:00 负载预测输入
                受到未来真实负载影响")
            b_block = base["records"][day]["adjusted_grid"][36:72]
            m_block = mutated["records"][day]["adjusted_grid"][36:72]
            self.assertLess(float(np.max(np.abs(b_block - m_block))), TOL, "6:00 优化结果受
                到未来真实负载/光伏影响")
            b_base = base["records"][day]["baseline_plan"].grid
            m_base = mutated["records"][day]["baseline_plan"].grid
            self.assertLess(float(np.max(np.abs(b_base - m_base))), TOL, "0:00 基准计划受到
                未来真实负载/光伏影响")
class Q4PriceCausalityTest(CausalityTestBase):
    def test_causal_price_ignores_future_realized_price(self):
        prices = read_attachment4(self.base).price
        base = self._run_q3(self.base, price_matrix=prices, price_mode="causal")
        mutated = self._run_q3(self.mut_price_after_6, price_matrix=prices, price_mode="
            causal")
        b = base["records"][TARGET_DAY]["adjusted_grid"][36:72]
        m = mutated["records"][TARGET_DAY]["adjusted_grid"][36:72]
        self.assertLess(float(np.max(np.abs(b - m))), TOL, "causal 电价模式下提前看到未来真
            实电价")
    def test_perfect_price_does_see_future_realized_price(self):
        base = self._run_q3(self.base, price_matrix=read_attachment4(self.base).price,
            price_mode="perfect")
        mutated = self._run_q3(
            self.mut_price_after_6,
            price_matrix=read_attachment4(self.mut_price_after_6).price,
            price_mode="perfect",
        )
        b = base["records"][TARGET_DAY]["adjusted_grid"][36:72]
        m = mutated["records"][TARGET_DAY]["adjusted_grid"][36:72]
        self.assertGreater(float(np.max(np.abs(b - m))), 1e-6)
```

```
if __name__ == "__main__":
    unittest.main()
```

B.27 主结果与计划持续性测试（test_q2_q3.py）

Listing 27:

```
import unittest
import numpy as np
from ..solve.io_data import read_attachment1,read_attachment2,read_attachment3,
    read_attachment4,hourly_to_ten_min
from ..solve.forecast import OnlinePVForecaster
from ..solve.lp_core import solve_dispatch,solve_adjustment_dispatch,SOC_MIN,SOC_MAX
from ..solve.q3 import run_q3
from ..solve.settle import execute_with_realtime_storage_feedback
class Q2Q3Tests(unittest.TestCase):
    def test_attachment_dimensions(self):
        data=read_attachment2(); forecast=read_attachment3(); prices=read_attachment4()
        self.assertEqual(data.load_kw.shape,(365,144)); self.assertEqual(data.pv_kw.shape
            ,(365,144))
        self.assertEqual(forecast.hourly_kw.shape,(365,4,24)); self.assertEqual(prices.price
            .shape,(365,144))
        step=hourly_to_ten_min(forecast.hourly_kw[0,0],"step")
        self.assertAlmostEqual(float(step.sum()/6),float(forecast.hourly_kw[0,0].sum()),
            places=9)
    def test_forecast_is_sequential(self):
        f=OnlinePVForecaster(np.ones(144)); first=f.predict(); self.assertEqual(first.
            history_days_used,0)
        actual=np.r_[np.zeros(40),np.ones(60)*100,np.zeros(44)]; f.observe(first,actual)
        second=f.predict(); self.assertEqual(second.history_days_used,1)
        self.assertTrue(np.allclose(second.bases["persistence"],actual))
        with self.assertRaises(ValueError): f.observe(first,actual)
    def test_realtime_controller_constraints(self):
        day=read_attachment1(); plan=solve_dispatch(day.load_kw,np.zeros(144),day.price,
            soc_initial=6000,soc_terminal=6000)
        actual=np.maximum(day.pv_kw*1.3,0); execution=execute_with_realtime_storage_feedback
            (plan,day.load_kw,actual,soc_initial=6000)
        balance=execution.grid+actual/6+execution.discharge+execution.emergency-day.load_kw
            /6-execution.charge-execution.curtail
        self.assertLess(float(np.max(np.abs(balance))),1e-7)
        self.assertGreaterEqual(float(execution.soc.min()),SOC_MIN-1e-7)
        self.assertLessEqual(float(execution.soc.max()),SOC_MAX+1e-7)
        self.assertLess(float(np.max(np.minimum(execution.charge,execution.discharge))),1e
            -7)
    def test_adjustment_linearization(self):
        gp=np.array([900.0,900.0,700.0]); ga=np.array([700.0,900.0,900.0])
```

```
        b=np.maximum(ga-gp,0); linear=0.5*gp+0.5*ga+b
        piece=np.minimum(gp,ga)+0.5*np.maximum(gp-ga,0)+1.5*np.maximum(ga-gp,0)
        self.assertTrue(np.allclose(linear,[800,900,1000])); self.assertTrue(np.allclose(
            linear,piece))
    def test_adjustment_solver_balance(self):
        day=read_attachment1(); base=solve_dispatch(day.load_kw,day.pv_kw,day.price,
            soc_initial=6000,soc_terminal=6000)
        result=solve_adjustment_dispatch(day.load_kw,day.pv_kw*0.9,day.price,base.grid,np.
            ones(144,dtype=bool),
                                         soc_initial=6000,soc_terminal=6000)
        d=result.dispatch
        residual=d.grid+day.pv_kw*0.9/6+d.discharge-day.load_kw/6-d.charge-d.curtail
        self.assertLess(float(np.max(np.abs(residual))),1e-7)
        self.assertLess(float(np.max(np.abs(result.increase-np.maximum(d.grid-base.grid,0)))
            ),1e-6)
    def test_additive_milp_enforces_storage_exclusivity(self):
        day=read_attachment1(); base=solve_dispatch(day.load_kw,day.pv_kw,day.price,
            soc_initial=6000,soc_terminal=6000)
        result=solve_adjustment_dispatch(day.load_kw,day.pv_kw*0.9,day.price,base.grid,np.
            ones(144,dtype=bool),
                                         soc_initial=6000,soc_terminal=6000,settlement_mode=
                                             "additive")
        self.assertLess(float(np.max(np.minimum(result.dispatch.charge,result.dispatch.
            discharge))),1e-9)
    def test_disabled_nodes_keep_latest_active_plan(self):
        result=run_q3(max_days=2,write_outputs=False,update_nodes=(0,36),settlement_mode="
            replacement")
        record=result["records"][-1]
        self.assertTrue(np.all(record["plan_source_nodes"][:36]==0))
        self.assertTrue(np.all(record["plan_source_nodes"][36:]==36))
        self.assertLess(float(np.max(np.abs(record["adjusted_grid"][72:]-record["revisions"
            ][1,72:]))),1e-9)
if __name__=="__main__": unittest.main()
```

B.28 论文数值与提交口径一致性测试（test_paper_consistency.py）

Listing 28:

```
from __future__ import annotations
import hashlib
import json
from pathlib import Path
ROOT = Path(__file__).resolve().parents[1]
VARIANTS = ROOT / "outputs" / "variants" / "model_audit"
DATES = ("2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21")
def _json(path: Path) -> dict:
```

```
    return json.loads(path.read_text(encoding="utf-8"))
def _sha256(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()
def test_additive_main_cost_decomposition_and_exclusivity() -> None:
    metrics = _json(VARIANTS / "q3_additive" / "S3_0_6_12_18" / "metrics.json")
    components = (
        metrics["baseline_purchase_cost_yuan"]
        + metrics["adjustment_cost_yuan"]
        + metrics["emergency_cost_yuan"]
    )
    assert abs(metrics["total_cost_yuan"] - components) < 1e-6
    assert metrics["max_simultaneous_charge_discharge"] < 1e-5
    assert metrics["passed"] is True
def test_paired_node_decomposition_is_exact() -> None:
    summary = _json(VARIANTS / "q3_additive" / "summary.json")
    chain = (
        ("S0_0", "C1_load6", "S1_0_6"),
        ("S1_0_6", "C2_load12", "S2_0_6_12"),
        ("S2_0_6_12", "C3_load18", "S3_0_6_12_18"),
    )
    for before, control, after in chain:
        total = summary[before]["total_cost_yuan"] - summary[after]["total_cost_yuan"]
        load_part = summary[before]["total_cost_yuan"] - summary[control]["total_cost_yuan"]
        pv_part = summary[control]["total_cost_yuan"] - summary[after]["total_cost_yuan"]
        assert abs(total - load_part - pv_part) < 1e-6
def test_required_date_tables_cover_all_contract_dates() -> None:
    generated = ROOT / "paper" / "generated"
    for name in ("q2_required_tables.tex", "q3_required_tables.tex", "q4_required_tables.tex
        "):
        text = (generated / name).read_text(encoding="utf-8")
        for date in DATES:
            assert date in text
def test_q4_submission_uses_attachment4_direct_results() -> None:
    pairs = (
        ("result4-2.xlsx", VARIANTS / "q4" / "q42_perfect_price" / "result4-2.xlsx"),
        ("result4-3.xlsx", VARIANTS / "q4" / "q43_perfect_price" / "result4-3.xlsx"),
    )
    for name, source in pairs:
        assert _sha256(ROOT / "submit" / name) == _sha256(source)
```

B.29 结算口径单元测试（test_settlement.py）

Listing 29:

```
from __future__ import annotations
import unittest
```

```
import numpy as np
from ..solve.lp_core import settlement_cost, solve_adjustment_dispatch
from ..solve.params import DEFAULT_STORAGE
class ReplacementFormulaTest(unittest.TestCase):
    def test_purchase_below_plan(self):
        value = settlement_cost(np.array([900.0]), np.array([700.0]), np.array([1.0]), "
            replacement")
        self.assertAlmostEqual(value, 800.0, places=9)
    def test_purchase_above_plan(self):
        value = settlement_cost(np.array([700.0]), np.array([900.0]), np.array([1.0]), "
            replacement")
        self.assertAlmostEqual(value, 1000.0, places=9)
    def test_unchanged(self):
        value = settlement_cost(np.array([800.0]), np.array([800.0]), np.array([1.0]), "
            replacement")
        self.assertAlmostEqual(value, 800.0, places=9)
class AdditiveFormulaTest(unittest.TestCase):
    def test_purchase_below_plan(self):
        value = settlement_cost(np.array([900.0]), np.array([700.0]), np.array([1.0]), "
            additive")
        self.assertAlmostEqual(value, 1000.0, places=9)
    def test_purchase_above_plan(self):
        value = settlement_cost(np.array([700.0]), np.array([900.0]), np.array([1.0]), "
            additive")
        self.assertAlmostEqual(value, 1000.0, places=9)
def _case(seed: int, t: int = 24, mask_all: bool = True):
    rng = np.random.default_rng(seed)
    load = 800.0 + 200.0 * rng.random(t)
    pv = 400.0 * rng.random(t)
    price = 0.3 + 0.7 * rng.random(t)
    baseline = np.maximum((load - pv) * (1 / 6), 0.0)
    mask = np.ones(t, dtype=bool) if mask_all else np.r_[np.ones(t - 6, dtype=bool), np.
        zeros(6, dtype=bool)]
    return load, pv, price, baseline, mask
def _model_objective(result, price, baseline, mode: str) -> float:
    mask = result.adjustment_mask
    p = price[mask]
    if mode == "replacement":
        return float(np.dot(p, result.dispatch.grid[mask] + 0.5 * result.increase[mask] +
            0.5 * result.decrease[mask]))
    return float(np.dot(p, baseline[mask]) + np.dot(p, 0.5 * result.decrease[mask] + 1.5 *
        result.increase[mask]))
class LinearizationTest(unittest.TestCase):
    def _gap(self, mode: str, seed: int) -> float:
        load, pv, price, baseline, mask = _case(seed)
        result = solve_adjustment_dispatch(
            load, pv, price, baseline, mask,
```

```
            soc_initial=6000.0, soc_terminal=6000.0, settlement_mode=mode,
        )
        direct = settlement_cost(baseline, result.dispatch.grid, price, mode)
        return abs(direct - _model_objective(result, price, baseline, mode))
    def test_replacement_linearization(self):
        for seed in range(5):
            self.assertLess(self._gap("replacement", seed), 1e-7, f"seed={seed}")
    def test_additive_linearization(self):
        for seed in range(5):
            self.assertLess(self._gap("additive", seed), 1e-7, f"seed={seed}")
    def test_two_modes_are_reoptimized_not_revalued(self):
        for seed in range(5):
            load, pv, price, baseline, mask = _case(seed)
            kwargs = dict(soc_initial=6000.0, soc_terminal=6000.0)
            replacement = solve_adjustment_dispatch(
                load, pv, price, baseline, mask, settlement_mode="replacement", **kwargs
            )
            additive = solve_adjustment_dispatch(
                load, pv, price, baseline, mask, settlement_mode="additive", **kwargs
            )
            best_replacement = settlement_cost(baseline, replacement.dispatch.grid, price, "
                replacement")
            cross_replacement = settlement_cost(baseline, additive.dispatch.grid, price, "
                replacement")
            best_additive = settlement_cost(baseline, additive.dispatch.grid, price, "
                additive")
            cross_additive = settlement_cost(baseline, replacement.dispatch.grid, price, "
                additive")
            tol_r = 1e-6 * max(1.0, abs(best_replacement))
            tol_a = 1e-6 * max(1.0, abs(best_additive))
            self.assertLessEqual(best_replacement, cross_replacement + tol_r, f"seed={seed}"
                )
            self.assertLessEqual(best_additive, cross_additive + tol_a, f"seed={seed}")
class StorageParamTest(unittest.TestCase):
    def test_energy_limit(self):
        self.assertAlmostEqual(DEFAULT_STORAGE.energy_limit, 5000.0 / 6.0, places=9)
if __name__ == "__main__":
    unittest.main()
```

B.30 历史配置回归测试（test_regression.py）

Listing 30:

```
from __future__ import annotations
import json
import unittest
```

```
4  from pathlib import Path
5  from ..solve.io_data import read_attachment4
6  from ..solve.q1 import run_q1
7  from ..solve.q2 import run_q2
8  from ..solve.q3 import run_q3
9  FIXTURE = Path(__file__).with_name("legacy_metrics.json")
10 LEGACY = json.loads(FIXTURE.read_text(encoding="utf-8"))
11 ABS_TOL = 1e-3
12 REL_TOL = 1e-9
13 def _close(test: unittest.TestCase, label: str, new: float, old: float, abs_tol: float =
       ABS_TOL) -> None:
14     tolerance = max(abs_tol, REL_TOL * abs(old))
15     test.assertLess(
16         abs(new - old),
17         tolerance,
18         f"{label} 未复现 legacy 数值: new={new!r} legacy={old!r} diff={new - old!r}",
19     )
20 class LegacyRegressionTest(unittest.TestCase):
21     @classmethod
22     def setUpClass(cls) -> None:
23         cls.prices = read_attachment4().price
24     def test_q1_unchanged(self):
25         metrics = run_q1()["metrics"]
26         _close(self, "Q1 费用", metrics["cost_yuan"], LEGACY["q1"]["cost_yuan"])
27         _close(self, "Q1 购电量", metrics["grid_energy_kwh"], LEGACY["q1"]["grid_energy_kwh"
               ])
28         _close(self, "Q1 充电量", metrics["charge_energy_kwh"], LEGACY["q1"]["
               charge_energy_kwh"])
29     def test_q2_legacy_benchmark(self):
30         metrics = run_q2(
31             max_days=365,
32             write_outputs=False,
33             load_mode="perfect",
34             pv_risk_mode="asymmetric",
35             realtime_feedback=True,
36             price_mode="perfect",
37             terminal_policy="daily_cycle",
38             use_typical_prior=False,
39         )["metrics"]
40         _close(self, "Q2 总费用", metrics["total_cost_yuan"], LEGACY["q2"]["total_cost_yuan"
               ])
41         _close(self, "Q2 计划费用", metrics["plan_purchase_cost_yuan"], LEGACY["q2"]["
               plan_purchase_cost_yuan"])
42         _close(self, "Q2 紧急电量", metrics["emergency_energy_kwh"], LEGACY["q2"]["
               emergency_energy_kwh"])
43         _close(self, "Q2 弃光量", metrics["curtailment_energy_kwh"], LEGACY["q2"]["
               curtailment_energy_kwh"])
```

```
    def test_q3_legacy_benchmark(self):
        metrics = run_q3(
            max_days=365,
            write_outputs=False,
            downscale="linear",
            calibrate=True,
            update_nodes=(0, 36, 72, 108),
            settlement_mode="replacement",
            load_mode="perfect",
            price_mode="perfect",
            realtime_feedback=True,
            terminal_policy="daily_cycle",
            use_typical_prior=False,
        )["metrics"]
        _close(self, "Q3 总费用", metrics["total_cost_yuan"], LEGACY["q3"]["total_cost_yuan"])
        _close(self, "Q3 基准计划费用", metrics["baseline_purchase_cost_yuan"], LEGACY["q3"]["baseline_purchase_cost_yuan"])
        _close(self, "Q3 紧急电量", metrics["emergency_energy_kwh"], LEGACY["q3"]["emergency_energy_kwh"])
        _close(self, "Q3 弃光量", metrics["curtailment_energy_kwh"], LEGACY["q3"]["curtailment_energy_kwh"], abs_tol=1e-2)
    def test_q4_legacy_benchmark(self):
        metrics2 = run_q2(
            max_days=365,
            write_outputs=False,
            price_matrix=self.prices,
            load_mode="perfect",
            pv_risk_mode="asymmetric",
            realtime_feedback=True,
            price_mode="perfect",
            use_typical_prior=False,
        )["metrics"]
        _close(self, "Q4-2 总费用", metrics2["total_cost_yuan"], LEGACY["q4-2"]["total_cost_yuan"])
        metrics3 = run_q3(
            max_days=365,
            write_outputs=False,
            price_matrix=self.prices,
            update_nodes=(0, 36, 72, 108),
            settlement_mode="replacement",
            load_mode="perfect",
            price_mode="perfect",
            realtime_feedback=True,
            use_typical_prior=False,
        )["metrics"]
        _close(self, "Q4-3 总费用", metrics3["total_cost_yuan"], LEGACY["q4-3"]["
```

```
            total_cost_yuan"])
if __name__ == "__main__":
    unittest.main()
```